



DevOps

INTERVIEW MASTERY

Real-World Scenarios & Architecture Insights



DevOps Interview Mastery

*Real-World Scenarios & Architecture
Insights*

PREPARED BY

ARVIND VERMA
DevOps Engineer

AWS

Azure

Kubernetes

Terraform

Jenkins

Docker

Ansible

TABLE OF CONTENTS

SECTION 1: LINUX & SHELL SCRIPTING (Foundation)	5
SECTION 2: JENKINS (50 Questions)	8
SECTION 3: DOCKER (50 Questions)	15
SECTION 4: GIT / GITHUB / GITLAB (50 Questions)	23
SECTION 5: AWS DEVOPS (50 Questions)	32
SECTION 6: KUBERNETES (75 Questions)	40
SECTION 7: PROMETHEUS & GRAFANA (30 Questions)	53
SECTION 8: ELK STACK (30 Questions)	58
SECTION 9: ANSIBLE (30 Questions)	63
SECTION 10: TERRAFORM (40 Questions)	69
SECTION 11: AZURE DEVOPS (25 Questions)	77
SECTION 12: GITHUB ACTIONS (25 Questions)	82
SECTION 13: SECURITY IN DEVOPS / DEVSECOPS (25 Questions)	87
SECTION 14: MONITORING, OBSERVABILITY & SRE (20 Questions)	91
SECTION 15: PRODUCTION PIPELINE — ADDITIONAL TOOLS (20 Questions)	94
SECTION 16: NETWORKING & CLOUD FUNDAMENTALS (20 Questions)	98
SECTION 17: DATABASES IN DEVOPS (10 Questions)	102
FINAL SECTION: SCENARIO-BASED & BEHAVIORAL QUESTIONS	104
CHAPTER 1: JENKINS	108
Introduction to Jenkins	108
CHAPTER 2: DOCKER	122
Introduction to Docker	122
CHAPTER 3: GIT, GITHUB & GITLAB	133
Introduction to Git in DevOps	133
CHAPTER 4: AWS DEVOPS	143
Introduction to AWS DevOps	143
CHAPTER 5: GITHUB ACTIONS	155
Introduction to GitHub Actions	155
CHAPTER 6: KUBERNETES	164
Introduction to Kubernetes	164
CHAPTER 7: PROMETHEUS & GRAFANA	188

Introduction to Observability	182
CHAPTER 8: ELK STACK (ELASTICSEARCH, LOGSTASH, KIBANA)	191
Introduction to Log Management	191
CHAPTER 9: ANSIBLE	198
Introduction to Ansible	198
CHAPTER 10: TERRAFORM	207
Introduction to Terraform	207
CHAPTER 11: AZURE DEVOPS	220
Introduction to Azure DevOps	220
CHAPTER 12: PRODUCTION-GRADE PIPELINE — ADDITIONAL TOOLS	226
Security Scanning, Quality Gates, and Supporting Tools	226
CHAPTER 13: PRACTICAL SCENARIO-BASED QUESTIONS	248
FINAL CHAPTER: KEY INTERVIEW TIPS	256
Key Takeaways — Complete Guide	256
Common Pitfalls Across All Tools	257
Operational Lessons	258
APPENDIX: Common Mistakes to Avoid During DevOps Interviews	259
PART 1: TECHNICAL KNOWLEDGE MISTAKES	259
PART 2: COMMUNICATION & BEHAVIORAL MISTAKES	262
PART 3: PREPARATION MISTAKES	265
PART 4: MINDSET & ATTITUDE MISTAKES	266
PART 5: INTERVIEW-DAY EXECUTION MISTAKES	267
PART 6: EXPERIENCE-LEVEL SPECIFIC MISTAKES	270
QUICK REFERENCE: PRE-INTERVIEW CHECKLIST	271
THE GOLDEN RULES — Summary	272
BONUS: PHRASES THAT HURT VS PHRASES THAT HELP	272
BONUS: TOPICS MOST COMMONLY UNDER-PREPARED	273
FINAL THOUGHT	274

SECTION 1: LINUX & SHELL SCRIPTING

(Foundation)

Q1. What is the difference between a process and a thread?

A process is an independent program with its own memory space. A thread is a lightweight unit of execution within a process, sharing the same memory. Processes are isolated; threads share resources and communicate faster but risk race conditions.

Q2. What does the `chmod 755` command do?

Sets file permissions: owner gets read+write+execute (7), group gets read+execute (5), others get read+execute (5). The $7 = 4+2+1$, $5 = 4+0+1$.

Q3. How do you find all files modified in the last 24 hours?

```
find /var/log -mtime -1 -type f
```

`-mtime -1` means modified less than 1 day ago.

Q4. What is the difference between `>` and `>>` in shell?

`>` overwrites the file. `>>` appends to the file.

Q5. How do you check which process is listening on port 8080?

```
ss -tlnp | grep 8080  
# or  
lsof -i :8080
```

Q6. What is a zombie process?

A process that has finished execution but still has an entry in the process table because its parent hasn't read its exit status via `wait()`. It consumes minimal resources but indicates a bug in the parent process.

Q7. How do you run a command in the background and keep it running after logout?

```
nohup ./my-script.sh &
# or better:
screen -S mysession ./my-script.sh
```

Q8. What does `2>&1` mean in shell?

Redirects stderr (file descriptor 2) to wherever stdout (file descriptor 1) is currently pointing. Used to capture both stdout and stderr together.

Q9. How do you find the top 10 largest files in a directory?

```
du -ah /var/log | sort -rh | head -10
```

Q10. What is the difference between `kill` and `kill -9`?

`kill` sends SIGTERM (15) — a polite request to terminate, allowing cleanup. `kill -9` sends SIGKILL — immediately terminates; cannot be caught or ignored by the process.

Q11. How do you create a cron job that runs every 15 minutes?

```
crontab -e
# Add:
*/15 * * * * /path/to/script.sh
```

Q12. What is `/etc/passwd` and what does it contain?

A file containing user account information: username, UID, GID, home directory, and default shell. Passwords are stored as `x` (pointing to `/etc/shadow`).

Q13. How do you check available disk space?

```
df -h          # Human-readable disk usage
du -sh /var/*  # Size of each subdirectory
```

Q14. What is `grep -r "error" /var/log`?

Recursively searches all files in `/var/log` for the string "error" and prints matching lines with filename.

Q15. How do you check CPU and memory usage in real time?

```
top           # Basic, built-in
htop          # Better UI
vmstat 1      # Per-second CPU/memory stats
```

Prepared by Arvind Verma

SECTION 2: JENKINS (50 Questions)

Q16. What is Jenkins?

An open-source automation server that orchestrates CI/CD pipelines — automatically building, testing, and deploying code on every commit.

Q17. What is a Jenkins Pipeline?

A suite of plugins that supports implementing and integrating CI/CD pipelines as code using a `Jenkinsfile`.

Q18. What is the difference between Declarative and Scripted pipelines?

Declarative has a rigid predefined structure (`pipeline {}` block), easier to read and validate. Scripted uses Groovy's `node {}` block with full programming flexibility but no structural validation.

Q19. What is a Jenkinsfile?

A text file defining the Jenkins pipeline stored in the project's source code repository. Enables pipeline-as-code.

Q20. What is a Jenkins agent?

A machine (physical, VM, or container) that executes build jobs delegated by the Jenkins controller. Agents allow distributed, parallel builds.

Q21. How do you trigger a Jenkins build automatically on a Git push?

Configure a webhook in GitHub/GitLab pointing to `http://jenkins/github-webhook/`. Jenkins receives the POST request and triggers the job.

Q22. What is the purpose of `post {}` in a Declarative pipeline?

Defines actions to run after all stages complete, regardless of the result — e.g., send notifications, clean workspace, archive artifacts. Supports `always`, `success`, `failure`, `unstable`.

Q23. How do you parameterize a Jenkins job?

```
parameters {  
    string(name: 'DEPLOY_ENV', defaultValue: 'staging', description: 'Target  
environment')  
    booleanParam(name: 'SKIP_TESTS', defaultValue: false)  
    choice(name: 'REGION', choices: ['us-east-1', 'ap-south-1'])  
}
```

Q24. What is a shared library in Jenkins?

Reusable Groovy code stored in a separate Git repository that can be imported into any Jenkinsfile. Eliminates duplication across pipelines.

Q25. What is the Blue Ocean plugin?

A redesigned UI for Jenkins that provides a visual representation of pipelines, making it easier to visualize stages, steps, and failures.

Q26. How do you archive artifacts in Jenkins?

```
post {  
    success {  
        archiveArtifacts artifacts: 'target/*.jar', fingerprint: true  
    }  
}
```

Q27. How do you run parallel stages in Jenkins?

```
stage('Parallel Tests') {  
    parallel {  
        stage('Unit Tests') { steps { sh 'mvn test' } }  
        stage('Integration Tests') { steps { sh 'mvn integration-test' } }  
    }  
}
```

```
}
```

Q28. How do you add a manual approval gate in Jenkins?

```
stage('Deploy to Production') {  
    steps {  
        input message: 'Deploy to production?', ok: 'Deploy'  
        sh './deploy-prod.sh'  
    }  
}
```

Q29. What is Jenkins X?

A cloud-native CI/CD platform built on top of Kubernetes using Jenkins as the underlying engine. Automates CI/CD for microservices with built-in GitOps.

Q30. How do you manage Jenkins credentials securely?

Use Jenkins Credentials Store (Manage Jenkins - Credentials). Reference in pipeline via `withCredentials([])` or `credentials()` binding. Never hardcode secrets in Jenkinsfile.

Q31. What is the role of the `when` directive in a Declarative pipeline?

Conditionally executes a stage based on criteria:

```
when {  
    branch 'main'  
    environment name: 'DEPLOY_TO', value: 'production'  
}
```

Q32. What is a multi-branch pipeline?

A Jenkins project type that automatically creates a pipeline for each branch in a repository. Useful for feature branch CI where each branch gets its own build.

Q33. How do you configure Jenkins to use dynamic Kubernetes agents?

Install the Kubernetes plugin. Configure the cloud in **Manage Jenkins** → **Clouds**. Define pod templates in the Jenkinsfile using the **kubernetes {}** agent block.

Q34. What is the **stash and **unstash** directive in Jenkins?**

stash saves files from one stage; **unstash** retrieves them in another stage or on another agent. Used to pass build artifacts between stages running on different agents.

Q35. How do you set build timeouts in Jenkins?

```
options {  
    timeout(time: 30, unit: 'MINUTES')  
    buildDiscarder(logRotator(numToKeepStr: '10'))  
}
```

Q36. What is Jenkins Groovy sandbox?

A security mechanism that restricts the Groovy code in pipelines from executing dangerous operations. Scripts needing privileged access must be approved by an administrator in "Script Approval."

Q37. How do you send email notifications from Jenkins?

```
post {  
    failure {  
        email {  
            to: 'team@company.com',  
            subject: "FAILED: ${env.JOB_NAME}",  
            body: "Build URL: ${env.BUILD_URL}"  
        }  
    }  
}
```

Q38. What is a Jenkins build trigger **pollSCM?**

Polls the version control system at a scheduled interval and triggers a build if changes are detected.

```
triggers { pollSCM('H/5 * * * *') } // Check every 5 minutes
```

Q39. How do you mark a build as unstable vs failed in Jenkins?

`currentBuild.result = 'UNSTABLE'` marks the build yellow (tests failed but build succeeded). `currentBuild.result = 'FAILURE'` marks it red. Use `error('message')` to immediately fail.

Q40. What is the Jenkins Master/Agent architecture?

The Master (Controller) manages job scheduling, configuration, and results. Agents execute the actual build steps. The controller should never run builds itself in production.

Q41. How do you back up Jenkins?

Back up the `$JENKINS_HOME` directory — contains all jobs, configurations, and credentials. Use the ThinBackup or Backup plugins to automate this. Critical subdirectories: `jobs/`, `credentials.xml`, `config.xml`.

Q42. What is the `retry` step in Jenkins?

```
retry(3) {  
    sh 'flaky-command.sh'  
}
```

Retries the block up to 3 times if it fails. Useful for flaky network-dependent steps.

Q43. How do you integrate SonarQube into Jenkins?

Install SonarQube Scanner plugin. Configure the SonarQube server in [Manage Jenkins](#) → [Configure System](#). Use `withSonarQubeEnv('server-name')` wrapper and `waitForQualityGate()` step.

Q44. What is the difference between `sh` and `bat` steps?

`sh` runs shell commands on Unix/Linux agents. `bat` runs batch commands on Windows agents.

Q45. How do you use environment variables in Jenkins?

```
environment {  
    APP_NAME = 'my-app'  
    BUILD_TAG = "${env.BUILD_NUMBER}-${env.GIT_COMMIT[0..7]}"  
}
```

Q46. What is the `script {}` block in Declarative pipelines?

Allows embedding Scripted pipeline Groovy code inside a Declarative pipeline for complex logic like loops, conditionals, and dynamic stage creation.

Q47. What Jenkins plugins are essential for DevOps?

Git, Docker Pipeline, Kubernetes, Blue Ocean, SonarQube Scanner, Credentials Binding, Slack Notification, Pipeline: Stage View, JUnit, HTML Publisher.

Q48. How do you implement a Canary deployment in Jenkins?

Use the `input` step with metrics check between stages: deploy to 10% of servers, check error rates via curl to monitoring API, if OK proceed with full rollout, otherwise rollback.

Q49. What is the Jenkinsfile `agent none` directive?

Sets no global agent — each stage must define its own `agent`. Allows different stages to run on different nodes.

Q50. How do you configure Jenkins for high availability?

Use Jenkins in Active/Standby mode with shared NFS storage for `$JENKINS_HOME`. Alternatively, use CloudBees Jenkins with HA support or migrate to cloud-native alternatives like Tekton.

Q51. What is the difference between `checkout scm` and `git step`?

`checkout scm` uses the SCM configuration from the job settings (automatically available in multibranch pipelines). `git url: '...'` explicitly specifies a repository.

Use the "Throttle Concurrent Builds" plugin to limit how many builds run simultaneously — useful when builds consume shared resources like a staging database.

Q53. What are build artifacts in Jenkins?

Files produced during a build (JARs, Docker image digests, test reports) that Jenkins archives and makes available for download or passing to downstream jobs.

Q54. What is Jenkins Configuration as Code (JCasC)?

A plugin that allows the entire Jenkins configuration (credentials, plugins, agents, job definitions) to be defined in YAML and stored in version control. Enables reproducible Jenkins setups.

Q55. How do you clean a workspace in Jenkins?

```
post {  
    always {  
        cleanWs() // Requires workspace cleanup plugin  
    }  
}
```

SECTION 3: DOCKER (50 Questions)

Q56. What is a Docker image vs a Docker container?

An image is a read-only template (like a class in OOP). A container is a running instance of that image (like an object). Multiple containers can run from one image.

Q57. What is Docker Hub?

A public container registry where Docker images are stored, versioned, and shared. Think of it as GitHub but for Docker images.

Q58. What is the difference between `CMD` and `ENTRYPOINT` in a Dockerfile?

`ENTRYPOINT` defines the executable that always runs; it cannot be overridden (only appended). `CMD` provides default arguments that can be overridden at `docker run`.

Together: `ENTRYPOINT ["python"] + CMD ["app.py"]`.

Q59. What is Docker Compose?

A tool for defining and running multi-container applications. A `docker-compose.yml` file defines services, networks, and volumes, and `docker-compose up` starts everything together.

Q60. How do you remove all stopped containers and unused images?

```
docker system prune -a    # Removes everything unused
docker container prune    # Only stopped containers
docker image prune -a     # Only unused images
```

Q61. What is a Docker registry?

A storage system for Docker images. Docker Hub is the public registry. Organizations run private registries using AWS ECR, GCP Artifact Registry, Azure ACR, or self-hosted Harbor/Nexus.

Q62. What is the difference between `COPY` and `ADD` in Dockerfile?

COPY simply copies files. **ADD** does everything **COPY** does plus: extracts tar archives automatically and can fetch URLs. Best practice: use **COPY** unless you specifically need **ADD**'s extra features.

Q63. How do you pass environment variables to a Docker container?

```
docker run -e APP_ENV=production -e DB_HOST=localhost myapp
# Or from a file:
docker run --env-file .env myapp
```

Q64. What is a multi-stage Docker build?

A Dockerfile technique using multiple **FROM** instructions. Each stage can use different base images. The final stage copies only what's needed from earlier stages, producing a minimal production image.

Q65. How do you view Docker container logs?

```
docker logs my-container
docker logs -f my-container          # Follow (stream)
docker logs --tail 100 my-container # Last 100 lines
docker logs --since 10m my-container
```

Q66. How do you limit CPU and memory for a Docker container?

```
docker run --memory="512m" --cpus="0.5" myapp
# Limit to 512MB RAM and 0.5 CPU cores
```

Q67. What is **docker exec** used for?

Runs a command inside a running container without starting a new one.

```
docker exec -it my-container /bin/bash
docker exec my-container ls /var/log
```

Q68. What is Docker Swarm?

Docker's native container orchestration solution. Manages a cluster of Docker nodes and deploys services across them. Simpler than Kubernetes but less feature-rich.

Q69. What is the `.dockerignore` file?

Lists files and directories to exclude from the Docker build context. Prevents unnecessary files (like `node_modules`, `.git`, test files) from being sent to the Docker daemon, speeding up builds.

Q70. How do you check Docker image layers?

```
docker history myimage:latest
docker inspect myimage:latest
# Better: use the 'dive' tool for detailed layer analysis
dive myimage:latest
```

Q71. What is Docker BuildKit?

An improved build engine with better performance, layer caching (`--mount=type=cache`), parallel builds, and secret mounting (`--mount=type=secret`). Enable with `DOCKER_BUILDKIT=1`.

Q72. How do you tag and push a Docker image to ECR?

```
aws ecr get-login-password --region ap-south-1 | \
  docker login --username AWS --password-stdin 123456789.dkr.ecr.ap-south-
1.amazonaws.com
docker tag myapp:latest 123456789.dkr.ecr.ap-south-
1.amazonaws.com/myapp:latest
docker push 123456789.dkr.ecr.ap-south-1.amazonaws.com/myapp:latest
```

Q73. What is Docker networking `bridge` mode?

The default network mode. Creates a virtual bridge on the host. Containers on the same bridge can communicate; they're isolated from other networks. Each container gets its own IP.

Q74. What is the difference between `docker stop` and `docker kill`?

`docker stop` sends SIGTERM, waits 10 seconds for graceful shutdown, then sends SIGKILL. `docker kill` sends SIGKILL immediately with no grace period.

Q75. How do you copy files between host and container?

```
docker cp myfile.txt my-container:/app/myfile.txt # Host to container
docker cp my-container:/app/log.txt ./log.txt      # Container to host
```

Q76. What is container image scanning?

Analyzing a Docker image for known security vulnerabilities in OS packages and application dependencies. Tools: Trivy, Snyk, Clair, AWS ECR built-in scanning.

Q77. What is a Docker volume vs a bind mount?

A volume is managed by Docker (stored in `/var/lib/docker/volumes/`), portable, and the recommended way for persistent data. A bind mount links a specific host path to a container path — tighter coupling to the host.

Q78. What is `docker-compose.override.yml`?

An automatically merged override file for `docker-compose.yml`. Used to customize configurations for local development without modifying the base compose file. Useful for adding debug ports or changing environment variables locally.

Q79. How do you build an image without using cache?

```
docker build --no-cache -t myapp:latest .
```

Q80. What is the `HEALTHCHECK` instruction in Dockerfile?

Defines a command Docker runs periodically to check if the container is healthy.

```
HEALTHCHECK --interval=30s --timeout=5s CMD wget -q -O-
http://localhost:8080/health || exit 1
```

Q81. What is a distroless image?

A minimal container image that contains only the application and its runtime dependencies — no shell, no package manager, no OS utilities. Reduces attack surface significantly.

Example: gcr.io/distroless/java.

Q82. How do you mount secrets at build time securely?

```
# syntax=docker/dockerfile:1.4
RUN --mount=type=secret,id=npn_token \
    NPM_TOKEN=$(cat /run/secrets/npn_token) npm install
```

Secrets are available only during build, not baked into the image.

Q83. What is the `USER` instruction in Dockerfile?

Sets the user for all subsequent instructions (`RUN`, `CMD`, `ENTRYPOINT`). Best practice: create a non-root user and switch to it before the `CMD` instruction.

Q84. What does `docker inspect` show?

Detailed JSON metadata about a container or image: network settings, volumes, environment variables, labels, resource limits, IP addresses, and mount points.

Q85. What is Docker content trust?

A feature that ensures image integrity and publisher verification using digital signatures. Enable with `DOCKER_CONTENT_TRUST=1`. Only signed images can be pulled or run.

Q86. How do you update a running container's image without downtime?

You don't update containers in-place. The correct approach is: pull new image, start new container, redirect traffic, stop old container. Orchestrators (Docker Swarm, Kubernetes) handle this automatically via rolling updates.

Q87. What is `docker stats`?


```
docker stats my-container
docker stats --no-stream    # One-time snapshot
```

Q88. What is the difference between `EXPOSE` and publishing a port?

`EXPOSE` in Dockerfile is documentation only — it doesn't actually publish the port. Publishing (`-p 8080:8080`) is done at `docker run` time and creates actual host-to-container port mapping.

Q89. How do you create a custom Docker network?

```
docker network create --driver bridge --subnet 172.20.0.0/16 my-network
docker run --network my-network --name api myapp
```

Q90. What is Docker's garbage collection?

Docker doesn't automatically delete unused images, containers, or volumes. You run `docker system prune` manually or schedule it as a cron job. Kubernetes uses `imageGCHighThresholdPercent` for automatic cleanup.

Q91. What is `docker manifest`?

Used to create and push multi-architecture images (amd64, arm64). A manifest list points to different images for different platforms under a single tag.

Q92. How do you run a Docker container with a read-only filesystem?

```
docker run --read-only --tmpfs /tmp myapp
```

Makes the root filesystem read-only; `/tmp` is writable via tmpfs.

Q93. What is Docker's `--init` flag?

Adds a minimal init process (tini) as PID 1 inside the container. This properly handles zombie processes and signal forwarding, which the application process often doesn't do correctly.

Q94. What is a Docker context?

A named set of endpoint and TLS configuration for a Docker host. Allows switching between different Docker environments (local, remote, production) using `docker context use production`.

Q95. What is containerd?

A lightweight container runtime that manages the complete container lifecycle (pulling images, creating containers, managing storage). Docker Engine uses containerd internally. Kubernetes defaults to containerd directly via CRI (Container Runtime Interface).

Q96. How do you scan for secrets accidentally baked into Docker images?

Use `docker history --no-trunc myimage` to inspect build commands. Use tools like `trufflesecurity/trufflehog` or Trivy's secret scanning mode against images.

Q97. What is the difference between overlay2 and other Docker storage drivers?

`overlay2` is the recommended storage driver for modern Linux kernels. It uses OverlayFS for efficient copy-on-write layering. Other drivers include `devicemapper` (older) and `btrfs/zfs` (filesystem-specific).

Q98. How do you run a Docker container as a specific UID?

```
docker run --user 1001:1001 myapp
# Or in Dockerfile:
RUN useradd -u 1001 appuser
USER 1001
```

Q99. What is Harbor?

An open-source self-hosted container registry with built-in vulnerability scanning (via Trivy), RBAC, content trust, replication, and audit logging. The enterprise alternative to Docker Hub.

Q100. How do you implement log rotation for Docker containers?

```
docker run --log-opt max-size=10m --log-opt max-file=3 myapp
# Or in docker-compose.yml:
```

```
# logging:  
#   driver: json-file  
#   options:  
#     max-size: "10m"  
#     max-file: "3"
```

Prepared by Arvind Verma

SECTION 4: GIT / GITHUB / GITLAB (50

Questions)

Q101. What is `git rebase` vs `git merge`?

`merge` creates a new merge commit preserving full history. `rebase` replays commits on top of another branch creating a linear history. `Rebase` creates cleaner history but rewrites commit SHAs — never rebase public/shared branches.

Q102. What is `git cherry-pick`?

Applies a specific commit from one branch to another without merging the entire branch.

```
git cherry-pick abc123f # Apply commit abc123f to current branch
```

Q103. What is a `git stash`?

Temporarily saves uncommitted changes to a stack, giving you a clean working directory.

```
git stash push -m "work in progress"
git stash pop # restore latest stash
git stash list # view all stashes
```

Q104. What is `git bisect`?

A binary search tool to find the commit that introduced a bug.

```
git bisect start
git bisect bad HEAD # Current commit is broken
git bisect good v1.0 # v1.0 was working
# Git checks out middle commits; you mark each good/bad
git bisect reset # Exit bisect mode
```

Q105. What is `git reflog`?

Shows the history of HEAD movements — every checkout, merge, rebase, and reset. Your last line of defense for recovering "lost" commits.

```
git reflog
git checkout HEAD@{3}    # Go back to state 3 operations ago
```

Q106. What is the difference between `git reset --soft`, `--mixed`, and `--hard`?

All move HEAD to a specified commit. `--soft` keeps changes staged. `--mixed` (default) keeps changes unstaged. `--hard` discards all changes — dangerous, irreversible.

Q107. What is `git tag`?

Creates a named reference to a specific commit, typically for release versions.

```
git tag -a v1.2.0 -m "Release 1.2.0"
git push origin v1.2.0
git push origin --tags    # Push all tags
```

Q108. What is a protected branch?

A branch with rules preventing direct pushes, requiring PR reviews, and mandating passing CI checks before merging. Critical for `main` and `release` branches.

Q109. What is `git submodule`?

Allows embedding one Git repository inside another. Used when you need to include an external dependency at a specific version.

```
git submodule add https://github.com/org/lib.git libs/mylib
git submodule update --init --recursive
```

Q110. How do you squash multiple commits into one?

```
git rebase -i HEAD~5    # Interactively rebase last 5 commits
# Change 'pick' to 'squash' (s) for commits to squash
# Write combined commit message
```

Q111. What is `git blame`?

Shows who last modified each line of a file and in which commit. Useful for understanding why code was written a certain way.

```
git blame -L 50,60 src/app.py    # Lines 50-60 only
```

Q112. What is a Git hook?

Scripts that run automatically at specific Git events: `pre-commit` (before committing), `pre-push` (before pushing), `post-merge` (after merging). Used to enforce code standards locally.

Q113. How do you undo the last commit without losing changes?

```
git reset --soft HEAD~1    # Undo commit, keep changes staged
git reset HEAD~1           # Undo commit, keep changes unstaged
```

Q114. What is `git worktree`?

Allows multiple working directories to be checked out from the same repository simultaneously. Useful for working on multiple branches at once without switching.

Q115. What is a GitLab Runner?

An agent that runs CI/CD jobs defined in `.gitlab-ci.yml`. Can be shared (GitLab-managed) or self-hosted. Supports Docker, Kubernetes, shell, and VirtualBox executors.

Q116. What is the difference between `git fetch` and `git pull`?

`git fetch` downloads changes from remote but doesn't integrate them into your working branch. `git pull` = `git fetch` + `git merge`. Prefer `fetch` then inspect before merging.

only/except (older): basic branch/tag filtering. **rules** (newer): more expressive conditions using **if**, **changes**, **exists**.

```
rules:
- if: '$CI_COMMIT_BRANCH == "main"'
- if: '$CI_PIPELINE_SOURCE == "merge_request_event"'
  changes: ["src/**/*"]
```

Q118. What is a GitLab environment?

A named deployment target (staging, production) with URL tracking, deployment history, and optional approval requirements before deployment.

Q119. What is **git sparse-checkout**?

Allows checking out only a subset of files from a large repository, reducing disk space and clone time. Useful in monorepos.

Q120. What is the **include** keyword in **.gitlab-ci.yml**?

Includes pipeline configuration from other YAML files — from the same repo, other repos, or GitLab templates.

```
include:
- project: 'company/shared-pipelines'
  file: '/templates/docker-build.yml'
  ref: main
```

Q121. What is **git diff --staged**?

Shows the diff between the staging area (index) and the last commit. Same as **git diff --cached**. Shows what will be in your next commit.

Q122. How do you delete a remote branch?

```
git push origin --delete feature/my-branch
```

```
git push origin :feature/my-branch
```

Q123. What is a GitHub Action's `GITHUB_TOKEN`?

An automatically created token with limited permissions scoped to the repository. Available to every workflow run as `secrets.GITHUB_TOKEN` without any configuration.

Q124. What is Conventional Commits?

A specification for commit message format: `type(scope): description`.

Types: `feat`, `fix`, `docs`, `chore`, `refactor`, `test`, `ci`. Enables automatic changelog generation and semantic versioning.

Q125. What is `git clean`?

Removes untracked files from the working directory.

```
git clean -fd    # Remove untracked files and directories
git clean -n     # Dry run - show what would be removed
```

Q126. What is a monorepo?

A single Git repository containing multiple projects/services. Tools like Nx, Turborepo, or Bazel help manage build dependencies. Tradeoff: easier code sharing vs. slower CI and larger clone times.

Q127. What is GitHub's Dependabot?

Automated tool that scans dependencies for known vulnerabilities and outdated packages, then opens PRs to update them. Configurable via `.github/dependabot.yml`.

Q128. How do you sign Git commits with GPG?

```
gpg --gen-key
git config --global user.signingkey YOUR_GPG_KEY_ID
git config --global commit.gpgsign true
git commit -S -m "signed commit"
```


Q129. What is the difference between `origin` and `upstream` in Git?

`origin` is your fork's remote repository. `upstream` is the original repository you forked from. Convention in open-source workflows.

Q130. What is GitLab's SAST (Static Application Security Testing)?

Built-in security scanning that analyzes source code for vulnerabilities. Enabled by including the SAST template in `.gitlab-ci.yml`. Supports Python, Java, JavaScript, Go, Ruby, and more.

Q131. What is `git shortlog`?

Summarizes `git log` output grouped by author — useful for release notes.

```
git shortlog -sn HEAD~20..HEAD # Last 20 commits grouped by author
```

Q132. What is a GitHub release?

A deployable software iteration built from a tag. Includes release notes, linked issues/PRs, and attached binary assets (zip files, executables).

Q133. What does `git log --oneline --graph` show?

A compact, visual branch graph showing commit history. Each commit is one line with its abbreviated hash and message.

Q134. What is a `.gitattributes` file?

Defines attributes for paths in the repository: line ending normalization (`text=auto`), diff algorithms for binary files, language statistics for GitHub's language bar.

Q135. How do you recover a deleted branch?

```
git reflog | grep "checkout: moving from deleted-branch"
# Find the commit SHA
git checkout -b recovered-branch abc123f
```

Q136. What is GitHub Actions' cache action?

```
- uses: actions/cache@v4
  with:
    path: ~/.npm
    key: ${{ runner.os }}-npm-${{ hashFiles('package-lock.json') }}
```

Caches dependencies between workflow runs, dramatically speeding up builds.

Q137. What is GitLab's Auto DevOps?

A set of predefined CI/CD templates that automatically build, test, deploy, and monitor applications using GitLab best practices — with zero configuration for common frameworks.

Q138. What is a Git mirror?

An exact copy of a repository including all branches, tags, and refs. Created with `git clone --mirror`. Used for backups and repository migrations.

Q139. What is the CODEOWNERS file?

A file (`.github/CODEOWNERS` or `.gitlab/CODEOWNERS`) that defines who owns specific files. Automatically adds those owners as required reviewers when their files are changed in a PR.

Q140. What is `git archive`?

Creates a zip or tar archive of a specific tree or commit without the `.git` directory.

```
git archive --format=zip HEAD > release.zip
```

Q141. What is GitHub Packages?

A package registry for Docker images, npm, Maven, NuGet, and RubyGems packages integrated with GitHub. Packages are linked to repositories and use the same authentication.

Q142. What is `git notes`?

Adds metadata to commits without changing the commit SHA. Useful for adding CI results, code review comments, or deployment records retroactively.

Q143. What is a detached HEAD state in Git?

When HEAD points directly to a commit instead of a branch. Changes made in this state won't belong to any branch and can be "lost" if you switch branches without creating a new branch.

Q144. What is `git rerere`?

"Reuse Recorded Resolution" — records how you resolved a merge conflict and automatically applies the same resolution when the same conflict occurs again.

Q145. How do you set up branch protection in GitHub?

`Settings` → `Branches` → `Add rule`. Configure: require PR reviews, require status checks, require signed commits, restrict pushes, require linear history.

Q146. What is GitLab's merge train?

A queue-based merge strategy where each MR is validated in sequence to ensure no merge conflicts occur before merging to the main branch. Prevents the "last green wins" race condition.

Q147. What is `git log -S "search_string"`?

The "pickaxe" flag — searches through Git history for commits where the search string was added or removed. Used to track when a specific function or variable was introduced.

Q148. What is conventional versioning (SemVer)?

`MAJOR.MINOR.PATCH` — `2.4.1`. MAJOR: breaking change. MINOR: new feature, backward-compatible. PATCH: bug fix, backward-compatible. Pre-release: `1.0.0-alpha.1`.

Q149. What is GitHub's code scanning?

Automated security analysis using CodeQL or third-party tools. Runs as part of CI and reports vulnerabilities directly on PRs. Powered by GitHub Advanced Security.

Q150. What is `git worktree` useful for in CI/CD?

Allows CI systems to check out multiple branches simultaneously in a single clone, enabling parallel builds without multiple clones. Saves disk space and time.

Prepared by Arvind Verma

SECTION 5: AWS DEVOPS (50 Questions)

Q151. What is AWS CodeCommit?

A fully managed private Git repository service. Secure, highly available, and integrated with IAM for access control. It's being deprecated — AWS announced end of new customer onboarding in 2024.

Q152. What is AWS CodeBuild?

A fully managed build service that compiles code, runs tests, and produces artifacts. No server management required. Charges per build minute.

Q153. What is AWS CodeDeploy?

A deployment service that automates application deployments to EC2, Lambda, or ECS. Supports rolling, blue/green, and in-place deployment strategies.

Q154. What is AWS CodePipeline?

An orchestration service that automates the steps from code commit to production. Connects source (GitHub), build (CodeBuild), and deploy (CodeDeploy/ECS) stages.

Q155. What is AWS ECR (Elastic Container Registry)?

A managed Docker container registry. Integrates natively with ECS and EKS. Supports vulnerability scanning, image lifecycle policies, and cross-account access.

Q156. What is an ECR lifecycle policy?

Automatically removes old Docker images based on rules.

```
{
  "rules": [{
    "rulePriority": 1,
    "selection": {
```

```

    "tagStatus": "untagged",
    "countType": "sinceImagePushed",
    "countUnit": "days",
    "countNumber": 7
  },
  "action": { "type": "expire" }
}]
}

```

Q157. What is AWS ECS vs EKS?

ECS (Elastic Container Service): AWS-native orchestrator, simpler to operate, tightly integrated with AWS services. EKS (Elastic Kubernetes Service): Managed Kubernetes, industry-standard API, more complex but portable. Choose ECS for simplicity, EKS for Kubernetes compatibility.

Q158. What is AWS Fargate?

A serverless compute engine for ECS and EKS. No EC2 instances to manage — you define CPU/memory per task and pay per second of usage.

Q159. What is an ECS Task Definition?

A blueprint for running a container in ECS — defines Docker image, CPU, memory, IAM role, environment variables, port mappings, and logging configuration.

Q160. What is the difference between ECS Task Role and Execution Role?

Execution Role: used by ECS agent to pull images from ECR and write logs to CloudWatch. **Task**

Role: assumed by the application container to access AWS services (S3, DynamoDB, etc.).

Q161. What is AWS CloudFormation?

AWS's native IaC service. You define resources in JSON or YAML templates; CloudFormation provisions and manages them as a stack with drift detection and rollback capability.

Q162. What is a CloudFormation Stack?

A collection of AWS resources created and managed together from a CloudFormation template.

A stack can be created, updated, or deleted as a unit.

Q163. What is CloudFormation Change Sets?

A preview of the changes CloudFormation will make before applying them. Like `terraform plan`.

Shows which resources will be added, modified, or deleted.

Q164. What is AWS SAM (Serverless Application Model)?

An extension of CloudFormation with simplified syntax for Lambda functions, API Gateway, DynamoDB tables, and other serverless resources. Uses `sam deploy` for deployment.

Q165. What is AWS CDK?

Cloud Development Kit — define AWS infrastructure using Python, TypeScript, Java, or C#. CDK synthesizes to CloudFormation templates. Provides high-level constructs that encode best practices.

Q166. What is AWS Systems Manager Parameter Store?

A service for storing configuration data and secrets as key-value pairs. Supports plain strings and SecureString (KMS-encrypted). Cheaper than Secrets Manager but fewer features.

Q167. What is AWS Secrets Manager?

A secrets management service with automatic rotation, cross-account access, and fine-grained IAM policies. More expensive than Parameter Store but supports native rotation for RDS, Redshift, and DocumentDB.

Q168. What is an IAM Role vs IAM User?

An IAM User has permanent credentials (access key + secret). An IAM Role has temporary credentials obtained via STS `AssumeRole`. Use roles for EC2 instances, Lambda, and CI/CD systems — never long-lived users.

Q169. What is the principle of least privilege in AWS?

Granting only the minimum permissions required for a task. IAM policies should be specific about actions, resources, and conditions. Avoid * wildcards in production policies.

Q170. What is AWS VPC?

Virtual Private Cloud — a logically isolated network within AWS. Contains subnets (public and private), route tables, internet gateways, NAT gateways, and security groups.

Q171. What is a NAT Gateway?

Allows EC2 instances in private subnets to initiate outbound internet connections (for updates, API calls) while blocking unsolicited inbound connections from the internet.

Q172. What is a Security Group vs NACL?

Security Group: stateful, instance-level firewall (return traffic automatically allowed). NACL (Network ACL): stateless, subnet-level firewall (return traffic must be explicitly allowed). Use both in defense-in-depth.

Q173. What is AWS CloudWatch?

A monitoring service for collecting metrics, logs, and events. Supports dashboards, alarms, anomaly detection, and log insights queries. The default observability service for all AWS services.

Q174. What is a CloudWatch Alarm?

Monitors a metric and performs actions when it crosses a threshold — send SNS notification, trigger Auto Scaling, or invoke Lambda.

Q175. What is CloudWatch Logs Insights?

A query language for analyzing logs stored in CloudWatch Logs.

```
fields @timestamp, @message
| filter @message like /ERROR/
| stats count() by bin(5m)
```


A distributed tracing service for analyzing and debugging distributed applications. Instruments requests as they flow through microservices and shows performance bottlenecks in a service map.

Q177. What is AWS Config?

Tracks configuration changes to AWS resources over time and evaluates configurations against desired rules. Used for compliance auditing and security analysis.

Q178. What is AWS CloudTrail?

Records every API call made in your AWS account — who did what, when, and from where. Used for security auditing, incident investigation, and compliance.

Q179. What is AWS Auto Scaling?

Automatically adjusts the number of EC2 instances or ECS tasks based on demand. Types: Target Tracking (maintain a metric), Step Scaling (scale by increments), Scheduled Scaling.

Q180. What is an Application Load Balancer (ALB)?

Operates at Layer 7 (HTTP/HTTPS). Supports host-based and path-based routing, WebSockets, HTTP/2, and redirects. Use for microservices and container-based applications.

Q181. What is a Network Load Balancer (NLB)?

Operates at Layer 4 (TCP/UDP). Handles millions of requests per second with ultra-low latency. Use for non-HTTP protocols, static IPs, and latency-sensitive applications.

Q182. What is AWS Route 53?

AWS's DNS service. Supports routing policies: Simple, Weighted (A/B testing), Latency-based (serve from nearest region), Failover (active-passive), Geolocation.

Q183. What is AWS S3 versioning?

Keeps multiple versions of an object. Protects against accidental deletion and overwrites.

Required for S3-backed Terraform state to enable recovery from corruption.

Q184. What is S3 bucket policy vs ACL?

Bucket policy: JSON-based resource policy for fine-grained access control. ACL: legacy, object-level access. Use bucket policies — ACLs are deprecated for most use cases.

Q185. What is AWS KMS?

Key Management Service — creates and controls encryption keys. Used for encrypting EBS volumes, S3 objects, RDS databases, and Secrets Manager secrets.

Q186. What is an AWS Lambda function?

A serverless compute service that runs code in response to events without provisioning servers. Executes in 15 minutes maximum. Used for data processing, API backends, and event-driven automation.

Q187. What is EventBridge?

A serverless event bus that connects AWS services, SaaS applications, and custom applications. Replaces CloudWatch Events with more features including schema registry and cross-account events.

Q188. What is AWS Step Functions?

A serverless workflow orchestration service for coordinating Lambda functions and AWS services into complex workflows. Supports sequential, parallel, conditional, and retry logic.

Q189. What is an Amazon SQS queue?

Simple Queue Service — fully managed message queue. Decouples producers and consumers. Supports standard queues (at-least-once delivery) and FIFO queues (exactly-once, ordered).

Q190. What is AWS SNS?

Simple Notification Service — managed pub/sub messaging. Publishers send to a topic; subscribers (email, SMS, Lambda, SQS, HTTP) receive notifications.

Q191. What is AWS EKS managed node groups?

EC2 instances managed by EKS for running Kubernetes worker nodes. AWS handles provisioning, updating, and draining nodes. Supports On-Demand and Spot instances.

Q192. What is Karpenter?

An open-source node provisioner for Kubernetes (developed by AWS). Faster and more cost-efficient than Cluster Autoscaler — provisions exactly the right EC2 instance type for pending pods.

Q193. What is AWS Elastic Beanstalk?

A PaaS service that handles infrastructure provisioning, load balancing, auto scaling, and monitoring for web applications. You upload code; Beanstalk manages the rest. Good for simple apps.

Q194. What is AWS Lightsail?

A simplified cloud platform for small workloads — virtual servers, databases, containers, and CDN with predictable pricing. Targeted at developers who don't need full AWS complexity.

Q195. What is AWS Cost Explorer?

A visualization tool for analyzing AWS spending over time, identifying cost drivers, and forecasting future costs. Supports filtering by service, tag, account, and region.

Q196. What is an AWS Budget alert?

Sends notifications (email/SNS) when actual or forecasted costs exceed a defined threshold. Essential for preventing bill surprises.

Q197. What is AWS Compute Optimizer?

Recommends optimal instance types for EC2, Lambda, ECS, EBS, and Auto Scaling groups based on historical utilization data. Typically identifies 20-30% cost savings.

Q198. What is AWS Trusted Advisor?

An automated advisor that checks your AWS infrastructure against best practices in cost optimization, performance, security, fault tolerance, and service limits.

Q199. What is AWS WAF?

Web Application Firewall — protects web applications from common exploits (SQL injection, XSS, bad bots). Integrates with ALB, CloudFront, and API Gateway.

Q200. What is AWS Shield?

A DDoS protection service. Shield Standard (free) provides automatic protection against common DDoS attacks. Shield Advanced (\$3,000/month) provides 24/7 DDoS response team support.

Prepared by Arvind Verma

SECTION 6: KUBERNETES (75 Questions)

Q201. What is a Namespace in Kubernetes?

A logical partition within a cluster that isolates resources. Different teams or environments (dev, staging, prod) use separate namespaces. Resource quotas and RBAC are applied per namespace.

Q202. What is a DaemonSet?

Ensures one Pod runs on every node (or selected nodes). Used for node-level services: log collectors (Filebeat), monitoring agents (node-exporter), network plugins.

Q203. What is a StatefulSet?

Manages stateful applications with stable Pod names (`pod-0`, `pod-1`), stable network identities, and ordered deployment/scaling. Used for databases, Kafka, Zookeeper.

Q204. What is a Job in Kubernetes?

Runs a Pod to completion (not continuously). A Job ensures a specified number of Pods successfully terminate. Used for batch processing, database migrations, data exports.

Q205. What is a CronJob?

A Kubernetes resource that creates Jobs on a scheduled basis using standard cron syntax.

```
schedule: "0 2 * * *" # Run at 2 AM daily
```

Q206. What is a PersistentVolume (PV) and PersistentVolumeClaim (PVC)?

PV is a piece of storage provisioned by an admin (or dynamically by a StorageClass). PVC is a request for storage by a user. The PVC binds to a matching PV.

Q207. What is a StorageClass?

Defines how storage is dynamically provisioned. Parameters include provisioner (aws EBS, Azure Disk), type (gp3, io1), reclaim policy (Delete, Retain).

Q208. What is RBAC in Kubernetes?

Role-Based Access Control — controls who can do what in the cluster. Core objects: `Role/ClusterRole` (permissions) + `RoleBinding/ClusterRoleBinding` (assigns roles to users/service accounts).

Q209. What is a ServiceAccount?

An identity for processes running in a Pod. Pods use ServiceAccounts to authenticate with the Kubernetes API and with external services (via IRSA in EKS).

Q210. What is IRSA (IAM Roles for Service Accounts)?

An EKS feature that allows Pods to assume IAM roles using Kubernetes ServiceAccounts, via OIDC federation. Replaces storing AWS credentials in secrets — fine-grained, pod-level AWS permissions.

Q211. What is a Kubernetes Operator?

A custom controller that extends Kubernetes to manage complex stateful applications using custom resources. Encodes operational knowledge into code. Examples: Prometheus Operator, Cert-Manager, Strimzi Kafka.

Q212. What is a Custom Resource Definition (CRD)?

Extends Kubernetes API with custom resource types. For example, `ExternalSecret`, `Rollout`, `Application` (Argo CD). Allows treating custom objects the same way as built-in Kubernetes objects.

Q213. What is Helm?

A package manager for Kubernetes. Helm Charts are templated Kubernetes manifests with default values. `helm install` deploys a release; `helm upgrade` updates it; `helm rollback` reverts.

Q214. What is a Helm values file?

A YAML file containing configuration overrides for a Helm chart. Allows customizing the same chart for different environments.

```
helm upgrade myapp ./chart -f values-prod.yaml --set image.tag=v1.2
```

Q215. What is a Kubernetes NetworkPolicy?

Controls traffic flow between Pods. Without NetworkPolicy, all Pods can communicate freely. With NetworkPolicy, you can restrict ingress/egress traffic based on Pod selectors, namespace selectors, and IP blocks.

Q216. What is a PodDisruptionBudget (PDB)?

Ensures a minimum number of Pods remain available during voluntary disruptions (node drain, rolling update).

```
spec:
  minAvailable: 2 # Always keep at least 2 pods running
```

Q217. What is a Kubernetes Admission Controller?

A webhook that intercepts API requests before resources are persisted. Validates (rejects invalid resources) or mutates (modifies resources). Examples: OPA Gatekeeper, Kyverno, PodSecurityAdmission.

Q218. What is OPA Gatekeeper?

An admission controller that enforces custom policies written in Rego. Example policies: all images must come from approved registries, all containers must have resource limits, no privileged containers.

Q219. What is Kyverno?

A policy engine for Kubernetes written in Go, with policies expressed as Kubernetes resources (no

Q220. What is Cert-Manager?

A Kubernetes operator that automates TLS certificate issuance and renewal from Let's Encrypt, HashiCorp Vault, or other CA. Eliminates certificate expiry incidents.

Q221. What is External DNS?

A Kubernetes controller that automatically updates DNS records (Route 53, Cloudflare, etc.) when Kubernetes Services and Ingresses are created.

Q222. What is the Kubernetes scheduler?

Assigns Pods to Nodes based on: resource availability, node affinity/anti-affinity rules, taints and tolerations, topology spread, and priority classes.

Q223. What are Taints and Tolerations?

Taints repel Pods from nodes unless the Pod has a matching Toleration. Used to dedicate nodes for specific workloads (GPU nodes, spot-only nodes).

```
# Node taint:
kubectl taint nodes gpu-node-1 gpu=true:NoSchedule
# Pod toleration:
tolerations:
- key: "gpu"
  value: "true"
  effect: "NoSchedule"
```

Q224. What are Node Affinity and Pod Affinity?

Node Affinity: attracts Pods to nodes with specific labels. Pod Affinity: co-locates Pods. Pod Anti-Affinity: spreads Pods across nodes/zones for high availability.

Q225. What is a TopologySpreadConstraint?

Distributes Pods evenly across nodes, zones, or regions.


```
topologySpreadConstraints:
  - maxSkew: 1
    topologyKey: topology.kubernetes.io/zone
    whenUnsatisfiable: DoNotSchedule
    labelSelector:
      matchLabels:
        app: my-api
```

Q226. What is Kubernetes resource quotas?

Limits total resource consumption per namespace.

```
apiVersion: v1
kind: ResourceQuota
spec:
  hard:
    pods: "50"
    requests.cpu: "20"
    requests.memory: 40Gi
    limits.memory: 80Gi
```

Q227. What is LimitRange?

Sets default requests/limits for containers in a namespace that don't specify their own. Prevents Pods from being created without resource specifications.

Q228. What is `kubectl rollout restart`?

Triggers a rolling restart of a Deployment without changing any configuration. Useful for picking up new ConfigMap values or reloading certificates.

```
kubectl rollout restart deployment/my-app -n production
```

Q229. What is the difference between `kubectl apply` and `kubectl create`?

`kubectl create` fails if the resource already exists. `kubectl apply` creates or updates — idempotent. Always use `apply` in pipelines.

Q230. What is a Kubernetes Service of type `ExternalName`?

Maps a Service to an external DNS name. Acts as a CNAME alias.

```
spec:
  type: ExternalName
  externalName: my.database.company.com
```

Q231. What is `kubectl port-forward`?

Forwards a local port to a port on a Pod or Service for debugging. Traffic doesn't go through the Service — connects directly to the Pod.

```
kubectl port-forward pod/my-pod 8080:8080 -n production
```

Q232. What is `kubectl debug`?

Creates an ephemeral debug container attached to a running Pod — useful for debugging minimal containers that don't have shell or debugging tools.

```
kubectl debug -it my-pod --image=busybox --target=app
```

Q233. What is Vertical Pod Autoscaler (VPA)?

Recommends or automatically adjusts CPU/memory requests and limits for containers based on historical usage. Complements HPA (HPA scales out horizontally; VPA right-sizes vertically).

Q234. What is KEDA (Kubernetes Event-Driven Autoscaling)?

An autoscaler that scales Pods based on external event sources — SQS queue depth, Kafka consumer lag, HTTP request rate, Redis list length. More flexible than HPA for event-driven workloads.

Q235. What is Istio?

A service mesh that adds traffic management, mutual TLS (mTLS), observability, and security policies to microservices without code changes. Uses Envoy proxy sidecars injected into each Pod.

Q236. What is a service mesh?

An infrastructure layer managing service-to-service communication. Provides: automatic mTLS, traffic splitting, retries, circuit breaking, distributed tracing, and traffic metrics — all without application code changes.

Q237. What is Linkerd?

A lightweight service mesh alternative to Istio. Simpler to operate, lower resource overhead, written in Rust. Provides mTLS, observability, and traffic management.

Q238. What is Kubernetes Federation?

(Deprecated in favor of GitOps approaches) A system for managing multiple Kubernetes clusters from a single control plane. Modern alternatives: Argo CD ApplicationSets, Fleet (Rancher).

Q239. What is a multi-tenant Kubernetes cluster?

A cluster shared by multiple teams/applications. Requires proper namespace isolation, RBAC, ResourceQuotas, NetworkPolicies, and admission controllers to prevent teams from affecting each other.

Q240. What is Kubernetes Priority Classes?

Assigns priority to Pods for scheduling and eviction. High-priority Pods preempt lower-priority Pods if the cluster is resource-constrained.

```
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
value: 1000000
globalDefault: false
```

```
description: "Critical production workloads"
```

Q241. What is `kubectl top`?

Displays CPU and memory usage for nodes or Pods. Requires metrics-server to be installed.

```
kubectl top nodes  
kubectl top pods -n production --sort-by=memory
```

Q242. What is Velero?

An open-source tool for backup and disaster recovery of Kubernetes cluster resources and persistent volumes. Backs up to object storage (S3, GCS, Azure Blob).

Q243. What is Rancher?

An open-source multi-cluster Kubernetes management platform. Provides a UI for managing multiple clusters, centralized authentication, and application catalog.

Q244. What is k9s?

A terminal-based Kubernetes UI for navigating and managing clusters. Faster than `kubectl` for day-to-day operations — navigate pods, view logs, exec into containers, all from the terminal.

Q245. What is a Kubernetes `initContainer`?

A container that runs to completion before the main application container starts. Used for initialization tasks: waiting for a service to be ready, running database migrations, setting up configuration files.

Q246. What is a sidecar container pattern?

Co-locating a helper container in the same Pod as the main application. Examples: Envoy proxy (Istio), Filebeat (log shipping), Vault Agent (secrets management), Nginx (SSL termination).

Q247. What is pod topology awareness?

Using affinity/anti-affinity rules and topology spread constraints to ensure Pods are distributed across failure domains (availability zones, racks) for high availability.

Q248. What is the Kubernetes API rate limiting?

The API server limits requests to prevent overload. `kubectl` uses exponential backoff when rate-limited. In CI/CD pipelines with many parallel `kubectl` calls, rate limiting can cause intermittent failures.

Q249. What is etcd and why is it critical?

Distributed key-value store that holds all Kubernetes cluster state. If etcd is lost without a backup, the entire cluster configuration is lost. Back up etcd regularly with `etcdctl snapshot save`.

Q250. What is PodSecurityAdmission (PSA)?

Replaces PodSecurityPolicy (deprecated). Enforces security profiles at namespace level. Three modes: `privileged` (no restrictions), `baseline` (minimal restrictions), `restricted` (hardened).

Q251. What is the Kubernetes downwardAPI?

Allows Pods to access information about themselves (Pod name, namespace, labels, resource requests) via environment variables or volume files — without the app needing to call the Kubernetes API.

Q252. What is a ClusterIP vs NodePort vs LoadBalancer Service?

ClusterIP: internal-only, cluster-accessible. NodePort: exposes on a static port on every node. LoadBalancer: creates a cloud load balancer — only works on supported cloud providers.

Q253. What is `kubectl drain`?

Safely evicts all Pods from a node before maintenance, respecting PodDisruptionBudgets.

```
kubectl drain node-1 --ignore-daemonsets --delete-emptydir-data
```

Q254. What is `kubectl cordon`?

Marks a node as unschedulable — no new Pods will be scheduled on it, but existing Pods continue running. Used before maintenance. `kubectl uncordon` restores scheduling.

Q255. What is the difference between Deployment and ReplicaSet?

A Deployment manages ReplicaSets. When you update a Deployment, it creates a new ReplicaSet and scales it up while scaling down the old one. ReplicaSet just ensures N replicas exist — no update strategy.

Q256. What is Kubernetes garbage collection?

Automatically deletes objects owned by deleted objects (cascading deletion). Also cleans up terminated Pods and completed Jobs based on `ttlSecondsAfterFinished`.

Q257. What is the `kubectl explain` command?

Shows documentation for any Kubernetes resource field from the command line.

```
kubectl explain deployment.spec.strategy.rollingUpdate
```

Q258. What is Crossplane?

A CNCF project that extends Kubernetes to provision and manage cloud infrastructure (AWS, Azure, GCP resources) using Kubernetes CRDs and controllers. "Terraform for Kubernetes."

Q259. What is Flux CD?

A GitOps tool (CNCF project) that continuously reconciles Kubernetes cluster state with Git repository definitions. Alternative to Argo CD — lighter weight but fewer UI features.

Q260. What is KubeVirt?

A Kubernetes extension for running virtual machines alongside containers on the same cluster. Useful for migrating VM-based workloads to Kubernetes without full containerization.

Q261. What is Knative?

A Kubernetes-based platform for serverless workloads. Provides auto-scaling to zero, event-driven invocation, and traffic splitting. Used by Google Cloud Run under the hood.

Q262. What is Kubernetes node pressure eviction?

When a node runs low on resources (memory, disk, PIDs), the kubelet evicts Pods to reclaim resources, starting with BestEffort QoS class Pods.

Q263. What is an Endpoint Slice?

A scalable replacement for Endpoints objects. Instead of one Endpoints object per Service (which can be very large), EndpointSlices break them into smaller chunks for better scalability.

Q264. What is the Kubernetes watch mechanism?

Clients (kubectl, controllers) can **watch** a resource type. The API server sends real-time updates when resources change — much more efficient than polling.

Q265. What is **kubectl diff?**

Shows what would change if you applied a manifest, without applying it.

```
kubectl diff -f deployment.yaml
```

Q266. What is a Kubernetes ConfigMap hot reload?

When a ConfigMap is mounted as a volume (not env vars), Kubernetes automatically updates the mounted files when the ConfigMap changes — without Pod restart. The application must watch for file changes.

Q267. What is Kubernetes Pod preemption?

When a high-priority Pod cannot be scheduled due to insufficient resources, the scheduler evicts lower-priority Pods to make room.

Q268. What is the Kubernetes **watch with **--watch-only** flag?**

`kubectl get pods -w` starts watching after showing current state. `--watch-only` skips the initial list and only shows changes.

Q269. What is pod overhead?

Resources consumed by the Pod sandbox (not the containers). Defined in RuntimeClass for specific container runtimes like Kata Containers.

Q270. What is AppArmor in Kubernetes?

A Linux security module enforced at the Pod level via annotations. Restricts what system calls and file operations containers can perform.

Q271. What is seccomp in Kubernetes?

System call filtering for containers. Restricts which Linux system calls a container can make, reducing the attack surface.

```
securityContext:
  seccompProfile:
    type: RuntimeDefault
```

Q272. What is `kubectl auth can-i`?

Checks if a user has permission to perform an action.

```
kubectl auth can-i create pods -n production
kubectl auth can-i --list -n production # List all permissions
```

Q273. What is the Kubernetes Lease object?

Used by control plane components for leader election and node heartbeats. Each node updates its Lease object every 10 seconds; if a node doesn't update for 40 seconds, it's considered NotReady.

Q274. What is `kubectl rollout history`?

Shows the revision history of a Deployment.

```
kubectl rollout history deployment/my-app -n production
kubectl rollout history deployment/my-app --revision=3 # Show specific
revision details
```

Q275. What is a Kubernetes Headless Service?

A Service with `clusterIP: None`. Returns the individual Pod IPs via DNS instead of a single virtual IP. Used by StatefulSets for direct Pod-to-Pod communication.

Prepared by Arvind Verma

SECTION 7: PROMETHEUS & GRAFANA

(30 Questions)

Q276. What is a Prometheus exporter?

A translator that converts an application's metrics format into Prometheus exposition format.

Examples: `node_exporter` (OS metrics), `mysqld_exporter` (MySQL), `redis_exporter` (Redis).

Q277. What is Prometheus federation?

A hierarchical setup where one Prometheus server scrapes metrics from another. Used in large deployments to aggregate metrics from multiple Prometheus instances.

Q278. What is Prometheus remote write?

Allows Prometheus to forward metrics to a remote storage system (Thanos, Cortex, VictoriaMetrics) for long-term retention beyond the local TSDB retention period.

Q279. What is Thanos?

A highly available Prometheus setup with long-term storage. Adds global query view across multiple Prometheus instances, unlimited retention via object storage (S3), and downsampling.

Q280. What is Cortex / Mimir?

Horizontally scalable, highly available Prometheus-as-a-service. Grafana Mimir is the open-source successor to Cortex. Used when a single Prometheus instance isn't sufficient.

Q281. What is VictoriaMetrics?

A high-performance, cost-effective time-series database compatible with Prometheus remote write. Uses 7-10x less storage than Prometheus and handles much higher ingestion rates.

Q282. What does `increase()` do in PromQL?

Returns the increase in a counter over a time range.

```
increase(http_requests_total[1h]) # Total new requests in the last hour
```

Q283. What is `irate()` vs `rate()` in PromQL?

`rate()` calculates the per-second average over the time window — more stable. `irate()` uses only the last two data points — more responsive to spikes. Use `rate()` for alerting, `irate()` for dashboards.

Q284. What is Prometheus `recording rules`?

Pre-computes expensive PromQL expressions and saves results as new metrics. Speeds up dashboard loading and reduces query load.

```
- record: job:http_requests:rate5m
  expr: sum by (job) (rate(http_requests_total[5m]))
```

Q285. What is Alertmanager deduplication?

Alertmanager groups identical alerts from multiple Prometheus instances and sends only one notification. Prevents alert storms when multiple instances detect the same issue.

Q286. What is a Grafana data source?

A connection to a metrics backend (Prometheus, CloudWatch, Elasticsearch, InfluxDB). One Grafana instance can display data from multiple data sources in the same dashboard.

Q287. What is Grafana Loki?

A horizontally scalable log aggregation system inspired by Prometheus. Uses labels for indexing (not full-text indexing), making it much cheaper than Elasticsearch for log storage.

Q288. What is LogQL?

Loki's query language, similar to PromQL. Used to query, filter, and aggregate log streams.

```
{namespace="production", app="api"} |= "ERROR" | json | latency > 2000
```

Q289. What is Grafana Tempo?

A distributed tracing backend by Grafana Labs. Stores traces from Jaeger, Zipkin, or OpenTelemetry. Integrates with Loki and Prometheus for correlated observability.

Q290. What is the Grafana Unified Alerting system?

A centralized alerting engine in Grafana 8+ that consolidates alerts from multiple data sources (Prometheus, CloudWatch, etc.) into a single pane with notification routing.

Q291. What is `absent()` in PromQL?

Returns a vector if the specified metric doesn't exist. Used to alert when a target stops sending metrics.

```
absent(up{job="api"}) # Alert if the API is no longer being scraped
```

Q292. What is the Prometheus Operator?

A Kubernetes operator that manages Prometheus instances using CRDs: `Prometheus`, `ServiceMonitor`, `PodMonitor`, `PrometheusRule`, `AlertmanagerConfig`. Installed via kube-prometheus-stack Helm chart.

Q293. What is a `ServiceMonitor`?

A CRD used by the Prometheus Operator to define how to scrape a Service. Replaces manually editing `prometheus.yml`.

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
spec:
  selector:
    matchLabels:
      app: my-api
```

```
- port: metrics
  interval: 30s
```

Q294. What is the `scrape_timeout` in Prometheus?

How long Prometheus waits for a scrape response before considering it failed. Must be less than `scrape_interval`. Default: 10s.

Q295. What is metric cardinality?

The number of unique label combinations for a metric. High cardinality (millions of unique values like user IDs as labels) causes memory issues and slow queries. Avoid high-cardinality labels.

Q296. What is Grafana's `$__interval` variable?

A built-in variable that Grafana calculates based on the dashboard time range and panel width. Used in `rate()` or `increase()` to automatically adjust the time window for the display resolution.

Q297. What is a Grafana Panel?

A single visualization element in a dashboard (graph, stat, table, gauge, heatmap). Each panel has its own query, display settings, and thresholds.

Q298. What is Grafana's Explore mode?

An ad-hoc query interface for troubleshooting — run one-off queries without creating a dashboard. Supports metrics (Prometheus), logs (Loki), and traces (Tempo) in the same view.

Q299. How do you silence an alert in Alertmanager?

Create a Silence with matchers for the alert labels. The silence inhibits notifications for a defined time period — useful during planned maintenance.

```
amtool silence add alertname="HighCPU" instance="web-01" --duration=2h --
comment="Planned maintenance"
```

Q300. What is Grafana OnCall?

An on-call management solution (part of the Grafana stack). Manages on-call schedules, escalation policies, and alert routing — integrates with PagerDuty, Slack, and phone calls.

Q301. What is `predict_linear()` in PromQL?

Predicts the future value of a metric using linear regression.

```
predict_linear(node_filesystem_free_bytes[6h], 24 * 3600) # Predict disk
space in 24 hours
```

Used in alerts: "disk will be full in less than 4 hours."

Q302. What is Grafana's alerting `evaluation interval`?

How frequently Grafana evaluates alert rules. A 1-minute interval means the alert condition is checked every 60 seconds. Shorter intervals = faster detection but higher load.

Q303. What is `topk()` in PromQL?

Returns the K time series with the highest values.

```
topk(5, sum by (pod) (rate(http_requests_total[5m])))
# Top 5 pods by request rate
```

Q304. What is `label_replace()` in PromQL?

Adds or modifies labels on a metric result using regex.

```
label_replace(up, "host", "$1", "instance", "([^:]+):.*")
# Extracts hostname from "hostname:port" instance label
```

Q305. What is Grafana's `transformations` feature?

Post-processing steps applied to query results before visualization — merge queries, filter rows, calculate new fields, rename columns. No additional query needed.

SECTION 8: ELK STACK (30 Questions)

Q306. What is an Elasticsearch index?

A collection of documents with similar characteristics stored in Elasticsearch — equivalent to a database table. Documents are JSON objects. An index is distributed across shards.

Q307. What is an Elasticsearch shard?

A single Lucene instance that holds a subset of an index's documents. Primary shards handle writes; replica shards provide redundancy and read scalability.

Q308. What is Elasticsearch mapping?

Defines how fields in a document are stored and indexed — field data types (text, keyword, date, integer), analyzers, and index settings. Similar to a database schema.

Q309. What is the difference between `text` and `keyword` field types?

`text` is analyzed (tokenized, lowercased) — used for full-text search. `keyword` is not analyzed — used for exact matching, aggregations, and sorting.

Q310. What is an Elasticsearch template?

Defines settings and mappings applied automatically to indices matching a pattern. Used to enforce consistent mapping for time-series indices like `logs-*`.

Q311. What is Index Lifecycle Management (ILM) in Elasticsearch?

Automates the lifecycle of indices through phases: Hot (active writes), Warm (reduced resources), Cold (infrequent access), Frozen (compressed, searchable), Delete. Saves significant storage costs.

Q312. What is Logstash Grok filter?

A pattern-matching filter that parses unstructured log lines into structured fields using predefined or custom patterns.

```
grok {
  match => { "message" => "%{TIMESTAMP_ISO8601:timestamp} %{LOGLEVEL:level}
%{GREEDYDATA:msg}" }
}
```

Q313. What is the Kibana Query Language (KQL)?

A simplified query syntax for searching in Kibana. Example: `status: 500 AND service: "api"`
`AND @timestamp > "now-1h"`.

Q314. What is Elasticsearch's `_bulk` API?

Allows indexing, updating, or deleting multiple documents in a single API request — much more efficient than individual requests for high-volume log ingestion.

Q315. What is a Kibana Dashboard?

A collection of visualizations (charts, tables, maps, metrics) pinned to a shared time range. Used for operational monitoring and business intelligence.

Q316. What is the Beats family of log shippers?

Lightweight data shippers: Filebeat (logs), Metricbeat (system metrics), Packetbeat (network data), Auditbeat (audit data), Heartbeat (uptime monitoring). Written in Go, minimal resource footprint.

Q317. What is Elasticsearch Hot-Warm architecture?

Hot nodes have fast SSDs for recent, frequently queried data. Warm nodes have cheaper HDDs for older data. ILM automatically moves indices from hot to warm based on age or size.

Q318. What is Elasticsearch `doc_values`?

An on-disk data structure for efficient aggregations and sorting. Enabled by default for most field types. Disabling saves space but prevents aggregations on that field.

Q319. What is the ECS (Elastic Common Schema)?

A standardized field naming convention for logs and events across all Elastic products. Fields like `@timestamp`, `host.name`, `service.name`, `event.type` are defined and shared.

Q320. What is Logstash pipeline throughput tuning?

Key settings: `pipeline.workers` (parallel filter threads, set to CPU count), `pipeline.batch.size` (events per batch, higher = more throughput, more memory), `pipeline.batch.delay` (wait time for batch to fill).

Q321. What is Kibana Lens?

A drag-and-drop visualization builder in Kibana. Suggests chart types based on the data you select. Faster for building dashboards than the older visualization editor.

Q322. What is Elasticsearch cross-cluster search (CCS)?

Allows querying indices on remote Elasticsearch clusters from a local cluster. Used for searching across multiple environments or regions without merging clusters.

Q323. What is curator for Elasticsearch?

A command-line tool (now largely replaced by ILM) for managing Elasticsearch indices — deleting old indices, taking snapshots, closing cold indices.

Q324. What is an Elasticsearch snapshot?

A point-in-time backup of indices stored in a repository (S3, GCS, Azure Blob). Used for disaster recovery and migrating data between clusters.

Q325. What is the difference between Logstash and Fluentd?

Logstash: JVM-based, rich plugin ecosystem, higher resource usage. Fluentd: written in Ruby/C, lower memory footprint, better for Kubernetes (Fluentbit is even lighter). Both support multiple inputs, filters, and outputs.

Q326. What is Fluent Bit?

An ultra-lightweight log processor and forwarder written in C. ~1MB memory footprint — ideal as a DaemonSet in Kubernetes. Often used as the shipper with Fluentd or Loki as the aggregator.

Q327. What is Elasticsearch `refresh_interval`?

How often Elasticsearch refreshes the index to make new documents searchable. Default: 1 second. Setting it higher (e.g., 30s) during bulk indexing dramatically improves write throughput.

Q328. What is a Kibana Space?

A tenant-like isolation unit in Kibana. Each Space has its own dashboards, visualizations, and saved searches. Used to separate different teams or projects in a shared Kibana instance.

Q329. What is the Elastic Agent?

A unified agent that replaces individual Beats agents. Managed centrally via Fleet Server in Kibana. One agent handles logs, metrics, security events, and more.

Q330. What is Elasticsearch ingest pipeline?

A set of processors applied to documents before indexing. Similar to Logstash filters but executed inside Elasticsearch — grok, date, rename, remove, convert processors.

Q331. What is Elasticsearch's `cat` API?

A human-readable API for cluster monitoring.

```
curl "localhost:9200/_cat/indices?v" # List all indices
curl "localhost:9200/_cat/nodes?v"  # Node status
curl "localhost:9200/_cat/health?v" # Cluster health
```

Q332. What is a Kibana Watcher (formerly Alerting)?

A scheduled job that queries Elasticsearch and triggers actions (email, webhook, Slack) when conditions are met. Example: alert when error count exceeds 100 in 5 minutes.

Q333. What is Elasticsearch `fielddata`?

In-memory data structure for aggregations on `text` fields (analyzed). Very memory-intensive — prefer `keyword` fields for aggregations instead.

Q334. What is log correlation in ELK?

Linking logs, metrics, and traces using a common trace ID. When a request generates an error, you can jump from Kibana (logs) to Grafana (metrics) to Jaeger (traces) using the same correlation ID.

Q335. What is Elasticsearch security (formerly X-Pack)?

Built-in security features: TLS encryption, user authentication, role-based access control, audit logging, field-level security, and document-level security.

Prepared by Arvind Verma

SECTION 9: ANSIBLE (30 Questions)

Q336. What is an Ansible inventory?

A list of hosts (managed nodes) Ansible operates on. Can be static (INI/YAML file) or dynamic (cloud provider APIs via inventory plugins like `aws_ec2`).

Q337. What is an Ansible role?

A reusable unit of automation with a predefined directory structure: `tasks/`, `handlers/`, `templates/`, `files/`, `vars/`, `defaults/`, `meta/`. Roles organize related tasks into reusable packages.

Q338. What is an Ansible Galaxy?

A community hub for sharing and downloading Ansible roles and collections. `ansible-galaxy install geerlingguy.nginx` installs a community role for Nginx configuration.

Q339. What is `ansible-lint`?

A linting tool that checks Ansible playbooks and roles for style issues, best practices violations, and common mistakes. Run in CI pipelines before applying playbooks.

Q340. What is `check_mode` in Ansible?

Dry-run mode (`--check`). Ansible reports what changes would be made without actually making them. Combine with `--diff` to see exact file differences.

Q341. What is the `delegate_to` directive?

Runs a task on a different host than the one being managed. Useful for registering DNS, updating load balancers, or running commands on a bastion host.

```
- name: Register with load balancer
  command: add_host.sh {{ inventory_hostname }}
  delegate to: loadbalancer.internal
```

Q342. What is Ansible **block**?

Groups related tasks for error handling. Supports **rescue** (run on block failure) and **always** (always run).

```
block:
  - name: Deploy application
    command: ./deploy.sh
rescue:
  - name: Rollback
    command: ./rollback.sh
always:
  - name: Send notification
    slack: msg="Deploy completed"
```

Q343. What is the **lineinfile** module?

Ensures a specific line is present or absent in a file. Uses regex to find the line.

```
- lineinfile:
  path: /etc/ssh/sshd_config
  regexp: '^#MaxAuthTries'
  line: 'MaxAuthTries 3'
```

Q344. What is **with_items** vs **loop** in Ansible?

Both iterate over a list. **loop** is the modern replacement for **with_items** (Ansible 2.5+). **with_items** flattens nested lists; **loop** doesn't by default.

Q345. What is the **register** directive in Ansible?

Captures the output of a task into a variable for use in subsequent tasks.

```
- command: cat /etc/app/version
  register: app_version
- debug: msg="Running version {{ app_version.stdout }}"
```

Q346. What is **when** in Ansible?

A conditional that determines whether a task runs.

```
- name: Restart service (only on Ubuntu)
  service:
    name: nginx
    state: restarted
  when: ansible_os_family == "Debian"
```

Q347. What is Ansible Tower / AWX?

A web-based UI, REST API, and task engine for Ansible. Provides role-based access, job scheduling, credential management, and real-time output. AWX is the open-source upstream of Red Hat Ansible Tower.

Q348. What is an Ansible collection?

A distribution format for packaging roles, modules, plugins, and playbooks together. The modern way to share Ansible content, replacing standalone roles.

```
ansible-galaxy collection install amazon.aws
```

Q349. What is **ansible-pull**?

An inverted approach where the managed node pulls and runs playbooks from a Git repository. Used for self-configuring nodes (IoT, auto-scaling instances).

Q350. What is the **template** module?

Renders a Jinja2 template file and copies it to the target host. Variables from Ansible are available in the template.

```
- template:
  src: nginx.conf.j2
  dest: /etc/nginx/nginx.conf
```

```
validate: nginx -t -c %s
```

Q351. What is **become** in Ansible?

Privilege escalation — runs tasks as a different user (typically root).

```
become: true
become_user: root
become_method: sudo
```

Q352. What is Ansible facts?

System information automatically collected at playbook start: OS type, IP addresses, CPU count, memory, disk partitions. Available as **ansible_*** variables. Disable with **gather_facts: false** for faster runs.

Q353. What is **notify** and **handler** relationship?

A task with **notify** triggers a named handler at the end of the play (once, even if notified multiple times). Handlers run only when their task actually made a change.

Q354. What is the **copy** module vs **synchronize**?

copy transfers files via SSH — suitable for small files. **synchronize** uses rsync — much faster for large files or directories. **synchronize** requires rsync on both ends.

Q355. What is a **dynamic inventory** in Ansible?

An inventory script or plugin that queries external sources (AWS, GCP, Azure, Terraform state) to build the host list dynamically. AWS EC2 plugin: **amazon.aws.aws_ec2**.

Q356. What is **serial** in Ansible?

Controls how many hosts are processed at once in a play. **serial: 1** processes hosts one at a time — useful for rolling deployments with zero downtime.

```
- hosts: web_servers
```

```
serial: 25%    # Update 25% of servers at a time
```

Q357. What is `max_fail_percentage` in Ansible?

Sets the maximum percentage of failed hosts before aborting the entire play. Useful for rolling deployments — stop if too many hosts fail.

Q358. What is Molecule?

A testing framework for Ansible roles. Creates temporary instances (Docker, Vagrant, cloud VMs), runs the role, verifies the result, then destroys the instances.

Q359. What is `ansible-doc`?

Built-in documentation tool for Ansible modules.

```
ansible-doc service    # Full documentation for the service module
ansible-doc -l          # List all available modules
```

Q360. What is the `uri` module in Ansible?

Makes HTTP requests from managed hosts. Used for health checks, API calls, webhook triggers.

```
- uri:
  url: http://localhost:8080/health
  status_code: 200
  retries: 5
  delay: 10
```

Q361. What is `tags` in Ansible?

Labels attached to plays, roles, or tasks. Allows running only tagged tasks: `ansible-playbook site.yml --tags "deploy,nginx"`.

Q362. What is `vars_prompt` in Ansible?

Prompts the user for input at playbook runtime. Used for sensitive values you don't want in files (though Ansible Vault is preferred for secrets).

Q363. What is `include_tasks` vs `import_tasks`?

`import_tasks`: static inclusion — tasks are loaded at parse time. Tags and conditions apply to individual tasks. `include_tasks`: dynamic inclusion — evaluated at runtime. Allows conditionally including different task files.

Q364. What is `connection: local` in Ansible?

Runs tasks on the control node (where Ansible is run) instead of the remote host. Used for managing local files, calling APIs, or running local commands.

Q365. What is the `assert` module?

Validates conditions and fails with a custom message if they're not met. Used for input validation at the start of playbooks.

```
- assert:
  that:
    - app_version is defined
    - app_version | length > 0
  fail_msg: "app_version must be provided"
```

SECTION 10: TERRAFORM (40 Questions)

Q366. What is a Terraform provider?

A plugin that interacts with a cloud platform or service API (AWS, Azure, GCP, Kubernetes, GitHub). Providers define available resources and data sources.

Q367. What is `terraform.tfvars`?

A file that automatically populates variable values. Avoid committing this file if it contains sensitive values. Use `terraform.tfvars.example` as a template.

Q368. What is a Terraform data source?

Reads existing infrastructure information without managing it.

```
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"] # Canonical
  filter {
    name     = "name"
    values   = ["ubuntu/images/hvm-ssd/ubuntu-*22.04-amd64-server-*"]
  }
}
```

Q369. What is `terraform output`?

Displays output values defined in `outputs.tf`. Used to extract resource attributes (IP addresses, ARNs) after apply.

```
terraform output instance_public_ip
terraform output -json # Machine-readable format
```

Q370. What is `terraform taint`?

```
terraform apply -replace="aws_instance.web"
```

Q371. What is `terraform import`?

Brings existing infrastructure under Terraform management without recreating it.

```
terraform import aws_s3_bucket.my_bucket my-existing-bucket
```

Note: `import` only updates state — you still need to write the matching HCL configuration.

Q372. What is Terraform `count` vs `for_each`?

`count` creates N identical resources by index. `for_each` creates one resource per map key or set element — resources have stable identifiers based on keys, not index positions.

Prefer `for_each` for most cases.

Q373. What is `terraform validate`?

Checks the syntax and internal consistency of Terraform configuration without connecting to any API.

Q374. What is `terraform fmt`?

Formats Terraform files to the canonical style. Run `terraform fmt -check` in CI to enforce formatting.

Q375. What is the `depends_on` meta-argument?

Explicitly declares a dependency between resources that Terraform can't infer automatically.

```
resource "aws_ecs_service" "app" {  
  depends_on = [aws_alb_listener.http]  
}
```

Q376. What is `lifecycle { prevent_destroy = true }`?

Prevents `terraform destroy` from deleting the resource. Throws an error if you try. Use on databases, S3 buckets, and other critical resources.

Q377. What is `lifecycle { create_before_destroy = true }`?

Creates the replacement resource before destroying the old one. Required for resources that can't have a brief downtime (security groups, certificates).

Q378. What is `lifecycle { ignore_changes = [] }`?

Tells Terraform to ignore changes to specific attributes. Common use: `ignore_changes = [desired_count]` for ECS services managed by autoscaling.

Q379. What are Terraform local values?

Named expressions that can be reused within a module.

```
locals {
  common_tags = {
    Environment = var.environment
    ManagedBy   = "Terraform"
    Team        = "platform"
  }
}
```

Q380. What is a Terraform module source?

Where a module is loaded from: local path (`./modules/vpc`), Terraform Registry (`hashicorp/vpc/aws`), Git (`git::https://github.com/org/modules.git/vpc?ref=v1.2.0`).

Q381. What is `terraform workspace`?

Manages multiple state files from the same configuration. Use for simple multi-environment setups.

```
terraform workspace new staging
terraform workspace select production
```

```
terraform workspace list
```

Q382. What is `terraform graph`?

Generates a visual dependency graph in DOT format.

```
terraform graph | dot -Tsvg > graph.svg
```

Q383. What is `terraform state mv`?

Moves a resource within state — used when refactoring (renaming resources, moving into modules) without recreating infrastructure.

Q384. What is `terraform console`?

An interactive REPL for evaluating Terraform expressions. Useful for testing complex functions and expressions before using them in code.

Q385. What is the `templatefile()` function?

Renders a template file with variable substitution.

```
user_data = templatefile("${path.module}/user_data.sh.tpl", {  
  app_name = var.app_name  
  db_host  = aws_db_instance.main.endpoint  
})
```

Q386. What is Terragrunt?

A thin wrapper around Terraform that adds DRY configurations, remote state management, and module dependency management. Used in large multi-account, multi-region setups.

Q387. What is `terraform providers lock`?

Creates or updates `.terraform.lock.hcl` — a lock file for provider versions and checksums.

Ensures reproducible builds across machines.

Q388. What is **checkov** for Terraform?

A static analysis tool that scans Terraform, CloudFormation, and Kubernetes manifests for security misconfigurations and compliance violations.

```
checkov -d terraform/ --framework terraform
```

Q389. What is **terraform-docs**?

A tool that automatically generates documentation for Terraform modules from variables, outputs, and providers.

```
terraform-docs markdown . > README.md
```

Q390. What is **infracost**?

A CLI tool that estimates the cost of Terraform changes before applying them. Integrates with CI/CD to show cost impact on every PR.

Q391. What is Sentinel in Terraform Cloud, Enterprise?

A policy-as-code framework that enforces governance policies on Terraform runs — preventing non-compliant infrastructure from being created.

Q392. What is **terraform apply -target**?

Applies changes only to a specific resource and its dependencies. Use cautiously — can cause state drift if dependencies aren't considered.

```
terraform apply -target=aws_instance.web_server
```

Q393. What is a **null_resource** in Terraform?

A resource that does nothing by default but can trigger **local-exec** or **remote-exec** provisioners. Used for running scripts as part of the Terraform workflow.

Downloads and outputs the current state from the remote backend. Useful for inspection or manual editing.

```
terraform state pull > current-state.json
```

Q395. What is the **moved** block in Terraform?

Refactors resource addresses in state without recreating resources.

```
moved {  
  from = aws_instance.web  
  to   = module.servers.aws_instance.web  
}
```

Q396. What is Atlantis?

An open-source Terraform pull request automation tool. Runs **terraform plan** on PR open and **terraform apply** on PR merge. Provides PR comments with plan output.

Q397. What is **aws_caller_identity** data source?

Returns the AWS account ID, ARN, and user ID of the caller.

```
data "aws_caller_identity" "current" {}  
output "account_id" { value = data.aws_caller_identity.current.account_id }
```

Q398. What is Terraform's **precondition** and **postcondition**?

Validation checks on resource attributes.

```
lifecycle {  
  precondition {  
    condition      = var.instance_type != "t2.micro" || var.environment ==  
    "dev"  
    error_message = "t2.micro is only allowed in dev environment"  
  }  
}
```

```
}
```

Q399. What is `toset()`, `tolist()`, `tomap()` in Terraform?

Type conversion functions. `toset()` converts a list to a set (removes duplicates). Used with `for_each` which requires a set or map.

Q400. What is the Terraform Registry?

A public repository of providers and modules at registry.terraform.io. Verified providers and community modules are available for immediate use.

Q401. What is `terraform show`?

Displays human-readable output of a state file or plan file.

```
terraform show          # Current state
terraform show tfplan    # Saved plan details
```

Q402. What is the `one()` function in Terraform?

Converts a list with 0 or 1 elements to either null or a single value. Used to safely handle optional data sources.

Q403. What is a Terraform Backend?

Defines where and how state is stored and operations are performed. Backends: S3, GCS, Azure Blob (remote state), Terraform Cloud, local (default). Remote backends enable team collaboration.

Q404. What is `terraform refresh`?

Updates the state file to match the real-world infrastructure by querying the provider APIs. Detects drift without making changes.

Q405. What is Terratest?

A Go library for writing automated tests for Terraform modules. Deploys real infrastructure, runs assertions, then destroys it. The gold standard for infrastructure testing.

Prepared by Arvind Verma

SECTION 11: AZURE DEVOPS (25

Questions)

Q406. What is an Azure DevOps Service Connection?

A stored connection to an external service (AWS, Kubernetes, Docker Hub, GitHub). Pipelines use service connections instead of hardcoded credentials.

Q407. What is an Azure Pipeline YAML stage?

A major division of a pipeline, containing jobs. Stages run sequentially by default but can be configured to run in parallel. Each stage can have its own approval requirements.

Q408. What is an Azure DevOps variable group?

A reusable set of variables shared across pipelines. Can be linked to Azure Key Vault to automatically sync secrets.

Q409. What is Azure Artifacts?

A package management service supporting NuGet, npm, Maven, Python, and Universal Packages. Integrates with Azure Pipelines for publishing build artifacts.

Q410. What is a deployment environment in Azure Pipelines?

A target for deployments (staging, production) with approval checks, deployment history, and resource tracking (Kubernetes, VMs). Approvers must authorize deployment before it proceeds.

Q411. What is Azure Boards?

Work item tracking for Agile development — user stories, tasks, bugs organized into sprints. Integrates with Azure Repos for linking commits to work items.

Q412. What is a pipeline template in Azure DevOps?

A reusable YAML component referenced by other pipelines. Supports parameters for customization.

```
- template: templates/docker-build.yml@templates-repo
  parameters:
    imageName: my-api
    dockerfile: Dockerfile
```

Q413. What is Azure Container Registry (ACR)?

Microsoft's managed Docker registry, tightly integrated with AKS and Azure Pipelines. Supports geo-replication, content trust, and built-in vulnerability scanning via Microsoft Defender.

Q414. What is AKS (Azure Kubernetes Service)?

Microsoft's managed Kubernetes service. Azure manages the control plane for free; you pay for worker nodes. Integrates with AAD, ACR, Azure Monitor, and Azure Policy.

Q415. What is Azure Monitor?

Azure's full-stack monitoring platform — metrics, logs (Log Analytics), application performance (Application Insights), and alerts. The Azure equivalent of CloudWatch + X-Ray.

Q416. What is Application Insights?

An APM (Application Performance Monitoring) service built into Azure Monitor. Auto-instruments .NET, Java, Node.js apps for request tracking, dependency calls, exceptions, and custom metrics.

Q417. What is Azure Key Vault?

A managed service for storing and accessing secrets, keys, and certificates. Integrates with Azure Pipelines (variable groups), AKS (CSI driver), and Azure services (managed identity access).

Q418. What is a Managed Identity in Azure?

An automatically managed identity in Azure AD for Azure resources (VMs, AKS pods, Functions). Eliminates the need for credentials — the resource authenticates to Key Vault and other services automatically.

Q419. What is `az cli`?

Azure's command-line interface for managing all Azure resources.

```
az login
az aks get-credentials --resource-group myRG --name myAKS
az acr build --registry myACR --image myapp:latest .
```

Q420. What is Azure Pipelines `condition`?

Controls when a stage, job, or step runs.

```
condition: and(succeeded(), eq(variables['Build.SourceBranch'],
'refs/heads/main'))
```

Q421. What is Azure DevOps self-hosted agent?

A build agent you run on your own infrastructure. Used when builds need access to on-premises resources or specific software not available on Microsoft-hosted agents.

Q422. What is Azure Pipelines matrix strategy?

Runs a job multiple times with different variable combinations — equivalent to GitHub Actions matrix.

```
strategy:
  matrix:
    Python38:
      PYTHON_VERSION: '3.8'
    Python311:
      PYTHON_VERSION: '3.11'
```

Azure's infrastructure as code service using JSON templates. The Azure equivalent of CloudFormation. Being superseded by Bicep for better readability.

Q424. What is Bicep?

A domain-specific language for Azure infrastructure as code. Compiles to ARM templates. Much cleaner syntax than JSON ARM templates.

```
resource storageAccount 'Microsoft.Storage/storageAccounts@2023-01-01' = {  
  name: 'mystorageaccount'  
  location: 'eastus'  
  sku: { name: 'Standard_LRS' }  
  kind: 'StorageV2'  
}
```

Q425. What is Azure Policy?

Enforces organizational standards across Azure resources. Policies can audit, deny, or remediate non-compliant resources. Example: require all resources to have tags, deny creation of non-encrypted storage.

Q426. What is the Azure DevOps Publish Test Results task?

Publishes test results (JUnit, NUnit, xUnit) to Azure DevOps, making them visible in the pipeline run summary with pass/fail trends over time.

Q427. What is dependsOn in Azure Pipelines?

```
stages:  
  - stage: Deploy  
    dependsOn: Test  
    condition: succeeded('Test')
```

Defines execution order. A stage won't start until its dependencies complete successfully.

Q428. What is a pipeline artifact in Azure DevOps?

Files produced by a pipeline stage that are published for download or use in subsequent stages/pipelines.

```
- publish: $(Build.ArtifactStagingDirectory)
  artifact: drop
- download: current
  artifact: drop
```

Q429. What is Azure DevOps Wiki?

A collaborative documentation space within Azure DevOps. Supports Markdown, code samples, and diagram rendering. Can be backed by a Git repository.

Q430. What is the difference between Classic and YAML pipelines in Azure DevOps?

Classic pipelines are GUI-configured (legacy). YAML pipelines are code-based, stored in the repository, reviewable via PRs, and support advanced features like templates and multi-stage deployments. Always use YAML for new pipelines.

SECTION 12: GITHUB ACTIONS (25

Questions)

Q431. What is a GitHub Actions runner?

The machine that executes workflow jobs. GitHub provides hosted runners (ubuntu-latest, windows-latest, macos-latest). Self-hosted runners run on your own infrastructure.

Q432. What is a composite action?

A reusable action made of multiple steps defined in an `action.yml` file. Shareable across workflows and repositories.

Q433. What is the `jobs.<job_id>.outputs` feature?

Passes data from one job to another in the same workflow.

```
jobs:
  build:
    outputs:
      image-tag: ${{ steps.build.outputs.tag }}
  deploy:
    needs: build
    steps:
      - run: echo "Deploying ${{ needs.build.outputs.image-tag }}"
```

Q434. What is GitHub Actions concurrency?

Prevents multiple workflow runs from executing simultaneously for the same trigger.

```
concurrency:
  group: production-deploy-${{ github.ref }}
  cancel-in-progress: false # Don't cancel in-progress; queue instead
```

Q435. What is a `workflow_dispatch` trigger?

Allows manually triggering a workflow from the GitHub UI or API, optionally with input parameters.

Q436. What is `actions/upload-artifact` and `actions/download-artifact`?

Upload: saves files from a job to GitHub's artifact store. Download: retrieves those files in another job or workflow. Artifacts expire after 90 days by default.

Q437. What is `github.event_name`?

A context variable containing the event that triggered the workflow (`push`, `pull_request`, `schedule`, `workflow_dispatch`). Used in conditional expressions.

Q438. What is `env` context vs `secrets` context?

`env` contains environment variables set in the workflow. `secrets` contains encrypted secrets from GitHub Settings. Secrets are masked in logs; env vars are not.

Q439. What is the `permissions` key in GitHub Actions?

Controls the permissions granted to `GITHUB_TOKEN` for a workflow or job.

```
permissions:
  contents: read
  packages: write
  id-token: write # Required for OIDC
```

Q440. What is `pull_request_target` event?

Runs in the context of the target branch (base branch), not the PR branch. Has access to secrets — use carefully to avoid secret exposure from forked PRs.

Q441. What is GitHub Actions `timeout-minutes`?

Sets a maximum runtime for a job or step. Prevents hung workflows from consuming runner


```
jobs:
  test:
    timeout-minutes: 30
```

Q442. What is a self-hosted runner group?

A collection of self-hosted runners assigned to specific organizations or repositories. Used for access control — only certain repos can use certain runners.

Q443. What is `continue-on-error` in GitHub Actions?

Allows subsequent steps/jobs to run even if this step fails.

```
- name: Optional lint check
  run: npm run lint
  continue-on-error: true
```

Q444. What is the `strategy.fail-fast` option?

When `true` (default), cancels all in-progress matrix jobs if any job fails. Set to `false` to let all matrix jobs complete regardless.

Q445. What is `fromJson()` in GitHub Actions expressions?

Parses a JSON string into an object in expressions.

```
${{ fromJSON(steps.build.outputs.matrix) }}
```

Q446. What is GitHub Environments protection rule `required reviewers`?

Named individuals or teams who must approve a deployment before the job runs. The workflow pauses and sends a review request.

Q447. What is `actions/github-script`?

Runs JavaScript code with access to the GitHub API and workflow context. Used for commenting

Q448. What is the `push.paths` trigger filter?

Triggers the workflow only when specific file paths are changed.

```
on:
  push:
    paths:
      - 'src/**'
      - 'package.json'
    paths-ignore:
      - 'docs/**'
```

Q449. What is `hashFiles()` function?

Generates a hash of specified files — commonly used in cache keys

```
key: ${{ runner.os }}-node-${{ hashFiles('**/package-lock.json') }}
```

Q450. What is a GitHub App vs Personal Access Token for CI/CD?

GitHub Apps have fine-grained permissions per repo, short-lived tokens, and an audit trail. PATs have broad user permissions, long-lived, and tied to an individual account. Always use GitHub Apps for production CI/CD.

Q451. What is `workflow_run` trigger?

Triggers a workflow when another workflow completes.

```
on:
  workflow_run:
    workflows: ["CI Pipeline"]
    types: [completed]
    branches: [main]
```

Q452. What is the GitHub Actions `runner.os` context?

Returns the operating system of the runner (Linux, Windows, macOS). Used in cross-platform workflows.

Q453. What is `github.sha` vs `github.ref`?

`github.sha` is the full commit SHA (40 characters) — used for image tagging. `github.ref` is the branch/tag reference (`refs/heads/main`) — used in conditional logic.

Q454. What is `dorny/paths-filter` action?

A popular community action that detects which paths changed, enabling conditional job execution in monorepos.

```
- uses: dorny/paths-filter@v3
  id: changes
  with:
    filters: |
      api:
        - 'services/api/**'
      web:
        - 'services/web/**'
```

Q455. What is GitHub Actions OpenID Connect (OIDC)?

Allows workflows to authenticate to cloud providers (AWS, Azure, GCP) without storing long-lived credentials. The workflow exchanges a short-lived JWT token for cloud credentials.

SECTION 13: SECURITY IN DEVOPS /

DEVSECOPS (25 Questions)

Q456. What is DevSecOps?

Integrating security practices into every phase of the DevOps lifecycle — "shift left" means catching security issues during development, not after deployment.

Q457. What is SAST (Static Application Security Testing)?

Analyzes source code without executing it. Tools: SonarQube, Semgrep, CodeQL, Bandit (Python), Gosec (Go). Catches SQL injection, hardcoded secrets, insecure deserialization.

Q458. What is DAST (Dynamic Application Security Testing)?

Tests a running application by simulating attacks. Tools: OWASP ZAP, Burp Suite. Discovers vulnerabilities that only appear at runtime (XSS, authentication issues).

Q459. What is SCA (Software Composition Analysis)?

Identifies vulnerabilities in open-source dependencies. Tools: Snyk, OWASP Dependency Check, Trivy `--security-checks vuln`. Critical for Log4Shell-type vulnerabilities.

Q460. What is a CVE?

Common Vulnerabilities and Exposures — a public database of known security vulnerabilities, each with a unique ID (e.g., CVE-2021-44228 for Log4Shell) and severity score (CVSS).

Q461. What is the OWASP Top 10?

The 10 most critical web application security risks: Broken Access Control, Cryptographic Failures, Injection, Insecure Design, Security Misconfiguration, Vulnerable Components, Authentication Failures, Integrity Failures, Security Logging Failures, SSRF.

Automatically detects and alerts on accidentally committed secrets (API keys, passwords, tokens) by scanning commits and repository content against known secret patterns.

Q463. What is `gitleaks`?

An open-source secret scanner for Git repositories. Run as a pre-commit hook or in CI pipelines.

```
gitleaks detect --source . --verbose
gitleaks protect --staged # Scan staged changes before commit
```

Q464. What is image signing and verification?

Cryptographically signing Docker images to verify they haven't been tampered with. Tools: cosign (Sigstore), Notary. Kubernetes can enforce that only signed images are allowed via admission controllers.

Q465. What is Cosign?

A tool for signing, verifying, and storing signatures for container images and other artifacts. Part of the Sigstore project — enables keyless signing using OIDC identity.

Q466. What is `falco`?

A runtime security tool for Kubernetes. Detects anomalous behavior at runtime: unexpected processes, unauthorized file system writes, network connections from unexpected containers.

Q467. What is a Software Bill of Materials (SBOM)?

A formal list of all components, dependencies, and their versions in a software artifact. Generated by tools like Syft, CycloneDX. Increasingly required for supply chain security compliance.

Q468. What is the principle of immutable infrastructure?

Never modifying running servers — replace them with new ones instead. Eliminates configuration drift and makes deployments reproducible. Containers and auto-scaling groups make this practical.

Q469. What is kube-bench?

A Go tool that checks whether Kubernetes is deployed according to the CIS (Center for Internet Security) Kubernetes Benchmark. Checks API server settings, etcd configuration, kubelet security, and more.

Q470. What is kube-hunter?

A penetration testing tool for Kubernetes. Actively probes the cluster for exploitable vulnerabilities and misconfigurations.

Q471. What is Aqua Security?

An enterprise container security platform covering image scanning, runtime protection, secrets management, and Kubernetes network policies.

Q472. What is conftest?

A tool for writing tests for configuration files (Kubernetes manifests, Terraform, Dockerfile) using Open Policy Agent (OPA) Rego policies.

```
conftest test deployment.yaml
```

Q473. What is terrascan?

A static code analyzer for IaC (Terraform, CloudFormation, Kubernetes) that detects security vulnerabilities and compliance violations.

Q474. What is network segmentation in Kubernetes?

Using NetworkPolicies to isolate workloads — frontend pods can only talk to API pods, API pods can only talk to database pods, no direct frontend-to-database communication.

Q475. What is mTLS (mutual TLS)?

Both client and server authenticate each other using certificates. In Kubernetes service meshes (Istio, Linkerd), mTLS is automatically applied between all services — zero-trust networking.

Q476. What is a container runtime security policy?

Rules enforced at container startup — prevents privileged containers, host network/pid namespace sharing, root user, writable root filesystem. Enforced by OPA Gatekeeper, Kyverno, or PSA.

Q477. What is software supply chain security?

Securing the entire software delivery pipeline from source code to production. Covers: signed commits, signed images, SBOM generation, dependency scanning, and SLSA (Supply-chain Levels for Software Artifacts) compliance.

Q478. What is SLSA (Supply-chain Levels for Software Artifacts)?

A security framework with four levels of increasing supply chain security. Level 1: build process documented. Level 2: version-controlled, auditable. Level 3: hardened build platform. Level 4: two-party review of all changes.

Q479. What is `trufflehog`?

A secret detection tool that scans Git history deeply, including commit diffs, for sensitive data like API keys, passwords, and private keys.

Q480. What is `semgrep`?

A static analysis tool for finding bugs and security issues using pattern-based rules. Faster and more customizable than traditional SAST tools. Rules can be written in YAML without knowing the language internals.

SECTION 14: MONITORING,

OBSERVABILITY & SRE (20 Questions)

Q481. What is the difference between monitoring and observability?

Monitoring tells you *when* something is wrong (dashboards, alerts). Observability tells you *why* — by exploring metrics, logs, and traces together. Observability requires no pre-defined questions.

Q482. What are the three pillars of observability?

Metrics: numeric measurements over time (CPU, request rate, error rate). **Logs:** event records with context (structured JSON logs). **Traces:** end-to-end journey of a request through multiple services.

Q483. What is SRE (Site Reliability Engineering)?

A discipline applying software engineering principles to infrastructure operations. Core concepts: SLI/SLO/SLA, error budgets, toil reduction, blameless postmortems.

Q484. What is an SLI (Service Level Indicator)?

A quantitative measure of service behavior: request latency, error rate, availability, throughput. The raw metric you measure.

Q485. What is an SLO (Service Level Objective)?

A target value for an SLI. Example: "P99 latency < 200ms for 99.9% of requests over a rolling 30-day window."

Q486. What is an SLA (Service Level Agreement)?

A contractual commitment to customers with consequences for breach. SLOs are internal targets; SLAs are external commitments. SLOs should be stricter than SLAs.

The allowed amount of unreliability in a service. $\text{Error Budget} = 1 - \text{SLO}$. If SLO = 99.9%, error budget = 0.1% = 43.8 minutes/month of allowed downtime. When the error budget is exhausted, new feature deployments pause.

Q488. What is MTTR (Mean Time To Recovery)?

Average time to restore service after a failure. A key SRE metric. Reduced by better alerting, automated rollbacks, runbooks, and practiced incident response.

Q489. What is MTBF (Mean Time Between Failures)?

Average time between failures. Increased by improving reliability, adding redundancy, and thorough testing.

Q490. What is toil in SRE?

Manual, repetitive, automatable operational work that scales with service growth. SRE teams aim to keep toil below 50% of their time, spending the rest on engineering to eliminate it.

Q491. What is a blameless postmortem?

A postmortem that focuses on system and process failures, not individual blame. The goal is learning and improvement, not punishment. Psychological safety is essential for honest reporting.

Q492. What is synthetic monitoring?

Proactive monitoring using scripted user journeys to test application functionality from external locations. Detects issues before real users encounter them. Tools: Grafana k6 Cloud, Pingdom, Datadog Synthetics.

Q493. What is real user monitoring (RUM)?

Collecting performance data from actual user browsers/devices. Captures real-world performance (not synthetic) including slow networks and diverse devices.

Q494. What is OpenTelemetry?

A CNCF project providing a standardized API, SDK, and tooling for collecting metrics, logs, and traces. Vendor-neutral — instrument once, send to any backend (Jaeger, Grafana, Datadog).

Q495. What is Jaeger?

An open-source distributed tracing platform. Collects traces, visualizes request flows through microservices, identifies bottlenecks and failures. CNCF graduated project.

Q496. What is PagerDuty?

An on-call management and incident response platform. Routes alerts to the correct on-call engineer, manages escalations, tracks incident history, and generates on-call schedules.

Q497. What is the USE method?

A resource analysis methodology by Brendan Gregg: **U**tilization, **S**aturation, **E**rrors. For every resource (CPU, memory, disk, network), check these three metrics to identify bottlenecks.

Q498. What is the RED method?

A service-level methodology: **R**ate (requests per second), **E**rrors (failed requests per second), **D**uration (latency distribution). Applied to every service endpoint for health monitoring.

Q499. What is Datadog?

A comprehensive SaaS monitoring platform covering infrastructure metrics, APM, logs, synthetics, and security. All-in-one alternative to self-managed Prometheus + Grafana + ELK stacks.

Q500. What is New Relic?

An observability platform offering APM, distributed tracing, infrastructure monitoring, and synthetic monitoring as a SaaS solution. Strong .NET and Java monitoring capabilities.

SECTION 15: PRODUCTION PIPELINE

ADDITIONAL TOOLS (20 Questions)

Q501. What is k6?

An open-source load testing tool by Grafana Labs. Tests are written in JavaScript.

```
k6 run --vus 100 --duration 60s script.js
# 100 virtual users for 60 seconds
```

Q502. What is Locust?

A Python-based load testing tool. Define user behavior in Python, then simulate thousands of concurrent users. Easy to write complex test scenarios.

Q503. What is Pact (contract testing)?

A framework for consumer-driven contract testing between microservices. The consumer defines what it expects from the provider; the provider verifies it can meet those expectations without a running environment.

Q504. What is Nexus Repository Manager?

An artifact repository manager supporting Docker, Maven, npm, PyPI, NuGet, Helm, and more. Acts as a proxy for external registries and as a private registry for internal artifacts.

Q505. What is JFrog Artifactory?

An enterprise-grade artifact management platform. Supports more formats than Nexus, stronger HA capabilities, and deep integration with JFrog's security tools (Xray for vulnerability scanning).

Q506. What is SonarCloud?

The cloud-hosted version of SonarQube. Integrates directly with GitHub, GitLab, and Azure DevOps without self-hosting. Free for public repositories.

Q507. What is Snyk?

A developer security platform that finds and fixes vulnerabilities in code, open-source dependencies, containers, and IaC. Integrates natively with GitHub, GitLab, Jenkins, and IDEs.

Q508. What is OWASP ZAP?

Zed Attack Proxy — a free, open-source DAST tool for finding security vulnerabilities in web applications. Can be run in CI pipelines in headless mode.

```
docker run -t owasp/zap2docker-stable zap-baseline.py -t  
https://staging.myapp.com
```

Q509. What is Chaos Monkey?

Netflix's tool that randomly terminates production instances to test system resilience. The original chaos engineering tool. Modern alternatives: Chaos Mesh, Litmus Chaos (Kubernetes-native).

Q510. What is Litmus Chaos?

A CNCF project for chaos engineering on Kubernetes. Provides pre-built chaos experiments: pod delete, node drain, network latency, CPU stress, disk fill.

Q511. What is kustomize?

A Kubernetes-native configuration management tool. Allows customizing YAML manifests without templating — uses patches and overlays.

```
kustomize build k8s/overlays/production | kubectl apply -f -
```

Q512. What is Skaffold?

A command-line tool that automates the build-push-deploy cycle during local Kubernetes development. Detects file changes, rebuilds images, and redeploys automatically.

Q513. What is Tilt?

A local Kubernetes development environment tool. Watches source code, rebuilds/syncs changes to running containers without full rebuilds. Faster iteration than Skaffold for some workflows.

Q514. What is telepresence?

Allows running a single service locally while connected to a remote Kubernetes cluster. Your local service intercepts traffic meant for the remote service — debug with real traffic and dependencies.

Q515. What is Tekton?

A Kubernetes-native CI/CD framework. Builds pipelines from reusable steps (**Task**), sequences of steps (**Pipeline**), and event triggers (**EventListener**). No external server — runs entirely in Kubernetes.

Q516. What is Spinnaker?

An open-source multi-cloud continuous delivery platform (Netflix/Google). Specializes in deployment strategies (Blue/Green, Canary), across AWS, GCP, Azure, and Kubernetes.

Q517. What is GitLab's Container Registry?

A built-in Docker registry in GitLab, automatically integrated with GitLab CI/CD. Every project gets its own registry namespace: registry.gitlab.com/group/project.

Q518. What is Portainer?

A lightweight management UI for Docker and Kubernetes. Provides a web interface for managing containers, images, volumes, and networks without using the CLI.

Q519. What is act?

A tool that runs GitHub Actions workflows locally using Docker. Test your workflows without pushing to GitHub.

```
act -j build    # Run the 'build' job locally
```

Q520. What is GitHub's CodeQL?

A semantic code analysis engine that treats code as data. Used for finding security vulnerabilities by writing queries in QL. Powers GitHub's code scanning feature.

Prepared by Arvind Verma

SECTION 16: NETWORKING & CLOUD

FUNDAMENTALS (20 Questions)

Q521. What is DNS and how does it work?

Domain Name System translates human-readable hostnames (`api.myapp.com`) to IP addresses.

Resolution: browser → local cache → OS resolver → recursive resolver → authoritative nameserver.

Q522. What is a CDN (Content Delivery Network)?

A geographically distributed network of servers that caches content close to users. Reduces latency, absorbs traffic spikes. Examples: CloudFront (AWS), Cloudflare, Akamai.

Q523. What is a reverse proxy?

A server that sits in front of application servers and forwards client requests. Provides: SSL termination, load balancing, caching, compression, and security. Examples: Nginx, HAProxy, Traefik.

Q524. What is the difference between TCP and UDP?

TCP: connection-oriented, reliable (acknowledgments, retransmission), ordered delivery. Slower.

UDP: connectionless, no guarantee of delivery or order. Faster — used for streaming, gaming, DNS.

Q525. What is a subnet mask?

Defines which part of an IP address is the network portion and which is the host portion. `/24` = 255.255.255.0 = 256 addresses. `/16` = 65,536 addresses.

Q526. What is CIDR notation?

Classless Inter-Domain Routing — a compact way to describe IP ranges. **10.0.0.0/8** means the first 8 bits are network (16.7 million addresses). Used extensively in VPC/subnet design.

Q527. What is a bastion host?

A hardened, publicly accessible server that provides SSH access to private network resources. The only entry point to private infrastructure. Also called a jump host.

Q528. What is TLS termination?

Decrypting TLS-encrypted traffic at a load balancer or reverse proxy, then forwarding unencrypted traffic to backend servers. Reduces CPU load on application servers.

Q529. What is HTTP/2?

Second major version of HTTP. Key improvements: multiplexing (multiple requests over one connection), header compression, server push. Supported by Nginx, Kubernetes Ingress, ALB.

Q530. What is gRPC?

A high-performance RPC framework using HTTP/2 and Protocol Buffers. Used for microservice-to-microservice communication. Significantly faster than REST/JSON for internal services.

Q531. What is a service mesh's east-west traffic?

Communication between services within the cluster (internal). Contrasted with north-south traffic (external → cluster). Service meshes primarily manage east-west traffic.

Q532. What is eBPF?

Extended Berkeley Packet Filter — a technology for running sandboxed programs in the Linux kernel. Enables high-performance networking, observability, and security without kernel modules. Powers Cilium (Kubernetes networking) and Falco.

Q533. What is Cilium?

A Kubernetes CNI (Container Network Interface) plugin that uses eBPF for high-performance networking, load balancing, and security policies. Provides better performance and visibility than traditional iptables-based networking.

Q534. What is CoreDNS?

The default DNS server for Kubernetes. Resolves service names to ClusterIP addresses. Configured via Corefile — supports plugins for custom DNS behavior.

Q535. What is iptables vs ipvs in Kubernetes?

Both are kube-proxy modes for implementing Service load balancing. `iptables` (default): sequential rule matching, slower at scale. `ipvs` (kernel-level): hash table lookups, O(1) performance, better for clusters with thousands of services.

Q536. What is the difference between Layer 4 and Layer 7 load balancing?

Layer 4 (TCP/UDP): routes based on IP/port, fast, no payload inspection. Layer 7 (HTTP): routes based on URL, headers, cookies — enables path-based routing, A/B testing, and SSL termination.

Q537. What is a TCP SYN flood attack?

A DDoS attack that overwhelms a server by sending many TCP SYN packets without completing the handshake. Exhausts connection tables. Mitigated by SYN cookies and rate limiting.

Q538. What is connection pooling?

Reusing existing database/HTTP connections instead of creating new ones for each request. Dramatically reduces connection overhead. Tools: pgBouncer (PostgreSQL), HikariCP (Java).

Q539. What is a health check in load balancing?

Periodic probes (TCP, HTTP) to verify backend servers are healthy. Load balancers only route traffic to healthy backends, automatically removing failed servers.

Q540. What is BGP (Border Gateway Protocol)?

The routing protocol of the internet. Used internally in Kubernetes by Calico and MetalLB for advertising Pod and Service IPs to network routers — enabling bare-metal Kubernetes networking.

Prepared by Arvind Verma

SECTION 17: DATABASES IN DEVOPS (10

Questions)

Q541. What is a database connection string best practice in containerized environments?

Store in Secrets Manager or Vault, inject at runtime via environment variable or volume mount. Never hardcode in application code or Docker image. Use connection pooling to handle multiple container instances.

Q542. What is pgBouncer?

A lightweight connection pooler for PostgreSQL. Maintains a pool of persistent database connections and multiplexes application connections onto them — critical when running many Kubernetes pods.

Q543. What is a read replica?

A copy of the primary database that handles read-only queries. Reduces load on the primary, improves read performance, and provides a standby for failover. AWS RDS Multi-AZ provides automatic failover.

Q544. What is database sharding?

Horizontally partitioning data across multiple database instances. Each shard contains a subset of data (e.g., users 1-1M on shard 1, 1M-2M on shard 2). Enables horizontal scaling beyond single-node limits.

Q545. What is the CAP theorem?

A distributed system can only guarantee two of three: **C**onsistency (all nodes see the same data), **A**vailability (every request gets a response), **P**artition tolerance (system works despite network failures). In practice, partition tolerance is required, so systems choose CP (Zookeeper, etcd) or AP (Cassandra, DynamoDB).

Q546. What is a database migration rollback strategy?

Write backward-compatible migrations so both old and new code versions can run against the same schema simultaneously. Use feature flags to control which code path runs. Never drop columns in the same release that removes the code using them.

Q547. What is RDS IAM authentication?

Authenticate to RDS using IAM tokens instead of database passwords. The application calls `generate-db-auth-token` and uses the token as the password — tokens expire in 15 minutes.

Q548. What is DynamoDB's partition key?

The primary attribute used to distribute data across partitions. Poor partition key choice (e.g., using date/status with low cardinality) causes "hot partitions" — all traffic goes to one partition, causing throttling.

Q549. What is Redis Sentinel vs Redis Cluster

Sentinel: high availability for a single Redis instance — automatic failover but no horizontal scaling. Cluster: horizontally shards data across multiple nodes — both HA and scalability.

Q550. What is `pg_dump` and how do you automate database backups?

```
pg_dump -h $DB_HOST -U $DB_USER -d mydb -Fc > backup-$(date +%Y%m%d).dump
# Upload to S3
aws s3 cp backup-$(date +%Y%m%d).dump s3://my-backups/postgres/
# Automate as Kubernetes CronJob:
# schedule: "0 2 * * *" # Daily at 2 AM
```

FINAL SECTION: SCENARIO-BASED & BEHAVIORAL QUESTIONS

Q551. Your Docker image build takes 20 minutes. How do you reduce it?

1. Add `.dockerignore`. 2) Reorder layers — copy `package.json` before source code. 3) Use multi-stage builds. 4) Enable BuildKit cache mounts. 5) Use layer caching in CI (`--cache-from`). 6) Use a smaller base image (alpine). Target: under 5 minutes.

Q552. A pod is in `CrashLoopBackOff`. How do you debug it?

```
kubectl describe pod <name>          # Check events and last state reason
kubectl logs <pod> --previous         # Logs from the crashed container
kubectl logs <pod> -c <container>     # Specific container logs
# Common causes: missing env vars, DB connection failure, OOMKill, bad CMD
```

Q553. Terraform plan shows a resource will be destroyed unexpectedly. What do you do?

1. Read the plan carefully — understand *why* it's being destroyed. 2) Check if it's a `force replace` due to immutable field change. 3) Use `terraform state mv` if it's a rename. 4) Check `lifecycle.create_before_destroy`. 5) **Never apply without understanding the cause.**

Q554. Your CI pipeline is flaky — tests pass locally but fail in CI 30% of the time. What are the causes?

1. Tests depend on external services (network, time). 2) Parallel test interference (shared database state). 3) Resource constraints (OOM in CI). 4) Timezone differences. 5) Non-deterministic test order. Fix: mock externals, isolate test databases, use `--runInBand`, set timeouts.

Q555. How do you migrate a monolith application to microservices with zero downtime?

Use the Strangler Fig pattern: 1) Put a proxy/API gateway in front of the monolith. 2) Extract one service at a time — start with the least coupled. 3) Route traffic for the extracted feature to the new service via the proxy. 4) Verify with feature flags. 5) Repeat until monolith is empty. Never do a big-bang rewrite.

Q556. Your Kubernetes cluster nodes are running at 90% memory. What do you do?

Immediate: 1) `kubectl top pods --sort-by=memory` — find memory hogs. 2) Check for OOMKilled pods. 3) Scale up node group. Long-term: 4) Set proper resource requests/limits. 5) Run VPA to get recommendations. 6) Enable Cluster Autoscaler. 7) Optimize application memory usage.

Q557. How do you roll back a bad database migration?

1. If backward-compatible migration: redeploy old app code — it works with new schema.
- 2) If breaking change was applied: run the rollback script (Liquibase rollback, Flyway undo).
- 3) Last resort: restore from pre-migration snapshot. Lesson: always test migrations on a production-size staging copy first.

Q558. Your Prometheus alerts are firing constantly (alert fatigue). How do you fix it?

1. Audit alerts — remove ones nobody acts on. 2) Increase `for:` duration on flapping alerts.
- 3) Add inhibition rules. 4) Group related alerts. 5) Route low-severity to Slack, critical to PagerDuty. 6) Add runbook links to every alert. Goal: every alert should be actionable and have a documented response.

Q559. A developer accidentally pushed a secret to GitHub. What do you do?

1. **Immediately revoke** the secret at the source (rotate the API key, change the password).
- 2) Remove from current code via new commit. 3) Purge from Git history (`git filter-repo` or BFG Repo Cleaner). 4) Force-push (only on non-protected branches). 5) Audit access logs for unauthorized use during exposure window. 6) Enable secret scanning to prevent recurrence.

Q560. How do you implement feature flags in a CI/CD pipeline?

Use a feature flag service (LaunchDarkly, Unleash, or custom Redis-backed). In the pipeline: 1) Merge incomplete features behind a `false` flag. 2) Deploy to production — feature is inactive. 3) Enable flag for internal users (dogfooding). 4) Gradually roll out by user percentage. 5) Full rollout. 6) Remove flag and code once stable. This separates *deployment* from *release*.

Pro Tip for Interviews: Don't just memorize answers — understand the *why* behind each tool. The best interviews are conversations, not recitations. When you say "we chose X over Y because of Z constraint," you demonstrate real-world judgment that no amount of memorization can fake.

Prepared by Arvind Verma

**ADVANCE
SECTION:
HAVING
HANDS-ON
QUESTIONS**

Prepared by Arvind Verma

CHAPTER 1: JENKINS

Introduction to Jenkins

Jenkins is the grandfather of CI/CD automation. Released in 2011 as a fork of Hudson, it has become the most widely deployed open-source automation server in the world. When someone says "we have a pipeline," there's a good chance Jenkins is somewhere in that picture.

Understanding Jenkins deeply — not just its UI, but its architecture, security model, and pipeline syntax — is what separates a DevOps engineer from someone who just presses buttons.

Q1. [Beginner] What is Jenkins, and why is it used in DevOps?

Answer:

Jenkins is an open-source automation server written in Java. Its primary job is to automate the repetitive parts of software development — specifically building, testing, and deploying code. In the context of DevOps, Jenkins sits at the heart of the **CI/CD pipeline** (Continuous Integration / Continuous Delivery).

Here's how to think about it: Every time a developer pushes code to a repository like GitHub, Jenkins can automatically:

1. Pull that code
2. Compile or build it
3. Run automated tests
4. Package it into a deployable artifact (like a Docker image or JAR file)
5. Deploy it to a staging or production environment

Why this matters: Before CI/CD tools like Jenkins, teams would "integrate" code once a week or once a sprint. By then, hundreds of conflicting changes had piled up, causing what developers call "integration hell." Jenkins solves this by integrating continuously — every commit, every day.

In an interview, say something like this:

"Jenkins is an automation server that enables Continuous Integration and Continuous Delivery. We use it to automate the software delivery pipeline — from code commit to production deployment. In practice, it reduces human error, speeds up release cycles, and gives teams fast feedback when something breaks."

Pro Tip: Jenkins itself doesn't do CI/CD. Jenkins is the *orchestrator*. The actual work is done by tools it calls — Maven, Docker, kubectl, etc. Always make this distinction clear in an interview.

Q2. [Beginner] What is the difference between Continuous Integration, Continuous Delivery, and Continuous Deployment?

Answer:

These three terms are often confused, even by experienced engineers. Here's the clearest way to explain them:

Continuous Integration (CI): The practice of automatically building and testing code every time a developer pushes a change. The goal is to detect bugs *early*, when they are cheap to fix.

- Trigger: Code push to repository
- Actions: Build → Unit Test → Code Quality Scan
- Output: Pass/Fail signal within minutes

Continuous Delivery (CD - Delivery): Extends CI by automatically preparing the code for release to production. The key word here is *prepared* — the deployment to production may still require a human to press a button.

- Trigger: Successful CI build
- Actions: Integration Tests → Staging Deploy → Smoke Tests
- Output: A release-ready artifact that *can* go to production anytime

Continuous Deployment (CD - Deployment): The most aggressive form. Every change that passes all automated tests goes *automatically* to production without any human intervention.

- Trigger: Successful staging tests
- Actions: Production Deploy → Health Checks → Rollback if needed

- Output: Code in production within minutes of commit

A practical analogy: Think of a car assembly line.

- CI = Quality checks at each station
- Continuous Delivery = Car is finished and parked, ready to ship
- Continuous Deployment = Car drives itself to the customer's driveway

Common Pitfall: Many teams say they do "Continuous Deployment" but actually do "Continuous Delivery." The distinction matters in interviews — be precise.

Q3. [Beginner] What is a Jenkinsfile, and why should it be stored in version control?

Answer:

A **Jenkinsfile** is a text file that contains the definition of a Jenkins pipeline. Instead of configuring your pipeline through the Jenkins web UI (which is fragile, hard to audit, and doesn't survive server crashes), you define your entire pipeline as code in a file called **Jenkinsfile** and commit it to your repository.

Basic Jenkinsfile example (Declarative syntax):

```
pipeline {
  agent any
  environment {
    APP_NAME = 'my-web-app'
    DOCKER_IMAGE = "myregistry/${APP_NAME}:${BUILD_NUMBER}"
  }
  stages {
    stage('Checkout') {
      steps {
        git branch: 'main', url: 'https://github.com/myorg/my-web-app.git'
      }
    }
  }
}
```

```

        steps {
            sh 'mvn clean package -DskipTests'
        }
    }
    stage('Test') {
        steps {
            sh 'mvn test'
        }
        post {
            always {
                junit 'target/surefire-reports/*.xml'
            }
        }
    }
    stage('Docker Build & Push') {
        steps {
            script {
                docker.build(DOCKER_IMAGE)
                docker.withRegistry('https://myregistry', 'docker-
credentials') {
                    docker.image(DOCKER_IMAGE).push()
                }
            }
        }
    }
    stage('Deploy to Staging') {
        steps {
            sh "kubectl set image deployment/my-web-app
app=${DOCKER_IMAGE} -n staging"
        }
    }
}
post {
    failure {
        mail to: 'team@company.com',
            subject: "Pipeline Failed: ${currentBuild.fullDisplayName}",
            body: "Check Jenkins: ${env.BUILD_URL}"
    }
}

```

```
}  
}
```

Why store it in version control?

- **Auditability:** You can see exactly when and why the pipeline changed
- **Disaster recovery:** If Jenkins crashes, your pipeline definition isn't lost
- **Code review:** Pipeline changes go through the same PR process as application code
- **Branch-specific pipelines:** Different branches can have different pipeline behaviors

Operational Lesson: Teams that configure pipelines through the UI almost always regret it. One server migration or accidental click and the pipeline is gone. Pipeline-as-Code is non-negotiable in mature DevOps teams.

Q4. [Intermediate] Explain the difference between Declarative and Scripted pipelines in Jenkins.

Answer:

Jenkins supports two syntaxes for writing pipelines, and understanding when to use each is a sign of real-world experience.

Declarative Pipeline: Introduced in Jenkins 2.x, this is the *recommended* approach for most teams. It has a rigid, opinionated structure that enforces best practices and is easier to read.

```
pipeline {  
  agent any  
  stages {  
    stage('Build') {  
      steps {  
        sh 'make build'  
      }  
    }  
  }  
}
```

Key characteristics:

- Must start with the **pipeline** block
- Has predefined sections: **agent**, **stages**, **steps**, **post**
- Built-in validation — Jenkins will tell you if the structure is wrong
- Supports **when** conditions, **parallel** stages, **input** steps natively
- Easier for beginners

Scripted Pipeline: The older format, based on Apache Groovy. It gives you full programmatic control but at the cost of complexity.

```
node {  
    stage('Build') {  
        sh 'make build'  
    }  
    stage('Test') {  
        try {  
            sh 'make test'  
        } catch (e) {  
            currentBuild.result = 'FAILURE'  
            throw e  
        }  
    }  
}
```

Key characteristics:

- Starts with **node** block
- Full Groovy programming — loops, conditionals, functions, classes
- No structural validation — errors only appear at runtime
- More flexible but harder to maintain
- Useful for very complex, dynamic pipeline logic

When to use which:

Scenario	Recommendation
----------	----------------

Standard build/test/deploy pipeline	Declarative
Dynamic stage generation based on file system	Scripted
New team, mixed experience levels	Declarative
Complex conditional branching logic	Scripted or Declarative with <code>script {}</code> block
Shared library development	Scripted

Pro Tip: In modern Jenkins, you can embed Scripted logic inside a Declarative pipeline using the `script {}` block. This gives you the best of both worlds — structured outer pipeline with programmatic inner logic where needed.

Q5. [Intermediate] What are Jenkins Agents/Nodes, and how do they work in a distributed build environment?

Answer:

By default, Jenkins runs on a single server called the **Master** (now called the **Controller** in newer Jenkins terminology). For small teams, this is fine. But in production environments, you almost always need multiple agents for:

- Running builds in parallel
- Using different operating systems (Linux agent for building, Windows agent for .NET)
- Isolating sensitive workloads
- Scaling build capacity

How it works:

The Jenkins Controller manages the overall orchestration — it reads the Jenkinsfile, schedules jobs, and records results. The actual build work happens on **Agents** (also called Nodes or Workers). Agents connect to the controller via JNLP (Java Web Start) or SSH.

```
Jenkins Controller
|
|--- Agent 1 (Linux, 8 CPU) — runs Java builds
```

```
|--- Agent 2 (Linux, 16 CPU) – runs Docker builds
|--- Agent 3 (Windows) – runs .NET builds
|--- Agent 4 (macOS) – runs iOS builds
```

In your Jenkinsfile, you target specific agents using labels:

```
pipeline {
  agent none // No global agent - each stage defines its own
  stages {
    stage('Build Java') {
      agent { label 'java-builder' }
      steps {
        sh 'mvn clean package'
      }
    }
    stage('Build Docker Image') {
      agent { label 'docker-builder' }
      steps {
        sh 'docker build -t myapp:latest .'
      }
    }
  }
}
```

Dynamic agents with Kubernetes: Modern Jenkins deployments use the **Kubernetes plugin** to spin up agents as pods on-demand and destroy them after the build. This is far more efficient than maintaining static VMs.

```
pipeline {
  agent {
    kubernetes {
      yaml """
apiVersion: v1
kind: Pod
spec:
  containers:
```



```

    image: maven:3.8-openjdk-11
    command: ['cat']
    tty: true
- name: docker
  image: docker:20-dind
  securityContext:
    privileged: true
"""
    }
  }
  stages {
    stage('Build') {
      steps {
        container('maven') {
          sh 'mvn clean package'
        }
      }
    }
  }
}

```

Why This Matters: Static Jenkins agents waste money when not in use. Dynamic Kubernetes-based agents are created when a build starts and destroyed when it ends — you pay for exactly what you use.

Q6. [Intermediate] How do you manage secrets and credentials securely in Jenkins?

Answer:

Hardcoding secrets in a Jenkinsfile is one of the most dangerous mistakes in DevOps. Jenkins provides a built-in **Credentials Store** that encrypts secrets at rest and injects them into pipelines without ever exposing them in logs.

Types of credentials Jenkins supports:

- Secret Text (API keys, tokens)
- SSH Private Key
- Certificate
- Docker Hub credentials
- AWS access keys

Using credentials in a Declarative pipeline:

```

pipeline {
  agent any
  environment {
    // Binds username/password to variables
    DOCKER_CREDS = credentials('docker-hub-credentials')
    // DOCKER_CREDS_USR = username
    // DOCKER_CREDS_PSW = password (masked in logs)
  }
  stages {
    stage('Docker Login') {
      steps {
        sh 'echo $DOCKER_CREDS_PSW | docker login -u $DOCKER_CREDS_USR
--password-stdin'
      }
    }
    stage('Use AWS Credentials') {
      steps {
        withCredentials([
          string(credentialsId: 'aws-access-key', variable:
'AWS_ACCESS_KEY_ID'),
          string(credentialsId: 'aws-secret-key', variable:
'AWS_SECRET_ACCESS_KEY')
        ]) {
          sh 'aws s3 ls' // AWS credentials available only in this
block
        }
        // Credentials are automatically revoked here
      }
    }
  }
}

```

```
}  
}
```

Best Practice — Integrate with HashiCorp Vault: For enterprise deployments, Jenkins integrates with Vault to dynamically generate short-lived credentials:

```
stage('Fetch DB Password') {  
    steps {  
        withVault(configuration: [vaultUrl: 'https://vault.company.com'],  
                    vaultSecrets: [[path: 'secret/db', secretValues: [  
                        [envVar: 'DB_PASSWORD', vaultKey: 'password']  
                    ]]]) {  
            sh 'deploy_app.sh'  
        }  
    }  
}
```

Common Pitfalls:

- Never use `echo $SECRET_VAR` — Jenkins masks secrets in logs, but this habit is dangerous
- Avoid storing secrets in `environment {}` blocks when `withCredentials {}` scoping is possible
- Rotate credentials regularly and update them in Jenkins Credentials Store

Q7. [Advanced] How do you implement a Blue-Green deployment strategy using Jenkins?

Answer:

Blue-Green deployment is a release strategy that eliminates downtime by maintaining two identical production environments — "Blue" (current live) and "Green" (new version). Traffic is switched from Blue to Green atomically. If Green has issues, you switch back instantly.

Here's a complete Jenkins pipeline implementing Blue-Green on Kubernetes:

```
pipeline {
```

```

agent any
environment {
    APP_NAME = 'my-api'
    DOCKER_IMAGE = "myregistry/${APP_NAME}:${BUILD_NUMBER}"
    KUBE_NAMESPACE = 'production'
}
stages {
    stage('Determine Active Environment') {
        steps {
            script {
                // Check which color is currently active via the service
                selector

                def activeColor = sh(
                    script: "kubectl get service ${APP_NAME} -n
${KUBE_NAMESPACE} -o jsonpath='{.spec.selector.color} ",
                    returnStdout: true
                ).trim()
                env.ACTIVE_COLOR = activeColor
                env.INACTIVE_COLOR = (activeColor == 'blue') ?
                'green' : 'blue'

                echo "Active: ${env.ACTIVE_COLOR}, Deploying to:
${env.INACTIVE_COLOR}"
            }
        }
    }
    stage('Build & Push Docker Image') {
        steps {
            sh "docker build -t ${DOCKER_IMAGE} ."
            sh "docker push ${DOCKER_IMAGE}"
        }
    }
    stage('Deploy to Inactive Environment') {
        steps {
            script {
                // Update the inactive deployment with new image
                sh """
                    kubectl set image deployment/${APP_NAME}-

```

```

        app=${DOCKER_IMAGE} \
        -n ${KUBE_NAMESPACE}
    kubectl rollout status deployment/${APP_NAME}-
${env.INACTIVE_COLOR} \
        -n ${KUBE_NAMESPACE} \
        --timeout=300s
    """
    }
}
}
stage('Smoke Test Inactive Environment') {
    steps {
        script {
            // Test the inactive environment via its internal service
            def internalUrl = "http://${APP_NAME}-
${env.INACTIVE_COLOR}.${KUBE_NAMESPACE}.svc.cluster.local"
            sh """
                for i in 1 2 3 4 5; do
                    STATUS=$(curl -s -o /dev/null -w '%{http_code}'
${internalUrl}/health)
                    if [ "$STATUS" = "200" ]; then
                        echo "Health check passed"
                        exit 0
                    fi
                    sleep 10
                done
                echo "Health check failed"
                exit 1
            """
        }
    }
}
stage('Switch Traffic') {
    steps {
        // Requires manual approval before cutting traffic
        input message: "Switch traffic from ${env.ACTIVE_COLOR} to
${env.INACTIVE_COLOR}?",

```

```

sh """
    kubectl patch service ${APP_NAME} -n ${KUBE_NAMESPACE} \
        -p
    '{"spec":{"selector":{"color":"${env.INACTIVE_COLOR}}}}'
    """
    echo "Traffic now routed to: ${env.INACTIVE_COLOR}"
}
}
}
post {
    failure {
        echo "Deployment failed. Traffic remains on: ${env.ACTIVE_COLOR}"
        // No rollback needed – traffic was never switched
    }
}
}
}

```

Why This Works: The critical insight of Blue-Green is that if the deployment to the inactive environment fails, production is completely unaffected — it's still running the old version. Only after explicit smoke tests and human approval does traffic switch.

Key Takeaways — Jenkins

- Jenkins is an automation orchestrator; the actual work is done by the tools it calls
- Always write pipelines as code (Jenkinsfile) and commit to version control
- Use dynamic Kubernetes-based agents for scalability and cost efficiency
- Never hardcode secrets; use the Credentials Store or Vault integration
- Blue-Green and Canary deployments require pipeline logic to be well-architected

Common Pitfalls — Jenkins

- Configuring pipelines in the UI instead of as code
- Running all builds on the master/controller node (security and performance risk)
- Not archiving build artifacts, causing lost deployable packages
- Ignoring failed tests instead of breaking the build
- Not setting build timeouts — a hung build can block the entire pipeline

CHAPTER 2: DOCKER

Introduction to Docker

Docker revolutionized software delivery by solving the oldest problem in software: "It works on my machine." By packaging an application with all its dependencies into a container, Docker ensures that the software runs identically in development, testing, and production.

Understanding Docker deeply — not just `docker run`, but networking, volumes, multi-stage builds, and security — is essential for any modern DevOps engineer.

Q8. [Beginner] What is Docker, and how is it different from a virtual machine?

Answer:

Docker is a platform for building, running, and managing **containers** — lightweight, portable, self-contained units of software that include everything needed to run an application: code, runtime, system libraries, and configuration.

The core difference between containers and VMs:

A Virtual Machine includes an entire operating system kernel, device drivers, and virtualized hardware. It's like renting an entire apartment building just for yourself.

A Docker container shares the host operating system's kernel. It only packages the application and its dependencies. It's like renting just your apartment in a shared building.

Virtual Machine Stack:	Docker Container Stack:
+-----+	+-----+
App + Dependencies	App + Dependencies
+-----+	+-----+
Guest OS (full OS)	Docker Engine
+-----+	+-----+
Hypervisor	Host OS Kernel

+-----+	+-----+
Physical Hardware	Physical Hardware
+-----+	+-----+

Practical differences:

Feature	Virtual Machine	Docker Container
Startup time	Minutes	Milliseconds to seconds
Size	Gigabytes	Megabytes
OS included	Yes (full)	No (shares host kernel)
Isolation level	Strong (hypervisor)	Process-level
Portability	Moderate	Excellent
Resource overhead	High	Very low

When VMs are still better: When you need strong security isolation (multi-tenant environments), or when running a different OS kernel entirely; (e.g., Windows containers on Linux require a VM).

Q9. [Beginner] What is a Dockerfile, and walk me through writing one for a Node.js application.

Answer:

A **Dockerfile** is a text file containing instructions that Docker follows to build a container image. Think of it as a recipe — it describes exactly what goes into your container, layer by layer.

Here's a production-quality Dockerfile for a Node.js application:

```
# Stage 1: Build stage
# We use a specific version tag (not 'latest') for reproducibility
FROM node:18-alpine AS builder
# Set working directory inside the container
WORKDIR /app
# Copy dependency files FIRST (before source code)
# This is a critical optimization – Docker caches this layer
```



```

COPY package.json package-lock.json ./
# Install dependencies
# --ci uses the lockfile exactly (more deterministic than npm install)
RUN npm ci --only=production
# Stage 2: Runtime stage
# Use a minimal base image for the final container
FROM node:18-alpine AS runtime
# Security best practice: Don't run as root
# Create a non-root user
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
WORKDIR /app
# Copy only the installed dependencies from builder stage
COPY --from=builder /app/node_modules ./node_modules
# Copy application source code
COPY src/ ./src/
# Set ownership to non-root user
RUN chown -R appuser:appgroup /app
# Switch to non-root user
USER appuser
# Document the port the app listens on (informational, doesn't publish it)
EXPOSE 3000
# Health check — Docker can monitor container health
HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
    CMD wget --quiet --tries=1 --spider http://localhost:3000/health || exit 1
# Use exec form (not shell form) for CMD to properly handle signals
CMD ["node", "src/index.js"]

```

Explaining each instruction:

- **FROM** — Base image to start from. Always use specific version tags, not `latest`
- **WORKDIR** — Sets the working directory. Creates it if it doesn't exist
- **COPY package.json first** — This is a *layer caching trick*. If your source code changes but `package.json` doesn't, Docker reuses the cached `npm install` layer
- **RUN npm ci** — Installs exact versions from lockfile, better for CI/CD
- Multi-stage build — The `builder` stage installs tools; the `runtime` stage only has what's needed to run

- **HEALTHCHECK** — Tells Docker how to check if the container is healthy

Pro Tip: Multi-stage builds are one of the most impactful Docker optimizations. They can reduce image sizes from 1GB to under 100MB by leaving build tools out of the final image.

Q10. [Intermediate] How does Docker networking work, and what are the different network types?

Answer:

Docker networking is how containers communicate with each other and with the outside world. Getting this right is crucial for multi-container applications.

Docker's built-in network drivers:

1. Bridge Network (default) When you run a container without specifying a network, it joins the default **bridge** network. Containers on the same bridge can communicate using IP addresses, but NOT by name (on the default bridge).

Creating a custom bridge network (recommended):

```
# Create a custom bridge network
docker network create --driver bridge my-app-network
# Run containers on this network
docker run -d --name postgres --network my-app-network postgres:14
docker run -d --name api-server \
  --network my-app-network \
  -e DB_HOST=postgres \ # Can use container name as hostname!
  my-api-image
```

On custom bridge networks, containers can resolve each other by name. This is DNS-based service discovery built into Docker.

2. Host Network The container shares the host's network namespace — no isolation. The container uses the host's IP address and ports directly.

```
docker run --network host nginx
# nginx now listens on the host's port 80 directly
```

Use case: High-performance applications where network overhead matters, or when you need to bind to host network interfaces.

3. None Network Complete network isolation. The container has no network access.

```
docker run --network none my-batch-job
```

Use case: Security-sensitive batch processing that shouldn't have any network access.

4. Overlay Network (for Docker Swarm/multi-host) Allows containers on different Docker hosts to communicate securely. This is what Docker Swarm uses internally and what you'd use in a distributed environment.

```
docker network create --driver overlay --attachable my-overlay-net
```

Practical networking scenario — a three-tier application:

```
# Create isolated networks for each tier
docker network create frontend-net
docker network create backend-net
# Database – only on backend network
docker run -d --name mysql \
  --network backend-net \
  -e MYSQL_ROOT_PASSWORD=secret \
  mysql:8
# API – on both networks (bridges frontend and backend)
docker run -d --name api \
  --network backend-net \
  my-api-image
docker network connect frontend-net api
# Web frontend – only on frontend network
docker run -d --name nginx \
  --network frontend-net \
```

```
-p 80:80 \  
nginx:alpine
```

This architecture means: the database is never directly accessible from the frontend, which is a basic security principle.

Q11. [Intermediate] What are Docker volumes, and how do you persist data in containers?

Answer:

Containers are ephemeral by design — when a container is removed, all data written inside it disappears. **Volumes** solve this problem by providing persistent storage that exists independently of containers.

Three ways to persist data:

1. Named Volumes (Recommended) Docker manages the storage location on the host.

```
# Create a named volume  
docker volume create postgres-data  
# Mount it when running the container  
docker run -d \  
    --name postgres \  
    -v postgres-data:/var/lib/postgresql/data \  
    -e POSTGRES_PASSWORD=secret \  
    postgres:14  
# Inspect the volume  
docker volume inspect postgres-data  
# The data persists even if you remove and recreate the container  
docker rm -f postgres  
docker run -d \  
    --name postgres-new \  
    -v postgres-data:/var/lib/postgresql/data \ # Same volume, data intact!  
    postgres:14
```

2. Bind Mounts Mount a specific directory from the host into the container. Useful in development.

```
# Mount current directory into container (live code reload in development)
docker run -d \
  --name dev-server \
  -v $(pwd)/src:/app/src \ # Host path : Container path
  -p 3000:3000 \
  node:18-alpine \
  node /app/src/index.js
```

Warning: Bind mounts create a tight coupling between host and container. They're great for development, but avoid them in production — they break portability.

3. tmpfs Mounts Data stored in host memory only — never written to disk. Disappears when container stops.

```
docker run --tmpfs /tmp my-app
```

Use case: Sensitive temporary data (like session tokens) that should never touch disk.

Volume management commands:

```
# List all volumes
docker volume ls

# Remove a specific volume
docker volume rm postgres-data

# Remove all unused volumes (be careful!)
docker volume prune

# Back up a volume
docker run --rm \
  -v postgres-data:/source \
  -v $(pwd):/backup \
  alpine tar czf /backup/postgres-backup.tar.gz -C /source .

# Restore from backup
docker run --rm \
```

```
-v postgres-data:/target \  
-v $(pwd):/backup \  
alpine tar xzf /backup/postgres-backup.tar.gz -C /target
```

Q12. [Advanced] Explain Docker layer caching and how to optimize Dockerfiles for faster CI/CD builds.

Answer:

Docker builds images in layers, where each instruction (`FROM`, `COPY`, `RUN`, etc.) creates a new layer. Docker caches these layers — if nothing has changed in a layer, Docker reuses the cached version instead of rebuilding it. Understanding this is what makes the difference between a 30-second build and a 15-minute build.

The golden rule: Order instructions from least-changed to most-changed.

Bad Dockerfile (cache is invalidated on every code change):

```
FROM node:18-alpine  
WORKDIR /app  
# WRONG: Copying all files first means ANY file change  
# invalidates the npm install cache  
COPY . .  
RUN npm install  
CMD ["node", "index.js"]
```

Every time you change a single line of application code, Docker has to re-run `npm install` from scratch.

Optimized Dockerfile:

```
FROM node:18-alpine  
WORKDIR /app  
# CORRECT: Copy only dependency files first  
COPY package.json package-lock.json ./  
# This layer is cached unless package.json changes
```

```
RUN npm ci
# Copy source code last – changes here don't invalidate npm cache
COPY src/ ./src/
CMD ["node", "src/index.js"]
```

Advanced: Multi-stage builds with cache mounts (BuildKit):

```
# syntax=docker/dockerfile:1.4
FROM golang:1.21-alpine AS builder
WORKDIR /app
COPY go.mod go.sum ./
# --mount=type=cache caches the Go module cache between builds
# This is a BuildKit feature – incredibly fast for Go builds
RUN --mount=type=cache,target=/go/pkg/mod \
    go mod download
COPY . .
RUN --mount=type=cache,target=/go/pkg/mod \
    --mount=type=cache,target=/root/.cache/go-build \
    go build -o /app/binary .
FROM alpine:3.18 AS runtime
COPY --from=builder /app/binary /usr/local/bin/myapp
CMD ["myapp"]
```

Enable BuildKit:

```
# Set environment variable
export DOCKER_BUILDKIT=1
# Or use buildx (BuildKit is always enabled)
docker buildx build -t myapp:latest .
```

Additional optimization techniques:

```
# Use .dockerignore to exclude unnecessary files
cat .dockerignore
node_modules
.git
```

```

README.md
*.log
dist/
coverage/
.env
# Use specific digest pins for base images (ultimate reproducibility)
FROM node:18.17.0-alpine3.18@sha256:abc123...
# Combine RUN commands to reduce layers
# BAD (3 layers):
RUN apt-get update
RUN apt-get install -y curl
RUN rm -rf /var/lib/apt/lists/*
# GOOD (1 layer):
RUN apt-get update && \
    apt-get install -y --no-install-recommends curl && \
    rm -rf /var/lib/apt/lists/*

```

Measuring your improvements:

```

# Time your builds
time docker build -t myapp:test .
# Analyze image layers
docker history myapp:test
# Use dive tool to analyze layer content
dive myapp:test

```

Key Takeaways — Docker

- Containers share the host kernel; VMs include a full guest OS — containers are faster and lighter
- Multi-stage builds are the single most impactful optimization — use them always
- Order Dockerfile instructions from least-changed to most-changed for optimal cache usage
- Use named volumes for persistence, bind mounts only in development
- Never run containers as root; always create a non-root user

Common Pitfalls — Docker

- Using `latest` tag — unpredictable, breaks reproducibility
- Copying files before installing dependencies (kills caching)
- Running as root user inside containers
- Not using `.dockerignore` — bloated images from `node_modules` etc.
- Storing secrets as ENV variables in Dockerfiles — they appear in `docker history`

Prepared by Arvind Verma

CHAPTER 3: GIT, GITHUB & GITLAB

Introduction to Git in DevOps

Git is the foundation of every DevOps practice. You cannot do CI/CD, Infrastructure as Code, or GitOps without understanding Git deeply. GitHub and GitLab extend Git with collaboration features, CI/CD platforms, package registries, and security scanning — making them complete DevOps platforms, not just code hosts.

Q13. [Beginner] Explain the Git branching strategy you use in production environments.

Answer:

A branching strategy is a set of rules about how your team creates, names, and merges branches. Without a clear strategy, teams end up with hundreds of mysterious branches, unclear release processes, and constant merge conflicts.

The most commonly used strategies:

1. Gitflow (Traditional, popular for versioned releases)

```
main (production code – tagged releases only)
|
develop (integration branch – all features merge here)
|--- feature/user-login      (developer A's work)
|--- feature/payment-gateway (developer B's work)
|--- release/v2.1           (release preparation)
|--- hotfix/critical-bug     (emergency production fix)
# Creating a feature branch
git checkout develop
git checkout -b feature/user-authentication
# Working on the feature...
git add .
```

```

git commit -m "feat: add JWT token validation"
git commit -m "test: add unit tests for auth middleware"
# Merge back to develop via Pull Request
git checkout develop
git merge --no-ff feature/user-authentication
git branch -d feature/user-authentication
# Create release branch when ready
git checkout -b release/v2.1 develop
# Bug fixes only in release branch
git commit -m "fix: correct token expiry calculation"
# Merge to main and tag
git checkout main
git merge --no-ff release/v2.1
git tag -a v2.1.0 -m "Release version 2.1.0"
# Also merge back to develop
git checkout develop
git merge --no-ff release/v2.1

```

2. Trunk-Based Development (Modern, used by Google, Facebook)

Everyone works directly on `main` (trunk). Feature branches are short-lived — hours to a couple of days, not weeks.

```

# Short-lived feature branch
git checkout -b feature/add-caching
# Work for a few hours
git push origin feature/add-caching
# Create PR immediately
# Merge same day

```

Use **feature flags** to merge incomplete features safely:

```

# Feature flag controls whether new feature is enabled
if feature_flags.is_enabled('new_checkout_flow', user_id):
    return new_checkout_flow(cart)
else:
    return legacy_checkout_flow(cart)

```

3. GitHub Flow (Simple, good for continuous deployment)

```
main → feature branch → Pull Request → code review → merge to main → auto-deploy
```

Which to recommend in an interview: *"For teams deploying continuously (multiple times per day), I prefer Trunk-Based Development with feature flags. It reduces merge conflicts, enforces small commits, and keeps the team moving fast. For products with versioned releases — like mobile apps or APIs with SLA commitments — Gitflow provides better control over what goes into each release."*

Q14. [Intermediate] What is a Pull Request / Merge Request, and how do you structure an effective code review process?

Answer:

A **Pull Request (GitHub)** or **Merge Request (GitLab)** is a formal request to merge changes from one branch into another. It's not just a technical operation — it's a communication tool, a quality gate, and a knowledge-sharing mechanism.

Anatomy of a good PR:

Summary

Adds Redis-based caching for user profile API responses.

Reduces average response time from 450ms to 12ms for repeat requests.

Changes Made

- Added Redis client initialization in `src/cache/redis.go`
- Wrapped user profile endpoint with cache middleware
- Added cache invalidation on profile updates
- Cache TTL: 5 minutes (configurable via env var `CACHE_TTL_SECONDS`)

Testing

- Unit tests: `go test ./...` (all passing)
- Load test: ran k6 against staging, P99 latency dropped 96%
- Manual test: verified cache invalidation when profile updated

Screenshots / Evidence

[Screenshot of Grafana showing latency improvement]

Potential Concerns

- Cache stampede possible under high load – added lock mechanism
- Memory usage on Redis will increase ~50MB for 10k active users

Related Issues

Closes #347 – User profile API performance SLA breach

Code review checklist (for the reviewer):

- Does the code do what the PR says it does?
- Are there appropriate tests (unit, integration)?
- Are there any security concerns (SQL injection, exposed secrets, auth bypass)?
- Is the performance acceptable for expected load?
- Is the code readable — would a new team member understand it in 6 months?
- Does it follow the team's coding standards?

GitLab-specific: Enforcing review policies in `.gitlab-ci.yml`:

```
# Require approvals before merge
rules:
  - if: '$CI_MERGE_REQUEST_TARGET_BRANCH_NAME == "main"'
    when: manual
    allow_failure: false
# Protected branch settings in GitLab:
# Settings → Repository → Protected Branches
# main: Require 2 approvals, no direct pushes
```

GitHub branch protection rules:

```
Settings → Branches → Branch protection rules
✓ Require pull request reviews before merging (minimum 2)
✓ Require status checks to pass before merging (CI pipeline)
✓ Require branches to be up to date before merging
✓ Restrict who can push to matching branches
```

Q15. [Intermediate] How does GitLab CI/CD work, and how is it different from Jenkins?

Answer:

GitLab CI/CD is a built-in CI/CD platform tightly integrated with GitLab's repository, issue tracker, and security features. Unlike Jenkins, which requires installation, plugin management, and separate configuration, GitLab CI/CD is configured entirely through a `.gitlab-ci.yml` file in your repository.

Basic `.gitlab-ci.yml` for a Python application:

```
# Define the stages in order
stages:
  - validate
  - test
  - build
  - deploy

# Variables available to all jobs
variables:
  DOCKER_IMAGE: $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA
  PYTHON_VERSION: "3.11"

# Default settings for all jobs
default:
  image: python:3.11-slim
  before_script:
    - pip install -r requirements.txt

# Stage 1: Validate
lint:
  stage: validate
  script:
    - pip install flake8 black
    - flake8 src/
    - black --check src/
  rules:
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'

security-scan:
  stage: validate
  image: python:3.11-slim
  script:
```

```

- pip install bandit
- bandit -r src/ -f json -o bandit-report.json
artifacts:
  reports:
    sast: bandit-report.json
    expire_in: 1 week
# Stage 2: Test
unit-tests:
  stage: test
  script:
    - pytest tests/unit/ --cov=src --cov-report=xml
  artifacts:
    reports:
      coverage_report:
        coverage_format: cobertura
        path: coverage.xml
    coverage: '/TOTAL.*\s+(\d+)%$/'
integration-tests:
  stage: test
  services:
    - name: postgres:14
      alias: db
  variables:
    POSTGRES_DB: testdb
    POSTGRES_USER: test
    POSTGRES_PASSWORD: test
    DATABASE_URL: postgresql://test:test@db/testdb
  script:
    - pytest tests/integration/
# Stage 3: Build
build-image:
  stage: build
  image: docker:24
  services:
    - docker:24-dind
  before_script:
    - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY

```

```

- docker build -t $DOCKER_IMAGE .
- docker push $DOCKER_IMAGE
only:
  - main
  - tags
# Stage 4: Deploy
deploy-staging:
  stage: deploy
  image: bitnami/kubectl:latest
  environment:
    name: staging
    url: https://staging.myapp.com
  script:
    - kubectl set image deployment/myapp app=$DOCKER_IMAGE -n staging
    - kubectl rollout status deployment/myapp -n staging
  only:
    - main
deploy-production:
  stage: deploy
  environment:
    name: production
    url: https://myapp.com
  when: manual # Requires human approval
  script:
    - kubectl set image deployment/myapp app=$DOCKER_IMAGE -n production
    - kubectl rollout status deployment/myapp -n production
  only:
    - tags # Only deploy tagged releases to production

```

GitLab vs Jenkins comparison:

Feature	GitLab CI/CD	Jenkins
Setup	Zero (built-in)	Requires installation
Configuration	<code>.gitlab-ci.yml</code> in repo	Jenkinsfile in repo
Plugins	Built-in features + limited plugins	1800+ plugins
Security scanning	Built-in SAST/DAST/SCA	Requires plugins

Container registry	Built-in	Requires external registry
Kubernetes integration	Built-in agents	Plugin required
Learning curve	Lower	Higher
Flexibility	High	Extremely high
Self-hosted cost	Free (Community)	Free

Interview answer summary: "GitLab CI/CD is more opinionated and batteries-included — it's faster to get started and has built-in security scanning, container registry, and Kubernetes integration. Jenkins is more flexible and has a larger ecosystem of plugins, but requires more operational overhead to maintain. For greenfield projects, I'd choose GitLab. For complex, heterogeneous environments with diverse toolchains, Jenkins gives you more control."

Q16. [Advanced] Explain Git internals — what actually happens when you run `git commit`?

Answer:

This question separates engineers who use Git from engineers who *understand* Git. The answer reveals whether you truly know how Git works under the hood.

Git's object model:

Git stores everything as objects in the `.git/objects/` directory. There are four types:

1. **Blob** — stores file content
2. **Tree** — stores directory structure (lists of blobs and trees)
3. **Commit** — stores a snapshot with metadata (author, timestamp, parent commit, tree)
4. **Tag** — stores an annotated reference to a commit

What happens on `git commit`:

```
git add src/app.py
git commit -m "feat: add user authentication"
```

Step 1: `git add` creates a blob object for `src/app.py`:

```
# Git hashes the content and stores it
echo "def authenticate(user): ..." | git hash-object -w --stdin
# Returns: a3f9b2c... (SHA-1 hash of content)
```

Step 2: `git commit` creates a tree object representing the directory structure:

```
tree abc123...
blob a3f9b2c...    src/app.py
blob d8e4f1a...    src/models.py
tree 9c3b2d1...    tests/
```

Step 3: Git creates a commit object:

```
commit 7e4a1b3...
tree    abc123...          (the root tree)
parent  5c8f0d2...          (previous commit)
author  Jane Doe <jane@example.com> 1711285200 +0530
committer Jane Doe <jane@example.com> 1711285200 +0530
feat: add user authentication
```

Step 4: Git updates the branch reference:

```
# HEAD points to main
cat .git/HEAD
# ref: refs/heads/main
# main now points to new commit
cat .git/refs/heads/main
# 7e4a1b3... (the new commit SHA)
```

Why SHA-1 matters: Every object's hash is computed from its content. This means:

- If any bit in a file changes, the hash changes completely
- You cannot quietly modify history — any change creates new objects
- Git is a **content-addressable filesystem**

Practical consequence — `git rebase` vs `git merge`:

```
# merge: creates a new merge commit, preserves history
git merge feature-branch
# Result: A---B---C---M (M is the merge commit)
# rebase: replays commits on top of target, rewrites history
git rebase main
# Original: A---B (feature commits)
# After rebase: A'---B' (NEW commit objects with new SHAs)
```

This is why `git push --force` is dangerous after rebase — you're replacing remote commits with new objects that other developers don't have.

Key Takeaways — Git/GitHub/GitLab

- Choose a branching strategy based on your release cadence — Trunk-Based for continuous deployment, Gitflow for versioned releases
- PRs are communication tools, not just technical gates — write good descriptions
- GitLab CI/CD is batteries-included; Jenkins is more flexible but has higher overhead
- Git stores content-addressed objects — commits are immutable snapshots, not diffs
- Branch protection rules are non-negotiable for production branches

CHAPTER 4: AWS DEVOPS

Introduction to AWS DevOps

AWS provides a complete DevOps platform — from code hosting (CodeCommit) and CI/CD (CodePipeline, CodeBuild, CodeDeploy) to infrastructure automation (CloudFormation, CDK) and monitoring (CloudWatch). But in practice, most teams use a hybrid: AWS infrastructure services combined with third-party tools like Jenkins, GitHub Actions, or Terraform.

Q17. [Beginner] What is AWS CodePipeline, and how do you set up a basic CI/CD pipeline?

Answer:

AWS CodePipeline is a fully managed continuous delivery service that automates your release pipelines. It orchestrates the stages of your build, test, and deploy process by connecting AWS services and third-party tools.

CodePipeline consists of:

- **Source stage** — where code lives (CodeCommit, GitHub, S3)
- **Build stage** — where code is compiled and tested (CodeBuild)
- **Deploy stage** — where artifacts are deployed (CodeDeploy, ECS, Elastic Beanstalk, CloudFormation)

Creating a pipeline using Terraform:

```
# CodePipeline for an ECS application
resource "aws_codepipeline" "app_pipeline" {
  name      = "my-app-pipeline"
  role_arn = aws_iam_role.codepipeline_role.arn
  artifact_store {
    location = aws_s3_bucket.pipeline_artifacts.bucket
    type     = "S3"
  }
}
```

```

    encryption_key {
      id    = aws_kms_key.pipeline_key.arn
      type = "KMS"
    }
  }
}
# Stage 1: Source – pull from GitHub
stage {
  name = "Source"
  action {
    name           = "GitHub_Source"
    category       = "Source"
    owner          = "ThirdParty"
    provider       = "GitHub"
    version        = "1"
    output_artifacts = ["source_output"]
    configuration = {
      Owner      = "myorg"
      Repo       = "my-app"
      Branch     = "main"
      OAuthToken = var.github_oauth_token
    }
  }
}
}
# Stage 2: Build – CodeBuild runs tests and builds Docker image
stage {
  name = "Build"
  action {
    name           = "Build"
    category       = "Build"
    owner          = "AWS"
    provider       = "CodeBuild"
    input_artifacts = ["source_output"]
    output_artifacts = ["build_output"]
    version        = "1"
    configuration = {
      ProjectName = aws_codebuild_project.app_build.name
    }
  }
}

```

```

}
# Stage 3: Deploy to ECS
stage {
  name = "Deploy"
  action {
    name          = "Deploy_to_ECS"
    category      = "Deploy"
    owner         = "AWS"
    provider      = "ECS"
    input_artifacts = ["build_output"]
    version       = "1"
    configuration = {
      ClusterName = aws_ecs_cluster.app_cluster.name
      ServiceName = aws_ecs_service.app_service.name
      FileName    = "imagedefinitions.json"
    }
  }
}
}
}
}

```

CodeBuild buildspec.yml:

```

version: 0.2
phases:
  pre_build:
    commands:
      # Log in to ECR
      - echo Logging in to Amazon ECR...
      - aws ecr get-login-password --region $AWS_DEFAULT_REGION |
        docker login --username AWS --password-stdin $ECR_REGISTRY
      - IMAGE_TAG=$CODEBUILD_RESOLVED_SOURCE_VERSION
  build:
    commands:
      - echo Build started on `date`
      - echo Building the Docker image...
      - docker build -t $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG .

```

```

    - docker tag $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG
$ECR_REGISTRY/$ECR_REPOSITORY:latest
post_build:
  commands:
    - echo Pushing the Docker image...
    - docker push $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG
    - docker push $ECR_REGISTRY/$ECR_REPOSITORY:latest
    # Create imagedefinitions.json for ECS deployment
    - printf '["name":"app","imageUri":"%s"]]'
$ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG > imagedefinitions.json
artifacts:
  files:
    - imagedefinitions.json
reports:
  pytest_reports:
    files:
      - 'test-results/**/*.xml'
    file-format: 'JUNITXML'

```

Q18. [Intermediate] How do you implement infrastructure as code on AWS using CloudFormation vs CDK?

What are the tradeoffs?

Answer:

AWS offers two first-party IaC approaches: **CloudFormation** (declarative JSON/YAML templates) and **CDK** (Cloud Development Kit — write infrastructure in Python, TypeScript, Java, etc.).

CloudFormation — Writing an ECS Service:

```

# cloudformation/ecs-service.yaml
AWSTemplateFormatVersion: '2010-09-09'
Description: ECS Service with ALB
Parameters:
  Environment:

```

```

    Type: String
    AllowedValues: [dev, staging, prod]
  ContainerImage:
    Type: String
    Description: Docker image URI from ECR
  Conditions:
    IsProduction: !Equals [!Ref Environment, prod]
  Resources:
    # ECS Task Definition
    TaskDefinition:
      Type: AWS::ECS::TaskDefinition
      Properties:
        Family: !Sub 'my-app-${Environment}'
        NetworkMode: awsvpc
        RequiresCompatibilities: [FARGATE]
        Cpu: !If [IsProduction, '1024', '256']
        Memory: !If [IsProduction, '2048', '512']
        ExecutionRoleArn: !GetAtt TaskExecutionRole.Arn
        ContainerDefinitions:
          - Name: app
            Image: !Ref ContainerImage
            PortMappings:
              - ContainerPort: 8080
            LogConfiguration:
              LogDriver: awslogs
              Options:
                awslogs-group: !Ref LogGroup
                awslogs-region: !Ref AWS::Region
                awslogs-stream-prefix: app
            Environment:
              - Name: ENVIRONMENT
                Value: !Ref Environment
            Secrets:
              - Name: DATABASE_URL
                ValueFrom: !Sub
                  'arn:aws:secretsmanager:${AWS::Region}:${AWS::AccountId}:secret:${Environment}
                    /db/url'

```



```

ECSService:
  Type: AWS::ECS::Service
  Properties:
    ServiceName: !Sub 'my-app-${Environment}'
    Cluster: !ImportValue
      Fn::Sub: '${Environment}-ECSCluster'
    TaskDefinition: !Ref TaskDefinition
    DesiredCount: !If [IsProduction, 3, 1]
    LaunchType: FARGATE
    NetworkConfiguration:
      AwsvpcConfiguration:
        Subnets: !Split [' ',''], !ImportValue PrivateSubnetIds]
        SecurityGroups: [!Ref ServiceSecurityGroup]
    LoadBalancers:
      - ContainerName: app
        ContainerPort: 8080
        TargetGroupArn: !ImportValue AppTargetGroupArn

```

CDK — Same ECS Service in Python:

```

# cdk/ecs_stack.py
from aws_cdk import (
    Stack, Duration,
    aws_ecs as ecs,
    aws_ecs_patterns as ecs_patterns,
    aws_ecr as ecr,
    aws_secretsmanager as secretsmanager,
    aws_logs as logs,
)
from constructs import Construct
class ECSStack(Stack):
    def __init__(self, scope: Construct, construct_id: str, env_name: str,
    **kwargs):
        super().__init__(scope, construct_id, **kwargs)
        is_production = env_name == 'prod'
        # Reference existing ECR repository
        repository = ecr.Repository.from_repository_name(

```

```

        self, "AppRepo", "my-app"
    )
    # Get secret from Secrets Manager
    db_secret = secretsmanager.Secret.from_secret_name_v2(
        self, "DbSecret", f"{env_name}/db/url"
    )
    # Create a load-balanced Fargate service (one construct does
everything!)
    fargate_service = ecs_patterns.ApplicationLoadBalancedFargateService(
        self, "FargateService",
        cluster=ecs.Cluster.from_cluster_attributes(
            self, "ImportedCluster",
            cluster_name=f"{env_name}-cluster",
            vpc=vpc,
            security_groups=[]
        ),
        cpu=1024 if is_production else 256,
        memory_limit_mib=2048 if is_production else 512,
        desired_count=3 if is_production else 1,

task_image_options=ecs_patterns.ApplicationLoadBalancedTaskImageOptions(
            image=ecs.ContainerImage.from_ecr_repository(repository),
            container_port=8080,
            secrets={
                "DATABASE_URL": ecs.Secret.from_secrets_manager(db_secret)
            }
        ),
        environment={
            "ENVIRONMENT": env_name
        },
        log_driver=ecs.LogDrivers.aws_logs(
            stream_prefix="app",
            log_retention=logs.RetentionDays.ONE_MONTH
        )
    ),
    )
    # Auto-scaling
    scalable_target = fargate_service.service.auto_scale_task_count(

```

```

        max_capacity=20 if is_production else 3
    )
    scalable_target.scale_on_cpu_utilization(
        "CpuScaling",
        target_utilization_percent=70,
        scale_in_cooldown=Duration.seconds(60),
        scale_out_cooldown=Duration.seconds(60)
    )

```

CloudFormation vs CDK Tradeoff Table:

Dimension	CloudFormation	CDK
Language	YAML/JSON	Python, TypeScript, Java, C#
Learning curve	Lower (if you know YAML)	Higher (requires programming knowledge)
Abstraction	Low (you write every resource)	High (constructs encapsulate best practices)
Code reuse	Limited (Nested Stacks)	Excellent (Constructs, packages)
IDE support	Basic	Full IntelliSense, type checking
Testing	cfn-lint, CloudFormation Guard	Unit tests with assertions library
Verbosity	Very verbose	Concise
Debugging	Harder	Easier (Python errors are clear)
Team adoption	Easier for ops teams	Easier for dev teams

Operational Lesson: CDK synthesizes to CloudFormation under the hood, so you get all the safety of CloudFormation state management with the developer experience of a real programming language. For new projects, CDK with Python is the recommendation — especially given your team's Python expertise.

Q19. [Advanced] How do you implement zero-downtime deployments on ECS Fargate with a production traffic cutover strategy?

Answer:

Zero-downtime deployments on ECS Fargate require a combination of rolling updates, health checks, and ALB target group management. The gold standard is **Blue/Green deployment using CodeDeploy with ECS**, which provides atomic traffic shifting with instant rollback capability.

Architecture:



CodeDeploy AppSpec for ECS Blue/Green:

```
# appspec.yaml
version: 0.0
Resources:
  - TargetService:
      Type: AWS::ECS::Service
      Properties:
        TaskDefinition: <TASK_DEFINITION>
        LoadBalancerInfo:
          ContainerName: "app"
          ContainerPort: 8080
          PlatformVersion: "LATEST"
Hooks:
  - BeforeAllowTraffic: "LambdaFunctionToValidateBeforeTrafficShift"
  - AfterAllowTraffic: "LambdaFunctionToValidateAfterTrafficShift"
```

Lambda validation hook:

```
import boto3
```

```

import json
def lambda_handler(event, context):
    """
    Called by CodeDeploy before shifting traffic.
    We run smoke tests against the new task version.
    """
    codedeploy = boto3.client('codedeploy')
    deployment_id = event['DeploymentId']
    lifecycle_event_hook_execution_id = event['LifecycleEventHookExecutionId']
    try:
        # Run smoke tests against the replacement task set
        # CodeDeploy provides a test listener on a different port
        test_endpoint = "http://internal-alb.company.internal:8081/health"
        import urllib.request
        response = urllib.request.urlopen(test_endpoint, timeout=10)
        data = json.loads(response.read())
        if response.status == 200 and data.get('status') == 'healthy':
            status = 'Succeeded'
        else:
            status = 'Failed'
    except Exception as e:
        print(f"Smoke test failed: {e}")
        status = 'Failed'
    # Report status back to CodeDeploy
    codedeploy.put_lifecycle_event_hook_execution_status(
        deploymentId=deployment_id,
        lifecycleEventHookExecutionId=lifecycle_event_hook_execution_id,
        status=status
    )

```

Traffic shifting strategies in CodeDeploy:

```

# Terraform: CodeDeploy deployment group with canary strategy
resource "aws_codedeploy_deployment_group" "ecs_dg" {
  app_name          = aws_codedeploy_app.ecs_app.name
  deployment_group_name = "my-app-production"
  service_role_arn   = aws_iam_role.codedeploy_role.arn

```

```

deployment_config_name = "CodeDeployDefault.ECSCanary10Percent5Minutes"
# Other options:
# CodeDeployDefault.ECSLinear10PercentEvery1Minutes
# CodeDeployDefault.ECSAllAtOnce
# CodeDeployDefault.ECSCanary10Percent5Minutes
ecs_service {
  cluster_name = aws_ecs_cluster.main.name
  service_name = aws_ecs_service.app.name
}
deployment_style {
  deployment_option = "WITH_TRAFFIC_CONTROL"
  deployment_type   = "BLUE_GREEN"
}
load_balancer_info {
  target_group_pair_info {
    prod_traffic_route {
      listener_arns = [aws_alb_listener.https.arn]
    }
    test_traffic_route {
      listener_arns = [aws_alb_listener.test.arn] # Port 8080
    }
    target_group {
      name = aws_alb_target_group.blue.name
    }
    target_group {
      name = aws_alb_target_group.green.name
    }
  }
}
# Automatically rollback on CloudWatch alarm
auto_rollback_configuration {
  enabled = true
  events  = ["DEPLOYMENT_FAILURE", "DEPLOYMENT_STOP_ON_ALARM"]
}
alarm_configuration {
  alarms = [aws_cloudwatch_metric_alarm.error_rate.name]
  enabled = true
}

```

```
}
```

The **ECSCanary10Percent5Minutes** strategy explained:

1. Deploy new tasks to the green target group
2. Shift 10% of production traffic to green
3. Wait 5 minutes — monitor error rates and latency
4. If no alarms trigger, shift remaining 90% to green
5. Old blue tasks remain running for the **termination wait time** (default 1 hour) for instant rollback

Key Takeaways — AWS DevOps

- CodePipeline orchestrates; CodeBuild builds; CodeDeploy deploys — understand each component
- CDK synthesizes to CloudFormation, giving you programmatic power with CloudFormation safety
- ECS Blue/Green with CodeDeploy enables zero-downtime deployments with automatic rollback
- Always use Secrets Manager for credentials — never environment variables in task definitions

CHAPTER 5: GITHUB ACTIONS

Introduction to GitHub Actions

GitHub Actions is GitHub's native CI/CD platform. Launched in 2019, it became dominant extremely quickly because of its deep integration with GitHub, massive community marketplace of pre-built actions, and generous free tier. If your code is on GitHub, GitHub Actions is often the simplest, most integrated choice.

Q20. [Beginner] What is a GitHub Actions workflow, and explain its key components.

Answer:

A GitHub Actions **workflow** is an automated process defined in a YAML file stored in `.github/workflows/`. It runs on GitHub-managed virtual machines (or your own self-hosted runners) in response to events like code pushes, pull requests, or scheduled timers.

The five core components:

1. **Events (triggers)** — what starts the workflow
2. **Jobs** — groups of steps that run on the same machine
3. **Steps** — individual tasks within a job
4. **Actions** — reusable units (from Marketplace or custom)
5. **Runners** — the machines where jobs execute

Complete example — a full CI/CD workflow:

```
# .github/workflows/ci-cd.yml
name: CI/CD Pipeline
# EVENTS: This workflow triggers on...
on:
  push:
```



```

pull_request:
  branches: [main]
schedule:
  - cron: '0 2 * * *' # Nightly at 2 AM UTC
# Environment variables available to all jobs
env:
  PYTHON_VERSION: '3.11'
  ECR_REPOSITORY: my-app
jobs:
  # JOB 1: Run tests
  test:
    name: Run Tests
    # RUNNER: GitHub-hosted Ubuntu runner
    runs-on: ubuntu-latest
    # Run tests against multiple Python versions
    strategy:
      matrix:
        python-version: ['3.10', '3.11', '3.12']
    steps:
      # STEP 1: Check out the repository
      - name: Checkout code
        uses: actions/checkout@v4
      # STEP 2: Set up Python (using a Marketplace Action)
      - name: Set up Python ${{ matrix.python-version }}
        uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}
          cache: 'pip' # Cache pip dependencies
      # STEP 3: Install dependencies
      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
          pip install pytest pytest-cov flake8
      # STEP 4: Lint
      - name: Lint with flake8
        run: flake8 src/ --max-line-length=120

```

```

- name: Run tests
  run: pytest tests/ --cov=src --cov-report=xml --cov-fail-under=80
# STEP 6: Upload coverage report
- name: Upload coverage to Codecov
  uses: codecov/codecov-action@v4
  with:
    token: ${ secrets.CODECOV_TOKEN }
# JOB 2: Build and push Docker image (only on main branch)
build:
  name: Build Docker Image
  runs-on: ubuntu-latest
  needs: test # Wait for test job to pass
  if: github.ref == 'refs/heads/main'
  outputs:
    image-tag: ${ steps.meta.outputs.tags }
  steps:
    - name: Checkout code
      uses: actions/checkout@v4
    - name: Configure AWS credentials
      uses: aws-actions/configure-aws-credentials@v4
      with:
        aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
        aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
        aws-region: ap-south-1
    - name: Login to Amazon ECR
      id: login-ecr
      uses: aws-actions/amazon-ecr-login@v2
    - name: Extract metadata (tags, labels) for Docker
      id: meta
      uses: docker/metadata-action@v5
      with:
        images: ${ steps.login-ecr.outputs.registry }/${
env.ECR_REPOSITORY }
        tags: |
          type=sha,prefix=sha-
          type=ref,event=branch
          type=semver,pattern={{version}}

```

```

    uses: docker/build-push-action@v5
    with:
      context: .
      push: true
      tags: ${{ steps.meta.outputs.tags }}
      cache-from: type=gha
      cache-to: type=gha,mode=max
# JOB 3: Deploy to staging
deploy-staging:
  name: Deploy to Staging
  runs-on: ubuntu-latest
  needs: build
  environment: staging
  steps:
    - name: Deploy to ECS Staging
      uses: aws-actions/amazon-ecs-deploy-task-definition@v1
      with:
        task-definition: task-definition-staging.json
        service: my-app-staging
        cluster: staging-cluster
        wait-for-service-stability: true

```

Explaining key syntax:

- `needs: test` — job dependency; `build` only runs if `test` succeeds
- `${{ secrets.AWS_ACCESS_KEY_ID }}` — accessing encrypted secrets stored in GitHub Settings
- `${{ matrix.python-version }}` — accessing matrix variables for parallel test runs
- `uses: actions/checkout@v4` — using a community action from the Marketplace
- `outputs:` — passing data between jobs

Q21. [Intermediate] How do you secure secrets and manage environments in GitHub Actions?

Answer:

GitHub Actions has a layered secrets management system. Understanding the difference between repository secrets, environment secrets, and organization secrets is important for enterprise setups.

Levels of secrets:

Organization Secrets	→ Available to all repos in the organization
Repository Secrets	→ Available to all workflows in one repo
Environment Secrets	→ Available only to jobs targeting a specific environment

Setting up environment protection rules (critical for production):

In GitHub: `Settings` → `Environments` → `New environment` → `production`

- Required reviewers: at least 2 senior engineers
- Wait timer: 5 minutes (gives time to cancel if something looks wrong)
- Deployment branches: `main` only

Using environment secrets in a workflow

```
jobs:
  deploy-production:
    runs-on: ubuntu-latest
    environment: production # Triggers protection rules
    steps:
      - name: Deploy
        env:
          DATABASE_URL: ${ secrets.DATABASE_URL } # Environment secret
          API_KEY: ${ secrets.API_KEY } # Environment secret
        run: ./deploy.sh
```

Using OIDC for AWS authentication (best practice — no long-lived keys):

Instead of storing `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY` as secrets, use OIDC to let GitHub Actions assume an IAM role directly. This eliminates static credentials entirely.

```

jobs:
  deploy:
    runs-on: ubuntu-latest
    permissions:
      id-token: write # Required for OIDC
      contents: read
    steps:
      - name: Configure AWS credentials via OIDC
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume:
arn:aws:iam::123456789012:role/GitHubActionsDeployRole
          role-session-name: GitHubActions-${{ github.run_id }}
          aws-region: ap-south-1
          # No access key or secret key needed!
      - name: Deploy to AWS
        run: aws ecs update-service --cluster prod --service my-app

```

The IAM trust policy for OIDC:

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Federated": "arn:aws:iam::123456789012:oidc-
provider/token.actions.githubusercontent.com"
      },
      "Action": "sts:AssumeRoleWithWebIdentity",
      "Condition": {
        "StringEquals": {
          "token.actions.githubusercontent.com:aud": "sts.amazonaws.com"
        },
        "StringLike": {
          "token.actions.githubusercontent.com:sub": "repo:myorg/my-
app:environment:production"

```

```

    }
  }
}
]
}

```

This policy only allows the `production` environment of `myorg/my-app` to assume this role — not any other repo or environment.

Q22. [Advanced] How do you write a reusable workflow in GitHub Actions?

Answer:

Reusable workflows are GitHub Actions' equivalent of shared libraries. Instead of copying the same YAML across dozens of repositories, you define the workflow once and call it from other workflows.

Defining a reusable workflow (in a shared repo):

```

# .github/workflows/reusable-docker-build.yml
name: Reusable Docker Build and Push
on:
  workflow_call: # This makes it reusable
  inputs:
    ecr-repository:
      required: true
      type: string
    aws-region:
      required: false
      type: string
      default: 'ap-south-1'
    environment:
      required: true
      type: string
  secrets:
    aws-role-arn:
      required: true

```

```

    outputs:
      image-uri:
        description: "The full URI of the pushed Docker image"
        value: ${ jobs.build.outputs.image-uri }
  jobs:
    build:
      runs-on: ubuntu-latest
      environment: ${ inputs.environment }
      outputs:
        image-uri: ${ steps.build-push.outputs.image-uri }
      steps:
        - uses: actions/checkout@v4
        - name: Configure AWS credentials
          uses: aws-actions/configure-aws-credentials@v4
          with:
            role-to-assume: ${ secrets.aws-role-arn }
            aws-region: ${ inputs.aws-region }
        - name: Login to ECR
          id: login-ecr
          uses: aws-actions/amazon-ecr-login@v2
        - name: Build and push
          id: build-push
          run: |
            IMAGE_URI="${ steps.login-ecr.outputs.registry }/${ inputs.ecr-
repository }:${ inputs.github.sha }"
            docker build -t $IMAGE_URI .
            docker push $IMAGE_URI
            echo "image-uri=$IMAGE_URI" >> $GITHUB_OUTPUT

```

Calling the reusable workflow from another repo:

```

# .github/workflows/deploy.yml (in any repo)
name: Deploy My App
on:
  push:
    branches: [main]
jobs:

```

```
build:
  uses: myorg/shared-workflows/.github/workflows/reusable-docker-
build.yml@main
  with:
    ecr-repository: my-specific-app
    environment: production
  secrets:
    aws-role-arn: ${ secrets.AWS_DEPLOY_ROLE_ARN }
deploy:
  needs: build
  runs-on: ubuntu-latest
  steps:
    - name: Deploy with image
      run: |
        echo "Deploying image: ${ needs.build.outputs.image-uri }"
        # kubectl set image deployment/app app=${
needs.build.outputs.image-uri }
```

Pro Tip: Reusable workflows are the key to maintaining consistency at scale. When you have 50 microservices, you can update the build process in one place and all services benefit.

CHAPTER 6: KUBERNETES

Introduction to Kubernetes

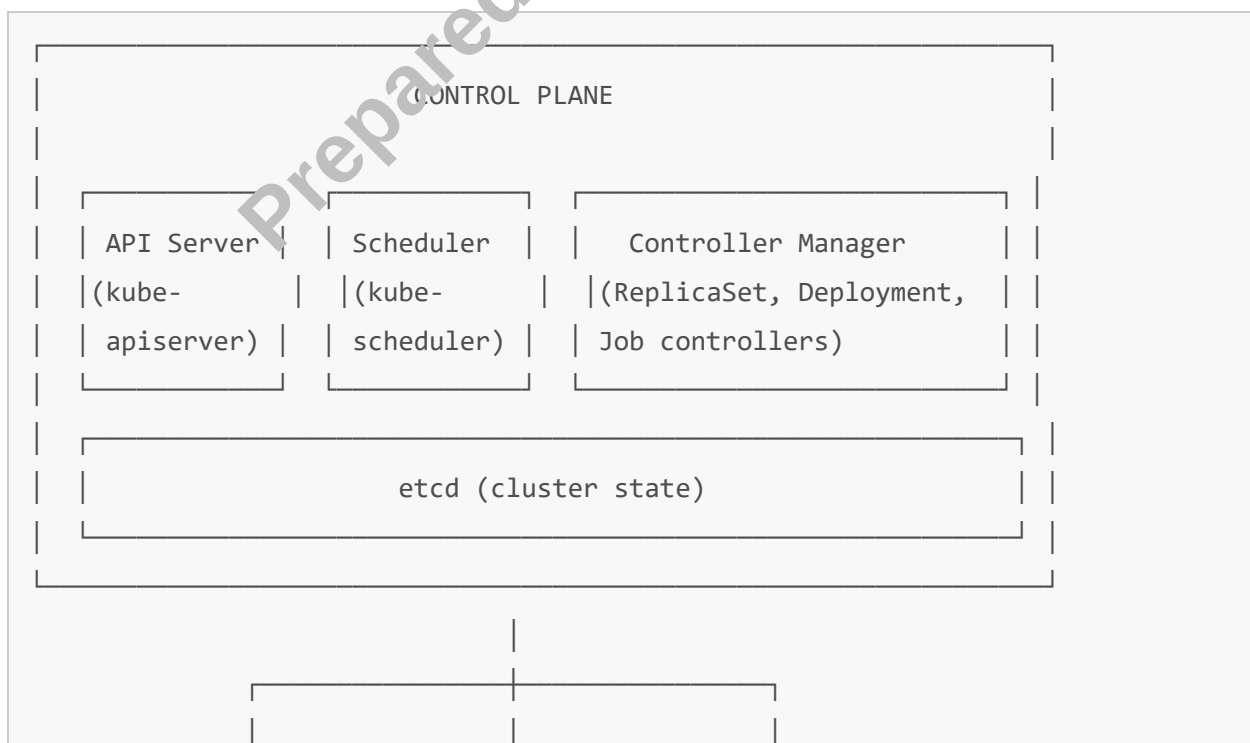
Kubernetes (K8s) is the de facto standard for container orchestration in production. It automates deployment, scaling, and management of containerized applications across clusters of machines. Docker runs a single container; Kubernetes runs thousands of them, distributed across many servers, with automatic failover, scaling, and networking.

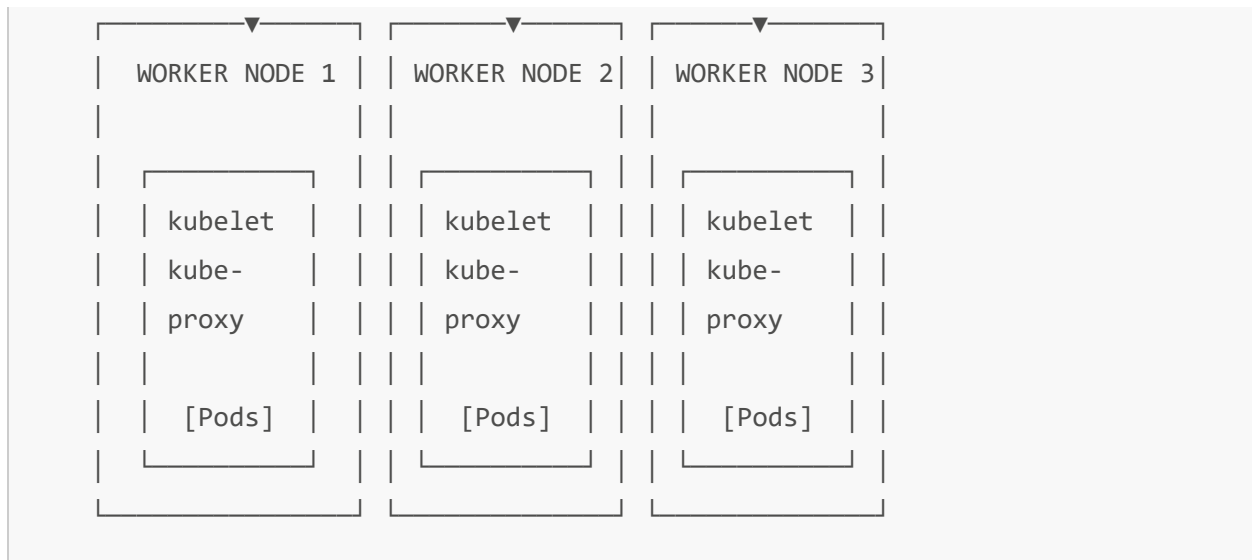
Q23. [Beginner] Explain the Kubernetes architecture — what are the main components?

Answer:

Kubernetes follows a **control plane + worker node** architecture. Think of the control plane as the brain (it makes decisions) and worker nodes as the muscles (they run the actual workloads).

Control Plane Components:





Control Plane Components explained:

- **kube-apiserver:** The front door of Kubernetes. Every command (`kubectl apply`, `kubectl get pods`) goes through the API server. It validates requests and updates state in etcd
- **etcd:** A distributed key-value store that holds the *entire state* of the cluster — every resource, configuration, and status. This is the single source of truth
- **kube-scheduler:** Watches for new Pods that have no Node assigned, and selects the best Node based on resource requirements, affinity rules, and policies
- **kube-controller-manager:** Runs controllers in a loop — ReplicaSet controller ensures the right number of pods, node controller handles node failures, etc.

Worker Node Components:

- **kubelet:** An agent running on every worker node. It receives Pod specs from the API server and ensures containers are running as expected
- **kube-proxy:** Maintains network rules on each node, enabling Service communication
- **Container Runtime:** Docker, containerd, or CRI-O — runs the actual containers

Q24. [Beginner] What is a Pod, Deployment, and Service in Kubernetes?

How do they relate to each other?

Answer:

These three objects are the foundation of almost every Kubernetes application.

Pod: The smallest deployable unit in Kubernetes. A Pod wraps one or more containers that share network space and storage. Think of a Pod as a "logical host" for your containers.

```
# pod.yaml - a basic Pod definition
apiVersion: v1
kind: Pod
metadata:
  name: my-app-pod
  labels:
    app: my-app
    version: v1.0
spec:
  containers:
    - name: app
      image: myregistry/my-app:1.0.0
      ports:
        - containerPort: 8080
      resources:
        requests:
          memory: "128Mi"
          cpu: "250m"
        limits:
          memory: "256Mi"
          cpu: "500m"
      readinessProbe:
        httpGet:
          path: /health
          port: 8080
        initialDelaySeconds: 10
        periodSeconds: 5
      livenessProbe:
        httpGet:
          path: /health
          port: 8080
        initialDelaySeconds: 30
```

```
periodSeconds: 10
```

In practice, you almost never create Pods directly. If a Pod crashes, it stays dead — there's nothing to restart it. That's where Deployments come in.

Deployment: A Deployment manages a *set of identical Pods* (called a ReplicaSet). It ensures the desired number of Pods is always running, handles rolling updates, and enables rollbacks.

```
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
  namespace: production
spec:
  replicas: 3                                # Always keep 3 Pods running
  selector:
    matchLabels:
      app: my-app
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1                    # At most 1 Pod unavailable during update
      maxSurge: 1                          # At most 1 extra Pod during update
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: app
          image: myregistry/my-app:1.0.0
          ports:
            - containerPort: 8080
          resources:
            requests:
              memory: "256Mi"
```

```
    cpu: "250m"
  limits:
    memory: "512Mi"
    cpu: "500m"
```

Service: Pods are ephemeral — they get new IP addresses when they restart. A Service provides a **stable network endpoint** (DNS name + IP) that always routes to the healthy Pods behind it, regardless of their individual IP addresses.

```
# service.yaml
apiVersion: v1
kind: Service
metadata:
  name: my-app-service
  namespace: production
spec:
  type: ClusterIP      # Internal only – use LoadBalancer or NodePort for
external
  selector:
    app: my-app        # Routes to all Pods with this label
  ports:
    - protocol: TCP
      port: 80          # Service port
      targetPort: 8080  # Container port
```

How they relate.

```
Internet Request
  ↓
LoadBalancer Service (external IP)
  ↓
ClusterIP Service (my-app-service:80)
  ↓ (routes to one of...)
Pod 1 (10.0.1.5:8080) ┌
Pod 2 (10.0.1.6:8080) ┤ Managed by Deployment
Pod 3 (10.0.1.7:8080) └
```

Useful kubectl commands:

```
# Apply all three manifests
kubectl apply -f pod.yaml
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
# Watch pods start up
kubectl get pods -n production -w
# Check deployment status
kubectl rollout status deployment/my-app -n production
# Update image (triggers rolling update)
kubectl set image deployment/my-app app=myregistry/my-app:1.1.0 -n production
# Rollback to previous version
kubectl rollout undo deployment/my-app -n production
# Scale up
kubectl scale deployment/my-app --replicas=5 -n production
# Describe a pod (great for debugging)
kubectl describe pod my-app-abc123 -n production
# View logs
kubectl logs -f my-app-abc123 -n production
# Exec into a running pod for debugging
kubectl exec -it my-app-abc123 -n production -- /bin/sh
```

Q25. [Intermediate] What is a ConfigMap and Secret in Kubernetes?

How do you inject configuration into Pods?

Answer:

ConfigMap stores non-sensitive configuration data as key-value pairs. **Secret** stores sensitive data (passwords, tokens, certificates) in base64-encoded form with additional access controls.

```
# configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
```

```

    namespace: production
data:
  APP_ENV: "production"
  LOG_LEVEL: "info"
  MAX_CONNECTIONS: "100"
  # Multi-line config files are also supported
  app.properties: |
    server.port=8080
    server.timeout=30
    feature.new_ui=true
# secret.yaml
apiVersion: v1
kind: Secret
metadata:
  name: app-secrets
  namespace: production
type: Opaque
# Values must be base64-encoded
# echo -n "mysupersecretpassword" | base64
data:
  DB_PASSWORD: bXlzdXB1cnNlY3JldHF1c3l3b3Jk
  API_KEY: c2VjcmlV0YXBpa2V5MTI=

```

Injecting into a Pod — three methods:

Method 1: Environment Variables

```

spec:
  containers:
    - name: app
      image: my-app:1.0
      env:
        # From ConfigMap
        - name: APP_ENV
          valueFrom:
            configMapKeyRef:
              name: app-config

```

```

        key: APP_ENV
# From Secret
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: app-secrets
      key: DB_PASSWORD
# Inject all keys from ConfigMap as env vars
envFrom:
- configMapRef:
    name: app-config
- secretRef:
    name: app-secrets

```

Method 2: Volume mount (for config files)

```

spec:
  volumes:
    - name: config-volume
      configMap:
        name: app-config
    - name: secrets-volume
      secret:
        secretName: app-secrets
        defaultMode: 0400 # Read-only for owner only
  containers:
    - name: app
      image: my-app:1.0
      volumeMounts:
        - name: config-volume
          mountPath: /etc/config
          readOnly: true
        - name: secrets-volume
          mountPath: /etc/secrets
          readOnly: true
      # Files are now at:
      # /etc/config/APP_ENV

```



```
# /etc/config/app.properties
# /etc/secrets/DB_PASSWORD
```

Best Practice — Use External Secrets Operator for production:

Storing plain secrets in Kubernetes is not ideal for production — they're only base64-encoded, not encrypted. The **External Secrets Operator** syncs secrets from AWS Secrets Manager or Vault into Kubernetes Secrets automatically:

```
# external-secret.yaml
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: app-secrets
  namespace: production
spec:
  refreshInterval: 5m      # Re-sync every 5 minutes
  secretStoreRef:
    name: aws-secrets-manager
    kind: ClusterSecretStore
  target:
    name: app-secrets      # Creates/updates this K8s Secret
    creationPolicy: Owner
  data:
    - secretKey: DB_PASSWORD
      remoteRef:
        key: production/app/db
        property: password
    - secretKey: API_KEY
      remoteRef:
        key: production/app/api
        property: key
```

Q26. [Intermediate] Explain Kubernetes resource requests and limits. What happens when a container exceeds its limits?

Answer:

Resource management in Kubernetes is one of the most practically important topics — get it wrong and your cluster either wastes money (over-provisioned) or crashes workloads (under-provisioned).

Requests vs Limits:

- **Request:** The amount of CPU/memory Kubernetes *guarantees* for the container. The Scheduler uses this to decide which node to place the Pod on
- **Limit:** The maximum amount of CPU/memory the container can use. If it exceeds this, it gets throttled (CPU) or killed (memory)

```
resources:
  requests:
    memory: "256Mi"    # Guaranteed 256MB RAM
    cpu: "250m"        # Guaranteed 0.25 CPU cores (250 millicores)
  limits:
    memory: "512Mi"    # Never use more than 512MB RAM
    cpu: "1000m"       # Never use more than 1 CPU core
```

What happens when limits are exceeded:

Resource	Behavior when limit exceeded
CPU	Container is throttled (slowed down) — not killed
Memory	Container is OOMKilled (killed by the OS) and restarted

```
# Checking OOMKilled events
kubectl describe pod my-app-abc123
# Look for:
# Last State: Terminated
#   Reason: OOMKilled
#   Exit Code: 137
# Check resource usage
kubectl top pods -n production
kubectl top nodes
```

Quality of Service (QoS) classes:

Kubernetes assigns a QoS class based on your resource configuration:

```
Guaranteed (highest priority – never evicted)
  requests == limits (both set and equal)
Burstable (medium priority)
  requests < limits (or only one set)
BestEffort (lowest priority – evicted first)
  no requests or limits set at all
```

Practical guidance:

```
# Production workload – Guaranteed QoS
resources:
  requests:
    memory: "512Mi"
    cpu: "500m"
  limits:
    memory: "512Mi"      # Same as request = Guaranteed
    cpu: "500m"

# Development/batch workload – Burstable QoS
resources:
  requests:
    memory: "128Mi"
    cpu: "100m"
  limits:
    memory: "1Gi"        # Can burst up to 1GB
    cpu: "2000m"         # Can burst to 2 cores
```

Operational Lesson: Never deploy to production without resource requests and limits. Without requests, the scheduler places Pods blindly. Without limits, one misbehaving container can starve neighboring Pods. Use `kubectl top pods` regularly to tune your resource settings based on actual usage.

Q27. [Advanced] How do you implement Horizontal Pod Autoscaling (HPA) in Kubernetes?

Answer:

HPA automatically scales the number of Pods in a Deployment based on observed metrics. The classic trigger is CPU utilization, but modern HPA supports custom metrics from Prometheus, memory, or even external metrics (like queue depth in SQS).

Basic CPU-based HPA:

```
# hpa-cpu.yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: my-app-hpa
  namespace: production
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: my-app
  minReplicas: 3
  maxReplicas: 20
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70 # Scale when avg CPU > 70%
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
```

```

behavior:
  scaleUp:
    stabilizationWindowSeconds: 30    # Wait 30s before scaling up again
  policies:
    - type: Pods
      value: 4                        # Add at most 4 pods at once
      periodSeconds: 60
  scaleDown:
    stabilizationWindowSeconds: 300   # Wait 5min before scaling down
  policies:
    - type: Percent
      value: 10                       # Remove at most 10% of pods at once
      periodSeconds: 60

```

Custom metrics HPA (e.g., requests per second from Prometheus):

```

# Requires prometheus-adapter to be installed
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: my-app-hpa-custom
  namespace: production
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: my-app
  minReplicas: 2
  maxReplicas: 50
  metrics:
    - type: Pods
      pods:
        metric:
          name: http_requests_per_second # Custom metric from Prometheus
        target:
          type: AverageValue
          averageValue: "100"           # Scale when > 100 req/s per pod

```

```

- type: External
  external:
    metric:
      name: sqs_messages_visible      # SQS queue depth (via CloudWatch
adapter)
    selector:
      matchLabels:
        queue: order-processing
    target:
      type: AverageValue
      averageValue: "50"              # Scale when queue depth > 50 per
pod

```

Monitoring HPA behavior:

```

# Watch HPA status in real time
kubectl get hpa -n production -w
# Describes the current scaling status and recent events
kubectl describe hpa my-app-hpa -n production
# Output includes:
# Metrics: 85% / 70% (current / target) ← will trigger scale-up
# Min replicas: 3, Max replicas: 20
# Deployment pods: 3 current, 5 desired ← scaling in progress
# Generate load to test autoscaling
kubectl run load-generator --image=busybox --restart=Never -- \
  /bin/sh -c "while true; do wget -q -O- http://my-app-service/api; done"
# Watch pods scale up
kubectl get pods -n production -w

```

Pro Tip: Always set `scaleDown.stabilizationWindowSeconds` to at least 300 seconds (5 minutes) for production workloads. Aggressive scale-down followed by sudden traffic spikes creates a "yo-yo effect" where pods are constantly being created and destroyed, causing latency spikes. The stabilization window prevents this.

Q28. [Advanced] What is a Kubernetes Ingress, and how does it differ from a LoadBalancer Service?

Answer:

Understanding the difference between Services and Ingress is crucial for designing Kubernetes networking correctly.

LoadBalancer Service: Creates an external cloud load balancer (like an AWS ALB or NLB) for *each* Service. If you have 10 microservices, you get 10 load balancers — 10 bills.

Ingress: A single entry point (one load balancer) that routes HTTP/HTTPS traffic to different Services based on host names and URL paths. One load balancer, many services.

Without Ingress:

```
api.myapp.com    → ALB #1 → api-service
web.myapp.com    → ALB #2 → web-service
admin.myapp.com  → ALB #3 → admin-service
(3 load balancers = 3 bills)
```

With Ingress:

```
api.myapp.com    ┌
web.myapp.com    ┌→ ALB #1 → Ingress Controller → routes to correct Service
admin.myapp.com  ┌
(1 load balancer = 1 bill, + Ingress Controller overhead)
```

Complete NGINX Ingress example.

```
# ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-app-ingress
  namespace: production
  annotations:
    kubernetes.io/ingress.class: "nginx"
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
    nginx.ingress.kubernetes.io/proxy-body-size: "50m"
    nginx.ingress.kubernetes.io/rate-limit: "100"
    # Automatic TLS certificate via cert-manager
    cert-manager.io/cluster-issuer: "letsencrypt-prod"
```

```

spec:
  tls:
    - hosts:
        - api.myapp.com
        - web.myapp.com
      secretName: myapp-tls-secret # cert-manager creates this
  rules:
    # Route based on hostname
    - host: api.myapp.com
      http:
        paths:
          - path: /v1
            pathType: Prefix
            backend:
              service:
                name: api-v1-service
                port:
                  number: 80
          - path: /v2
            pathType: Prefix
            backend:
              service:
                name: api-v2-service
                port:
                  number: 80
    - host: web.myapp.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: web-frontend-service
                port:
                  number: 80

```

AWS-specific: Using AWS Load Balancer Controller (ALB Ingress):


```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-app-ingress
  namespace: production
  annotations:
    kubernetes.io/ingress.class: alb
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
    alb.ingress.kubernetes.io/certificate-arn: arn:aws:acm:ap-south-
1:123456789:certificate/abc
    alb.ingress.kubernetes.io/listen-ports: '[{"HTTP": 80}, {"HTTPS": 443}]'
    alb.ingress.kubernetes.io/actions.ssl-redirect: >
{"Type": "redirect", "RedirectConfig": {"Protocol": "HTTPS", "Port": "443", "StatusCo
de": "HTTP_301"}}
    alb.ingress.kubernetes.io/group.name: my-app # Share ALB across
ingresses
spec:
  rules:
    - http:
        paths:
          - path: /api/*
            pathType: ImplementationSpecific
            backend:
              service:
                name: api-service
                port:
                  number: 80
          - path: /*
            pathType: ImplementationSpecific
            backend:
              service:
                name: web-service
                port:
                  number: 80

```

Key Takeaways — Kubernetes

- Pods are ephemeral; Deployments manage Pod lifecycle and rolling updates
- Services provide stable DNS endpoints; Ingress routes external HTTP traffic
- Always set resource requests (for scheduling) and limits (for stability)
- HPA autoscales based on CPU, memory, or custom Prometheus metrics — tune scale-down carefully
- Use External Secrets Operator to sync secrets from AWS Secrets Manager — don't store plain secrets in K8s

Common Pitfalls — Kubernetes

- Not setting resource requests/limits — the cluster scheduler makes poor decisions
- Using `latest` image tag — no way to track what's actually deployed
- Not configuring liveness and readiness probes — Kubernetes can't detect unhealthy apps
- Ignoring Pod Disruption Budgets — rolling updates can take down too many pods at once
- Running privileged containers — major security risk in multi-tenant clusters

CHAPTER 7: PROMETHEUS & GRAFANA

Introduction to Observability

Observability answers the question: "What is my system doing right now, and why?" Prometheus and Grafana form the industry-standard monitoring stack. Prometheus collects and stores metrics; Grafana visualizes them in dashboards and sends alerts. Together, they give engineering teams the visibility needed to operate production systems confidently.

Q29. [Beginner] What is Prometheus, and how does it collect metrics?

Answer:

Prometheus is an open-source **time-series metrics collection and alerting system**. It works on a **pull model** — instead of applications pushing metrics to a central server, Prometheus *scrapes* (polls) metrics endpoints at regular intervals.

The pull model explained:

```
[Application] exposes metrics at GET /metrics
      ↑
[Prometheus] scrapes /metrics every 15 seconds
      ↓
[Prometheus TSDB] stores time-series data
      ↓
[Grafana / AlertManager] reads and visualizes
```

What metrics look like at `/metrics` endpoint:

```
# HELP http_requests_total Total number of HTTP requests
# TYPE http_requests_total counter
http_requests_total{method="GET", endpoint="/api/users", status="200"} 15234
http_requests_total{method="GET", endpoint="/api/users", status="500"} 23
http_requests_total{method="POST", endpoint="/api/orders", status="201"} 4521
```

```
# HELP http_request_duration_seconds HTTP request duration in seconds
# TYPE http_request_duration_seconds histogram
http_request_duration_seconds_bucket{le="0.1"} 9800
http_request_duration_seconds_bucket{le="0.5"} 14900
http_request_duration_seconds_bucket{le="1.0"} 15200
http_request_duration_seconds_bucket{le="+Inf"} 15234
http_request_duration_seconds_sum 1523.4
http_request_duration_seconds_count 15234
```

Prometheus configuration (`prometheus.yml`):

```
global:
  scrape_interval: 15s # Scrape every 15 seconds
  evaluation_interval: 15s # Evaluate alerting rules every 15 seconds
alerting:
  alertmanagers:
    - static_configs:
        - targets: ['alertmanager:9093']
rule_files:
  - "alerts/*.yml"
scrape_configs:
  # Scrape Prometheus itself
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']
  # Scrape Kubernetes nodes via node-exporter
  - job_name: 'kubernetes-nodes'
    kubernetes_sd_configs:
      - role: node
    relabel_configs:
      - action: labelmap
        regex: __meta_kubernetes_node_label_(.+)
  # Scrape all pods with annotation prometheus.io/scrape: "true"
  - job_name: 'kubernetes-pods'
    kubernetes_sd_configs:
      - role: pod
    relabel_configs:
```

```

- source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_scrape]
  action: keep
  regex: true
- source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_path]
  action: replace
  target_label: __metrics_path__
  regex: (.+)
- source_labels: [__address__,
__meta_kubernetes_pod_annotation_prometheus_io_port]
  action: replace
  regex: ([^:]+)(?::\d+)?;(\d+)
  replacement: $1:$2
  target_label: __address__

```

The four metric types:

Type	Description	Example
Counter	Always increases; resets on restart	http_requests_total
Gauge	Can go up or down	memory_usage_bytes
Histogram	Samples observations in buckets	request_duration_seconds
Summary	Similar to histogram, client-side quantiles	rpc_duration_seconds

Q30. [Intermediate] How do you write PromQL queries for practical alerting scenarios?

Answer:

PromQL (Prometheus Query Language) is a functional query language for time-series data. Writing effective PromQL is a core skill — it powers both Grafana dashboards and Alertmanager rules.

The four key PromQL operations:

1. Instant vector selectors:

```
# Current value of a metric with label filter
http_requests_total{job="api-server", status="500"}
# All HTTP 5xx errors across all jobs
http_requests_total{status=~"5.."} # =~ means regex match
```

2. Range vector + rate() — the most used pattern:

```
# Request rate over the last 5 minutes (requests per second)
rate(http_requests_total[5m])
# Error rate per second for the API
rate(http_requests_total{job="api", status=~"5.."}[5m])
```

3. Aggregation:

```
# Total request rate across all instances
sum(rate(http_requests_total[5m]))
# Request rate per endpoint
sum by (endpoint) (rate(http_requests_total[5m]))
# P99 latency across all pods
histogram_quantile(0.99, sum by (le)
(rate(http_request_duration_seconds_bucket[5m])))
```

4. Arithmetic and comparison

```
# Error ratio (errors / total requests)
sum(rate(http_requests_total{status=~"5.."}[5m]))
/
sum(rate(http_requests_total[5m]))
# Memory usage percentage
(node_memory_MemTotal_bytes - node_memory_MemAvailable_bytes)
/ node_memory_MemTotal_bytes * 100
```

Production alerting rules:

```
# alerts/api-alerts.yml
```

```

- name: api-alerts
rules:
  # Alert: High error rate
  - alert: HighErrorRate
    expr: |
      sum(rate(http_requests_total{status=~"5.."}[5m]))
      /
      sum(rate(http_requests_total[5m])) > 0.05
    for: 2m      # Must be true for 2 consecutive minutes before firing
    labels:
      severity: critical
      team: platform
    annotations:
      summary: "High error rate detected"
      description: "Error rate is {{ $value | humanizePercentage }}
(threshold: 5%)"
      runbook: "https://wiki.company.com/runbooks/high-error-rate"
  # Alert: High P99 latency
  - alert: HighLatency
    expr: |
      histogram_quantile(0.99
        sum by (le, service) (
          rate(http_request_duration_seconds_bucket[5m])
        )
      ) > 2.0
    for: 5m
    labels:
      severity: warning
    annotations:
      summary: "P99 latency above 2 seconds for {{ $labels.service }}"
      description: "P99 latency is {{ $value }}s"
  # Alert: Pod OOMKilled
  - alert: PodOOMKilled
    expr:
kube_pod_container_status_last_terminated_reason{reason="OOMKilled"} == 1
    for: 0m      # Fire immediately
    labels:

```

```

    annotations:
      summary: "Pod {{ $labels.pod }} was OOMKilled"
      description: "Container {{ $labels.container }} in pod {{
$labels.pod }} was killed due to OOM"
    # Alert: Disk space running low
    - alert: DiskSpaceLow
      expr: |
        (node_filesystem_avail_bytes{fstype!="tmpfs"}
        / node_filesystem_size_bytes{fstype!="tmpfs"}) * 100 < 15
      for: 5m
      labels:
        severity: warning
      annotations:
        summary: "Disk space below 15% on {{ $labels.instance }}"
        description: "Available: {{ $value | humane }}%"

```

Q31. [Intermediate] How do you set up a production-grade Grafana dashboard?

Answer:

A Grafana dashboard is only as useful as the questions it answers. Production dashboards should give you the answer to *"Is my service healthy right now?"* within 5 seconds of opening it.

The RED method (the gold standard for service dashboards):

- **Rate** — how many requests per second is the service handling?
- **Errors** — what fraction of requests are failing?
- **Duration** — how long are requests taking?

Dashboard as Code using Grafana's JSON model (provisioned via ConfigMap in Kubernetes):

```

# grafana-dashboard-configmap.yaml
apiVersion: v1
kind: ConfigMap

```



```

metadata:
  name: api-dashboard
  namespace: monitoring
  labels:
    grafana_dashboard: "1" # Grafana sidecar picks this up automatically
data:
  api-dashboard.json: |
    {
      "title": "API Service - RED Dashboard",
      "uid": "api-service-red",
      "panels": [
        {
          "title": "Request Rate (req/s)",
          "type": "stat",
          "gridPos": {"x": 0, "y": 0, "w": 6, "h": 4},
          "targets": [{
            "expr": "sum(rate(http_requests_total{job='api'}[5m]))",
            "legendFormat": "Requests/s"
          }],
          "fieldConfig": {
            "defaults": {
              "color": {"mode": "thresholds"},
              "thresholds": {
                "steps": [
                  {"color": "green", "value": null},
                  {"color": "yellow", "value": 500},
                  {"color": "red", "value": 1000}
                ]
              }
            }
          }
        },
        {
          "title": "Error Rate (%)",
          "type": "stat",
          "gridPos": {"x": 6, "y": 0, "w": 6, "h": 4},
          "targets": [{

```

```

      "expr":
"sum(rate(http_requests_total{job='api',status=~'5..'}[5m])) /
sum(rate(http_requests_total{job='api'}[5m])) * 100",
      "legendFormat": "Error %"
    }],
    "fieldConfig": {
      "defaults": {
        "unit": "percent",
        "thresholds": {
          "steps": [
            {"color": "green", "value": null},
            {"color": "yellow", "value": 1},
            {"color": "red", "value": 5}
          ]
        }
      }
    }
  ]
}

```

Alertmanager routing configuration

```

# alertmanager.yml
global:
  slack_api_url: 'https://hooks.slack.com/services/YOUR/SLACK/WEBHOOK'
route:
  group_by: ['alertname', 'cluster', 'service']
  group_wait: 30s      # Wait 30s before sending first notification (batches
                        # alerts)
  group_interval: 5m   # Send updates every 5m if alert is still firing
  repeat_interval: 4h  # Re-notify every 4h if not resolved
  receiver: 'slack-default'
  routes:
    - match:
        severity: critical
      receiver: 'pagerduty-critical'

```

```

    continue: true    # Also send to default receiver
  - match:
      team: database
      receiver: 'slack-database-team'
receivers:
  - name: 'slack-default'
    slack_configs:
      - channel: '#alerts'
        title: '{{ .GroupLabels.alertname }}'
        text: |
          {{ range .Alerts }}
          *Alert:* {{ .Annotations.summary }}
          *Details:* {{ .Annotations.description }}
          *Severity:* {{ .Labels.severity }}
          {{ end }}
        send_resolved: true
  - name: 'pagerduty-critical'
    pagerduty_configs:
      - service_key: '{{ .SecretKeyRef.pagerduty_key }}'
inhibit_rules:
  # If a critical alert fires, suppress warning-level alerts for the same
  service
  - source_match:
      severity: critical
    target_match:
      severity: warning
    equal: ['alertname', 'service']

```

Key Takeaways — Prometheus & Grafana

- Prometheus uses a pull model — it scrapes `/metrics` endpoints on a schedule
- Use `rate()` for counters and `histogram_quantile()` for latency percentiles
- The RED method (Rate, Errors, Duration) is the foundation of any service dashboard
- Use `for: 2m` in alert rules to avoid alerting on transient spikes
- Route critical alerts to PagerDuty, warnings to Slack — not everything needs to wake someone up

CHAPTER 8: ELK STACK (ELASTICSEARCH, LOGSTASH, KIBANA)

Introduction to Log Management

Logs are the audit trail of your system — they tell you *what happened* when a problem occurred, where metrics only tell you *that* a problem occurred. The ELK Stack (Elasticsearch + Logstash + Kibana) provides log ingestion, storage, and visualization. In modern deployments, Filebeat or Fluentd typically replaces Logstash for log shipping.

Q32. [Beginner] Explain the ELK Stack architecture. What does each component do?

Answer:

The ELK Stack components:



Elasticsearch: A distributed search and analytics engine built on Apache Lucene. It stores data as JSON documents and provides near-real-time full-text search. Key concepts:

- **Index** — equivalent to a database table (e.g., `logs-production-2026.03.24`)

- **Document** — a single log entry stored as JSON
- **Shard** — an index is divided into shards, distributed across nodes for scalability
- **Replica** — a copy of a shard for high availability

Logstash: A data processing pipeline. Takes data from various inputs (Beats, Kafka, syslog), applies filters (grok parsing, timestamp extraction, geolocation lookup), and outputs to Elasticsearch or elsewhere.

Kibana: The visualization layer. Provides Discover (search logs), Visualize (charts), Dashboard (combine visualizations), and Lens (drag-and-drop analytics).

Filebeat: A lightweight Go agent that tails log files and ships them to Logstash or directly to Elasticsearch. Deployed as a DaemonSet in Kubernetes so every node has one.

Q33. [Intermediate] How do you deploy the ELK stack on Kubernetes for production log aggregation?

Answer:

In production Kubernetes environments, the modern approach is: **Filebeat DaemonSet** → **Logstash (or directly to Elasticsearch)** → **Elasticsearch Cluster** → **Kibana**.

Filebeat DaemonSet — ships logs from all pods:

```
# filebeat-daemonset.yaml
apiVersion: app/v1
kind: DaemonSet
metadata:
  name: filebeat
  namespace: logging
spec:
  selector:
    matchLabels:
      app: filebeat
  template:
    metadata:
```

```
labels:
  app: filebeat
spec:
  serviceAccountName: filebeat
  terminationGracePeriodSeconds: 30
  containers:
    - name: filebeat
      image: docker.elastic.co/beats/filebeat:8.12.0
      args: ["-c", "/etc/filebeat.yml", "-e"]
      env:
        - name: ELASTICSEARCH_HOST
          value: "elasticsearch:9200"
        - name: NODE_NAME
          valueFrom:
            fieldRef:
              fieldPath: spec.nodeName
      resources:
        requests:
          memory: "100Mi"
          cpu: "100m"
        limits:
          memory: "200Mi"
          cpu: "200m"
      volumeMounts:
        - name: config
          mountPath: /etc/filebeat.yml
          subPath: filebeat.yml
        - name: varlog
          mountPath: /var/log
          readOnly: true
        - name: varlibdockercontainers
          mountPath: /var/lib/docker/containers
          readOnly: true
  volumes:
    - name: config
      configMap:
        name: filebeat-config
```

```

    hostPath:
      path: /var/log
  - name: varlibdockercontainers
    hostPath:
      path: /var/lib/docker/containers

```

Filebeat configuration:

```

# filebeat-config ConfigMap
filebeat.inputs:
  - type: container
    paths:
      - /var/lib/docker/containers/**/*.log
    processors:
      - add_kubernetes_metadata:
          host: ${NODE_NAME}
          matchers:
            - logs_path:
                logs_path: "/var/lib/docker/containers/"
      - drop_event:
          when:
            contains:
              kubernetes.namespace: "kube-system" # Skip system logs
processors:
  - add_cloud_metadata: ~
  - add_host_metadata: ~
output.logstash:
  hosts: ["logstash:5044"]
# OR output directly to Elasticsearch:
# output.elasticsearch:
#   hosts: ["https://elasticsearch:9200"]
#   index: "k8s-logs-%[kubernetes.namespace]}-%{+yyyy.MM.dd}"
#   username: "${ELASTICSEARCH_USER}"
#   password: "${ELASTICSEARCH_PASSWORD}"

```

Logstash pipeline for parsing application logs:

```

# logstash-pipeline.conf
input {
  beats {
    port => 5044
    ssl => false
  }
}
filter {
  # Parse JSON logs from applications that log in JSON format
  if [message] =~ /^{/ {
    json {
      source => "message"
      target => "app"
    }
  }
  # Parse traditional Apache/Nginx access logs
  if [kubernetes][container][name] == "nginx" {
    grok {
      match => {
        "message" => '%{IPORHOST:client_ip} - %{DATA:user}
\[%{HTTPDATE:timestamp}\] "%{WORD:method} %{DATA:request}
HTTP/%{NUMBER:http_version}" %{NUMBER:response_code} %{NUMBER:bytes}'
      }
    }
    mutate {
      convert => {
        "response_code" => "integer"
        "bytes" => "integer"
      }
    }
  }
  # Add geo-location for client IPs
  geoip {
    source => "client_ip"
    target => "geo"
  }
  # Drop health check logs to reduce noise

```



```

if [request] =~ "/health" {
  drop {}
}
# Add severity field based on log level
if [app][level] {
  mutate {
    add_field => { "log_severity" => "%{[app][level]}" }
  }
}
}
output {
  elasticsearch {
    hosts => ["https://elasticsearch:9200"]
    index => "k8s-logs-%{[kubernetes][namespace]}-%{+YYYY MM.dd}"
    user => "${ES_USER}"
    password => "${ES_PASSWORD}"
    ssl => true
    cacert => "/etc/logstash/certs/ca.crt"
  }
}

```

Kibana search queries (KQL — Kibana Query Language):

```

# Find all error logs from the API service
kubernetes.namespace: "production" AND kubernetes.app: "api" AND app.level:
"ERROR"
# Find logs with specific error message
message: "connection refused" AND NOT kubernetes.namespace: "kube-system"
# Find 5xx errors in the last hour
response_code >= 500 AND @timestamp > "now-1h"
# Find slow requests (> 2 seconds)
app.duration_ms > 2000 AND kubernetes.namespace: "production"

```

Key Takeaways — ELK Stack

- Filebeat is the lightweight shipper (DaemonSet in K8s); Logstash handles transformation;

- Always structure your logs as JSON in applications — it eliminates the need for complex Grok patterns
- Use index lifecycle management (ILM) in Elasticsearch to automatically delete old indices and save storage costs
- Add Kubernetes metadata (namespace, pod name, container name) to every log entry — it's invaluable during incidents

Prepared by Arvind Verma

CHAPTER 9: ANSIBLE

Introduction to Ansible

Ansible is an agentless configuration management and automation tool. Unlike Puppet or Chef, which require agents on target machines, Ansible connects via SSH and runs tasks on remote systems using Python. It's widely used for server configuration, application deployment, and orchestrating complex multi-step operations.

Q34. [Beginner] What is Ansible, and how does it differ from Terraform?

Answer:

This is a very common interview question, and the key is understanding that **Ansible and Terraform solve different problems** — they're complementary, not competing.

Ansible is a configuration management tool:

- Installs software, configures services, manages files
- Procedural — you define *how* to get to the desired state
- Agentless — uses SSH
- Good for: Server setup, application deployment, ad-hoc task automation
- State management: Stateless (runs tasks top-to-bottom; no state file)

Terraform is an infrastructure provisioning tool:

- Creates, modifies, and destroys cloud resources (EC2 instances, VPCs, RDS databases)
- Declarative — you define *what* the desired state is
- API-based — communicates with cloud APIs
- Good for: Provisioning cloud infrastructure
- State management: Maintains a state file (terraform.tfstate)

Typical combined workflow:

Terraform provisions:	Ansible configures:
→ EC2 instances	→ Installs Nginx on EC2
→ Security groups	→ Deploys application code
→ RDS database	→ Configures application settings
→ VPC/subnets	→ Sets up monitoring agent

Interview answer: "We use Terraform to provision the infrastructure and Ansible to configure what runs on it. Terraform tells AWS 'create 3 EC2 instances,' and Ansible tells those instances 'install Nginx, deploy the application, and configure the firewall rules.' They work at different layers of the stack."

Q35. [Intermediate] Write an Ansible playbook for a production-grade web server setup.

Answer:

```
# playbook-webserver.yml
---
- name: Configure Production Web Server
  hosts: web_servers
  become: true          # Run tasks as sudo
  gather_facts: true    # Collect system facts first
  vars:
    app_name: my-api
    app_user: api user
    app_dir: /opt/{{ app_name }}
    nginx_config_dir: /etc/nginx/conf.d
    node_version: "18"
  pre_tasks:
    - name: Update apt cache
      apt:
        update_cache: yes
        cache_valid_time: 3600    # Only update if cache is older than 1 hour
    - name: Ensure required packages are installed
      apt:
        name:
```

```

        - git
        - curl
        - unzip
        - python3-pip
        - ufw

    state: present

tasks:
  # ---- User Setup ----
  - name: Create application user
    user:
      name: "{{ app_user }}"
      system: true
      shell: /bin/false
      home: "{{ app_dir }}"
      create_home: true
  # ---- Node.js Installation ----
  - name: Add NodeSource apt repository
    shell: |
      curl -fsSL https://deb.nodesource.com/setup_{{ node_version }}.x |
bash -
      args:
        creates: /etc/apt/sources.list.d/nodesource.list # Don't run if
already exists
  - name: Install Node.js
    apt:
      name: nodejs
      state: present
  # ---- Application Deployment ----
  - name: Clone application repository
    git:
      repo: "https://github.com/myorg/{{ app_name }}.git"
      dest: "{{ app_dir }}/src"
      version: "{{ app_version | default('main') }}"
      force: yes
      notify: restart application # Trigger handler on change
  - name: Install npm dependencies
    npm:

```

```

    production: yes    # --only=production
- name: Set correct file ownership
  file:
    path: "{{ app_dir }}"
    owner: "{{ app_user }}"
    group: "{{ app_user }}"
    recurse: true
# ---- Environment Configuration ----
- name: Create environment file from template
  template:
    src: templates/env.j2
    dest: "{{ app_dir }}/.env"
    owner: "{{ app_user }}"
    mode: '0600'    # Read/write by owner only
  notify: restart application
# ---- Systemd Service ----
- name: Create systemd service file
  template:
    src: templates/app.service.j2
    dest: /etc/systemd/system/{{ app_name }}.service
  notify:
    - reload systemd
    - restart application
- name: Enable and start application service
  systemd:
    name: "{{ app_name }}"
    enabled: true
    state: started
# ---- Nginx Configuration ----
- name: Install Nginx
  apt:
    name: nginx
    state: present
- name: Deploy Nginx virtual host config
  template:
    src: templates/nginx-vhost.conf.j2
    dest: "{{ nginx_config_dir }}/{{ app_name }}.conf"

```

```

    notify: reload nginx
- name: Remove default Nginx site
  file:
    path: /etc/nginx/sites-enabled/default
    state: absent
  notify: reload nginx
# ---- Firewall Configuration ----
- name: Allow SSH
  ufw:
    rule: allow
    port: '22'
    proto: tcp
- name: Allow HTTP
  ufw:
    rule: allow
    port: '80'
    proto: tcp
- name: Allow HTTPS
  ufw:
    rule: allow
    port: '443'
    proto: tcp
- name: Enable UFW
  ufw:
    state: enabled
    policy: deny # Default deny all inbound
# ---- Security Hardening ----
- name: Disable root SSH login
  lineinfile:
    path: /etc/ssh/sshd_config
    regexp: '^PermitRootLogin'
    line: 'PermitRootLogin no'
    backup: yes
  notify: restart sshd
- name: Disable password authentication
  lineinfile:
    path: /etc/ssh/sshd_config

```

```

    line: 'PasswordAuthentication no'
    notify: restart sshd
handlers:
    # Handlers only run when notified AND only once at the end of the play
    - name: reload systemd
      systemd:
        daemon_reload: true
    - name: restart application
      systemd:
        name: "{{ app_name }}"
        state: restarted
    - name: reload nginx
      service:
        name: nginx
        state: reloaded
    - name: restart sshd
      service:
        name: sshd
        state: restarted
post_tasks:
    - name: Verify application is responding
      uri:
        url: "http://localhost:3000/health"
        return_content: yes
        status_code: 200
      retries: 5
      delay: 10
      register: health_check
    - name: Print health check result
      debug:
        msg: "Application health: {{ health_check.json.status }}"

```

Running the playbook:

```

# Check inventory
cat inventory/production.ini
# [web_servers]

```



```

# web01.company.com
# web02.company.com
# web03.company.com
# Dry run first (--check mode)
ansible-playbook -i inventory/production.ini playbook-webserver.yml \
  --check \
  --diff \
  -e "app_version=v1.2.3"
# Actual run
ansible-playbook -i inventory/production.ini playbook-webserver.yml \
  --extra-vars "app_version=v1.2.3" \
  --tags "deploy,nginx" \
  -v      # verbose output
# Run only specific tags
ansible-playbook -i inventory/production.ini playbook-webserver.yml \
  --tags "deploy"
# Run against a single host for testing
ansible-playbook -i inventory/production.ini playbook-webserver.yml \
  --limit "web01.company.com"

```

Q36. [Advanced] How do you use Ansible Vault to manage secrets, and how does it integrate with a CI/CD pipeline?

Answer:

Ansible Vault encrypts sensitive data (passwords, API keys, certificates) using AES256 encryption, allowing you to safely commit encrypted secrets to version control.

```

# Create an encrypted variables file
ansible-vault create group_vars/production/vault.yml
# Opens editor – type your secrets:
# vault_db_password: "super-secret-db-password"
# vault_api_key: "sk-prod-abc123xyz"
# vault_ssl_certificate: |
#   -----BEGIN CERTIFICATE-----
#   ...

```

```

ansible-vault encrypt group_vars/production/vars.yml
# View encrypted file (prompts for password)
ansible-vault view group_vars/production/vault.yml
# Edit encrypted file
ansible-vault edit group_vars/production/vault.yml
# Re-key (change the vault password)
ansible-vault rekey group_vars/production/vault.yml
# Encrypt a single string (useful for embedding in non-encrypted files)
ansible-vault encrypt_string 'super-secret-password' --name 'db_password'
# Output:
# db_password: !vault |
#   $ANSIBLE_VAULT;1.1;AES256
#   38623665353561343...

```

CI/CD integration — storing vault password in GitHub Secrets:

```

# .github/workflows/deploy.yml
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Install Ansible
        run: pip install ansible
      - name: Create vault password file
        run: |
          echo "${{ secrets.ANSIBLE_VAULT_PASSWORD }}" > .vault-password
          chmod 600 .vault-password
      - name: Run Ansible Playbook
        run: |
          ansible-playbook \
            -i inventory/production.ini \
            playbook-webserver.yml \
            --vault-password-file .vault-password \
            -e "app_version=${{ github.sha }}"
      - name: Clean up vault password file
        if: always() # Run even if previous step failed

```

```
run: rm -f .vault-password
```

Directory structure with Vault best practices:

```
ansible/
├─ group_vars/
│   └─ all/
│       └─ vars.yml          ← non-sensitive variables
│       └─ vault.yml        ← ENCRYPTED sensitive variables
│   └─ production/
│       └─ vars.yml
│       └─ vault.yml        ← production-specific secrets
├─ inventory/
│   └─ production.ini
│   └─ staging.ini
├─ roles/
│   └─ webserver/
│   └─ database/
└─ playbook-webserver.yml
```

Key Takeaways — Ansible

- Ansible is for configuration management; Terraform is for infrastructure provisioning — use both together
- Idempotency is the key principle — running a playbook 10 times should have the same result as running it once
- Handlers only run when notified and only once — perfect for service restarts
- Use `--check --diff` before every production run to preview changes
- Ansible Vault + CI/CD secrets = safe, auditable secret management

CHAPTER 10: TERRAFORM

Introduction to Terraform

Terraform by HashiCorp is the most widely used Infrastructure as Code tool across multiple cloud providers. Unlike AWS CloudFormation (AWS-only), Terraform works with 300+ providers — AWS, Azure, GCP, Kubernetes, Datadog, GitHub, and more. Its declarative HCL syntax and state-based approach make it the go-to choice for managing infrastructure at scale.

Q37. [Beginner] Explain Terraform's core workflow: init, plan, apply, destroy.

Answer:

Terraform follows a three-command workflow for all infrastructure changes. Understanding what each command does — and does not do — is fundamental.

```
# 1. terraform init
# Downloads providers, initializes backend, prepares working directory
terraform init
# What it does:
# - Downloads provider plugins (aws, kubernetes, etc.) to .terraform/
# - Initializes the backend (where state file is stored)
# - Downloads modules referenced in configuration
# - Does NOT connect to your infrastructure or make any changes
# 2. terraform plan
# Shows what changes Terraform will make – without making them
terraform plan -out=tfplan
# Output symbols:
# + = resource will be CREATED
# - = resource will be DESTROYED
# ~ = resource will be MODIFIED IN-PLACE
# -/+ = resource will be DESTROYED and RECREATED
# 3. terraform apply
# Makes the actual changes
```

```
# OR (interactive):
terraform apply    # Shows plan again and asks for confirmation
# 4. terraform destroy
# Destroys all resources managed by this configuration
terraform destroy
# DANGER: This destroys real infrastructure!
# Always run with -target for specific resources in production
terraform destroy -target=aws_instance.web_server
```

A complete, practical example — provisioning an EC2 instance:

```
# main.tf
terraform {
  required_version = ">= 1.7.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"    # Allow 5.x but not 6.x
    }
  }
}
# Remote state backend – critical for team collaboration
backend "s3" {
  bucket         = "my-terraform-state-bucket"
  key             = "production/ec2/terraform.tfstate"
  region         = "ap-south-1"
  encrypt        = true
  dynamodb_table = "terraform-state-lock"    # Prevents concurrent runs
}
}
provider "aws" {
  region = var.aws_region
  default_tags {
    tags = {
      Environment = var.environment
      ManagedBy   = "Terraform"
      Project     = var.project_name
    }
  }
}
```

```

    }
}
# variables.tf
variable "aws_region" {
    description = "AWS region for all resources"
    type        = string
    default     = "ap-south-1"
}
variable "environment" {
    description = "Deployment environment"
    type        = string
    validation {
        condition     = contains(["dev", "staging", "prod"], var.environment)
        error_message = "Environment must be dev, staging, or prod."
    }
}
variable "instance_type" {
    description = "EC2 instance type"
    type        = string
    default     = "t3.medium"
}
# Data sources – read existing resources
data "aws_ami" "amazon_linux_2" {
    most_recent = true
    owners      = ["amazon"]
    filter {
        name     = "name"
        values   = ["amzn2-ami-hvm-*-x86_64-gp2"]
    }
}
data "aws_vpc" "default" {
    default = true
}
# Security group
resource "aws_security_group" "web" {
    name_prefix = "${var.environment}-web-"
    vpc_id      = data.aws_vpc.default.id
}

```

```

    from_port    = 443
    to_port      = 443
    protocol     = "tcp"
    cidr_blocks  = ["0.0.0.0/0"]
}
ingress {
    from_port    = 80
    to_port      = 80
    protocol     = "tcp"
    cidr_blocks  = ["0.0.0.0/0"]
}
egress {
    from_port    = 0
    to_port      = 0
    protocol     = "-1"
    cidr_blocks  = ["0.0.0.0/0"]
}
lifecycle {
    create_before_destroy = true    # Create new SG before destroying old one
}
}
# EC2 instance
resource "aws_instance" "web" {
    ami          = data.aws_ami.amazon_linux_2.id
    instance_type = var.instance_type
    vpc_security_group_ids = [aws_security_group.web.id]
    user_data = base64encode(<<-EOF
        #!/bin/bash
        yum update -y
        yum install -y nginx
        systemctl enable nginx
        systemctl start nginx
    EOF
    )
    root_block_device {
        volume_type = "gp3"
        volume_size = 20
    }
}

```

```

}
metadata_options {
  http_endpoint      = "enabled"
  http_tokens        = "required" # IMDSv2 only – security best
practice
  http_put_response_hop_limit = 1
}
}
# outputs.tf
output "instance_public_ip" {
  description = "Public IP of the web server"
  value       = aws_instance.web.public_ip
}
output "instance_id" {
  description = "EC2 instance ID"
  value       = aws_instance.web.id
}

```

Q38. [Intermediate] How do you structure Terraform code for a large production environment with multiple environments?

Answer:

Structuring Terraform for multiple environments is where many teams make mistakes. The goal is to avoid code duplication while maintaining environment isolation.

Recommended: Terraform Workspaces + modules (or separate state per environment)

The module-based approach (most production-grade):

```

terraform/
├─ modules/                ← reusable building blocks
│   └─ vpc/
│       └─ main.tf
│       └─ variables.tf
│       └─ outputs.tf
│   └─ s3-service/

```



```

|   |   |─ main.tf
|   |   |─ variables.tf
|   |   └─ outputs.tf
|   └─ rds/
|       |─ main.tf
|       |─ variables.tf
|       └─ outputs.tf
|
|─ environments/
|   |─ dev/
|   |   |─ main.tf          ← calls modules with dev values
|   |   |─ variables.tf
|   |   └─ terraform.tfvars ← dev-specific values
|   |─ staging/
|   |   |─ main.tf
|   |   |─ variables.tf
|   |   └─ terraform.tfvars
|   └─ prod/
|       |─ main.tf
|       |─ variables.tf
|       └─ terraform.tfvars
|
|─ global/
|   |─ iam/          ← IAM roles (shared across envs)
|   └─ route53/      ← DNS (shared)

```

The ECS service module:

```

# modules/ecs-service/main.tf
variable "service_name" { type = string }
variable "container_image" { type = string }
variable "cpu" { type = number; default = 256 }
variable "memory" { type = number; default = 512 }
variable "desired_count" { type = number; default = 1 }
variable "environment_variables" { type = map(string); default = {} }
variable "cluster_id" { type = string }
variable "subnet_ids" { type = list(string) }

```

```

variable "security_group_ids" { type = list(string) }
variable "target_group_arn" { type = string }
resource "aws_ecs_task_definition" "service" {
  family           = var.service_name
  network_mode     = "awsvpc"
  requires_compatibilities = ["FARGATE"]
  cpu              = var.cpu
  memory           = var.memory
  execution_role_arn = aws_iam_role.task_execution.arn
  task_role_arn     = aws_iam_role.task.arn
  container_definitions = jsonencode([
    {
      name = var.service_name
      image = var.container_image
      portMappings = [
        {
          containerPort = 8080
          protocol       = "tcp"
        }
      ]
      environment = [
        for key, value in var.environment_variables : {
          name = key
          value = value
        }
      ]
      logConfiguration = {
        logDriver = "awslogs"
        options = {
          "awslogs-group"           = "/ecs/${var.service_name}"
          "awslogs-region"         = data.aws_region.current.name
          "awslogs-stream-prefix" = "ecs"
        }
      }
    }
  ])
}

resource "aws_ecs_service" "service" {
  name           = var.service_name
  cluster        = var.cluster_id
  task_definition = aws_ecs_task_definition.service.arn

```

```

launch_type      = "FARGATE"
network_configuration {
    subnets          = var.subnet_ids
    security_groups   = var.security_group_ids
    assign_public_ip  = false
}
load_balancer {
    target_group_arn = var.target_group_arn
    container_name   = var.service_name
    container_port   = 8080
}
lifecycle {
    ignore_changes = [desired_count] # Don't reset when autoscaling changes
count
}
}
output "service_name" { value = aws_ecs_service.service.name }
output "task_definition_arn" { value = aws_ecs_task_definition.service.arn }

```

Calling the module from environment:

```

# environments/prod/main.tf
module "api_service" {
    source = "../../modules/ecs-service"
    service_name      = "my-api"
    container_image    = "123456789.dkr.ecr.ap-south-1.amazonaws.com/my-
api:${var.image_tag}"
    cpu                = 1024    # Production gets more resources
    memory             = 2048
    desired_count      = 3
    cluster_id         = module.ecs_cluster.cluster_id
    subnet_ids         = module.vpc.private_subnet_ids
    security_group_ids = [module.app_sg.id]
    target_group_arn   = module.alb.api_target_group_arn
    environment_variables = {
        ENVIRONMENT = "production"
        LOG_LEVEL   = "warn"
    }
}

```

```
CACHE_TTL    = "300"
}
}
```

Q39. [Advanced] How do you handle Terraform state management and prevent state corruption in a team environment?

Answer:

Terraform state is the file that maps your configuration to real-world resources. Corrupting or losing this file is one of the most painful experiences in infrastructure management.

Remote state with S3 + DynamoDB locking:

```
# Setting up the S3 backend (done once)
terraform {
  backend "s3" {
    bucket      = "company-terraform-state"
    key         = "services/api/production/terraform.tfstate"
    region      = "ap-south-1"
    encrypt     = true
    kms_key_id  = "arn:aws:kms:ap-south-1:123456789:key/abc123"
    # DynamoDB table for state locking
    # Prevents two people from running terraform apply simultaneously
    dynamodb_table = "terraform-state-locks"
  }
}
```

Creating the S3 bucket and DynamoDB table (bootstrap):

```
# bootstrap/main.tf – run this ONCE manually before using remote state
resource "aws_s3_bucket" "terraform_state" {
  bucket = "company-terraform-state"
}
resource "aws_s3_bucket_versioning" "terraform_state" {
  bucket = aws_s3_bucket.terraform_state.id
}
```

```

    versioning_configuration {
      status = "Enabled"    # Enables recovery from accidental state changes
    }
  }
  resource "aws_s3_bucket_server_side_encryption_configuration"
  "terraform_state" {
    bucket = aws_s3_bucket.terraform_state.id
    rule {
      apply_server_side_encryption_by_default {
        sse_algorithm      = "aws:kms"
        kms_master_key_id = aws_kms_key.terraform.arn
      }
    }
  }
}
resource "aws_s3_bucket_public_access_block" "terraform_state" {
  bucket                  = aws_s3_bucket.terraform_state.id
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls     = true
  restrict_public_buckets = true
}
resource "aws_dynamodb_table" "terraform_locks" {
  name          = "terraform_state-locks"
  billing_mode  = "PAY_PER_REQUEST"
  hash_key     = "LockID"
  attribute {
    name = "LockID"
    type = "S"
  }
}
}

```

Handling state operations safely:

```

# Import existing resources into Terraform state
# (for resources created outside Terraform)
terraform import aws_s3_bucket.my_bucket my-existing-bucket-name
# Move resources in state (after refactoring)

```

```
terraform state mv aws_instance.web aws_instance.web_server
# Remove resource from state WITHOUT deleting it
# (useful when you want Terraform to "forget" about a resource)
terraform state rm aws_instance.temp_test_server
# List all resources in state
terraform state list
# Show detailed state for a specific resource
terraform state show aws_instance.web_server
# Pull remote state to local
terraform state pull > state-backup.json
# Force-unlock state (if a previous run died mid-way and left a lock)
terraform force-unlock LOCK_ID
```

CI/CD pipeline with Terraform:

```
# .github/workflows/terraform.yml
name: Terraform CI/CD
on:
  pull_request:
    paths: ['terraform/**']
  push:
    branches: [main]
    paths: ['terraform/**']
jobs:
  terraform-plan:
    runs-on: ubuntu-latest
    permissions:
      id-token: write
      contents: read
      pull-requests: write
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
        with:
          terraform_version: 1.7.0
      - name: Configure AWS via OIDC
        uses: aws-actions/configure-aws-credentials@v4
```

```

    with:
      role-to-assume: arn:aws:iam::123456789:role/TerraformPlanRole
      aws-region: ap-south-1
- name: Terraform Init
  working-directory: terraform/environments/prod
  run: terraform init
- name: Terraform Format Check
  run: terraform fmt -check -recursive
- name: Terraform Validate
  working-directory: terraform/environments/prod
  run: terraform validate
- name: Terraform Plan
  id: plan
  working-directory: terraform/environments/prod
  run: terraform plan -no-color -out=tfplan
  continue-on-error: true
- name: Comment Plan on PR
  uses: actions/github-script@v7
  if: github.event_name == 'pull_request'
  with:
    script: |
      const output = `### Terraform Plan 📋
      \\\`
      ${ steps.plan.outputs.stdout }
      \\\`;
      github.rest.issues.createComment({
        issue_number: context.issue.number,
        owner: context.repo.owner,
        repo: context.repo.repo,
        body: output
      })

terraform-apply:
  runs-on: ubuntu-latest
  needs: terraform-plan
  if: github.ref == 'refs/heads/main' && github.event_name == 'push'
  environment: production # Requires manual approval
  steps:

```

```

- uses: hashicorp/setup-terraform@v3
- name: Configure AWS via OIDC
  uses: aws-actions/configure-aws-credentials@v4
  with:
    role-to-assume: arn:aws:iam::123456789:role/TerraformApplyRole #
Higher permissions
    aws-region: ap-south-1
- run: terraform init
  working-directory: terraform/environments/prod
- name: Terraform Apply
  working-directory: terraform/environments/prod
  run: terraform apply -auto-approve tfplan

```

Key Takeaways — Terraform

- Always use remote state (S3 + DynamoDB lock) — never commit `terraform.tfstate` to Git
- Use modules to avoid code duplication across environments
- Separate plan and apply roles — `plan` needs read-only permissions; `apply` needs write permissions
- `terraform plan -out=tfplan` saves the exact plan so `apply` executes what was reviewed
- Use `lifecycle { prevent_destroy = true }` on critical resources like databases to prevent accidental deletion

Common Pitfalls — Terraform

- Committing state files to Git — contains sensitive data and causes concurrent-access corruption
- Not pinning provider versions — a provider update can break your configuration
- Using `terraform apply` without a saved plan — the actual change might differ from what you reviewed
- Not using modules — leads to massive copy-paste across environments that drift over time
- Forgetting `ignore_changes` on autoscaling `desired_count` — Terraform resets it every `apply`

CHAPTER 11: AZURE DEVOPS

Introduction to Azure DevOps

Azure DevOps is Microsoft's end-to-end DevOps platform. It includes Azure Boards (project management), Azure Repos (Git hosting), Azure Pipelines (CI/CD), Azure Artifacts (package registry), and Azure Test Plans. It's deeply integrated with the Microsoft ecosystem and widely used in enterprise environments, especially those running .NET workloads or using Azure cloud.

Q40. [Beginner] What is Azure DevOps, and what are its main components?

Answer:

Azure DevOps is a suite of development services that together cover the full software development lifecycle:

Service	Purpose	Equivalent
Azure Boards	Agile project management, work items, sprints	Jira
Azure Repos	Git repository hosting	GitHub/GitLab
Azure Pipelines	CI/CD automation	Jenkins/GitHub Actions
Azure Artifacts	Package registry (NuGet, npm, Maven, Docker)	Nexus/JFrog Artifactory
Azure Test Plans	Manual and exploratory testing	TestRail

Azure Pipelines is the most commonly used component in DevOps interviews. It supports:

- YAML-based pipelines (pipeline as code)
- Classic pipelines (GUI-based, legacy)
- Multi-stage pipelines
- Environments with approval gates
- Integration with GitHub, Bitbucket, and Azure Repos

Q41. [Intermediate] Write a production-grade Azure Pipeline YAML for a .NET application deployment to AKS.

Answer:

```
# azure-pipelines.yml
trigger:
  branches:
    include:
      - main
      - release/*
  paths:
    exclude:
      - docs/**
      - README.md
pr:
  branches:
    include:
      - main
variables:
  - group: production-secrets      # Variable group from Azure Key Vault link
  - name: dockerfilePath
    value: '$(Build.SourcesDirectory)/Dockerfile'
  - name: imageRepository
    value: 'my-dotnet-api'
  - name: containerRegistry
    value: 'mycompanyacr.azurecr.io'
  - name: tag
    value: '$(Build.BuildId)'
  - name: k8sNamespace
    value: 'production'
stages:
  # ---- STAGE 1: Build and Test ----
  - stage: BuildAndTest
    displayName: 'Build and Test'
    jobs:
      - job: Build
        displayName: 'Build .NET Application'
        pool:
          vmImage: 'ubuntu-latest'
```

```

steps:
  - task: UseDotNet@2
    displayName: 'Use .NET 8 SDK'
    inputs:
      packageType: 'sdk'
      version: '8.0.x'
  - task: DotNetCoreCLI@2
    displayName: 'Restore NuGet packages'
    inputs:
      command: 'restore'
      projects: '**/*.csproj'
      feedsToUse: 'select'
      vstsFeed: '$(System.TeamProjectId)/my-artifacts-feed'
  - task: DotNetCoreCLI@2
    displayName: 'Build'
    inputs:
      command: 'build'
      projects: '**/*.csproj'
      arguments: '--no-restore --configuration Release'
  - task: DotNetCoreCLI@2
    displayName: 'Run Unit Tests'
    inputs:
      command: 'test'
      projects: '*/*Tests.csproj'
      arguments: '--no-build --configuration Release --collect:"XPlat
Code Coverage" --results-directory $(Agent.TempDirectory)'
  - task: PublishCodeCoverageResults@2
    displayName: 'Publish Code Coverage'
    inputs:
      summaryFileLocation:
'$(Agent.TempDirectory)**/coverage.cobertura.xml'
  - task: DotNetCoreCLI@2
    displayName: 'Publish Application'
    inputs:
      command: 'publish'
      publishWebProjects: true
      arguments: '--no-build --configuration Release --output

```

```

- task: PublishBuildArtifacts@1
  displayName: 'Publish Build Artifacts'
  inputs:
    PathToPublish: '$(Build.ArtifactStagingDirectory)'
    ArtifactName: 'drop'
# ---- STAGE 2: Build Docker Image ----
- stage: DockerBuild
  displayName: 'Build Docker Image'
  dependsOn: BuildAndTest
  condition: succeeded()
  jobs:
    - job: DockerBuild
      pool:
        vmImage: 'ubuntu-latest'
      steps:
        - task: Docker@2
          displayName: 'Build and Push to ACR'
          inputs:
            command: 'buildAndPush'
            repository: '$(imageRepository)'
            dockerfile: '$(dockerfilePath)'
            containerRegistry: 'my-acr-service-connection' # Service
connection in Azure DevOps
            tags: |
              $(tag)
              latest
# ---- STAGE 3: Deploy to Staging ----
- stage: DeployStaging
  displayName: 'Deploy to Staging'
  dependsOn: DockerBuild
  condition: succeeded()
  jobs:
    - deployment: DeployToStaging
      displayName: 'Deploy to AKS Staging'
      pool:
        vmImage: 'ubuntu-latest'
      environment: 'staging' # Environment with optional approval gates

```

```

    runOnce:
      deploy:
        steps:
          - task: KubernetesManifest@1
            displayName: 'Deploy to Kubernetes'
            inputs:
              action: 'deploy'
              connectionType: 'azureResourceManager'
              azureSubscriptionConnection: 'my-azure-subscription'
              azureResourceGroup: 'my-rg'
              kubernetesCluster: 'my-aks-cluster'
              namespace: 'staging'
              manifests: |
                kubernetes/deployment.yaml
                kubernetes/service.yaml
            containers:
              '$(containerRegistry)/$(imageRepository):$(tag)'
# ---- STAGE 4: Deploy to Production ----
- stage: DeployProduction
  displayName: 'Deploy to Production'
  dependsOn: DeployStaging
  condition: and(succeeded(), eq(variables['Build.SourceBranch'],
'refs/heads/main'))
  jobs:
    - deployment: DeployToProduction
      pool:
        vmImage: 'ubuntu-latest'
      environment: 'production' # This environment has approval gates
configured
      strategy:
        canary:
          increments: [10, 50] # Deploy to 10%, then 50%, then 100%
          deploy:
            steps:
              - task: KubernetesManifest@1
                inputs:
                  action: 'deploy'

```

```

        azureSubscriptionConnection: 'my-azure-subscription'
        azureResourceGroup: 'my-rg'
        kubernetesCluster: 'my-aks-cluster'
        namespace: '${k8sNamespace}'
        manifests: 'kubernetes/deployment.yaml'
        containers:
        '$(containerRegistry)/$(imageRepository):$(tag)'
        postRouteTraffic:
        steps:
        - bash: |
            # Run smoke tests against canary pods
            kubectl run smoke-test --image=curlimages/curl:latest --
restart=Never \
            -- curl -f http://my-api-service/health
            kubectl wait --for=condition=complete --timeout=60s
job/smoke-test
        displayName: 'Run Smoke Tests'
on:
failure:
steps:
- task: KubernetesManifest@1
  inputs:
    action: 'reject'
    namespace: '${k8sNamespace}'

```

CHAPTER 12: PRODUCTION-GRADE

PIPELINE — ADDITIONAL TOOLS

Security Scanning, Quality Gates, and Supporting Tools

A production pipeline is not just build-test-deploy. It includes security scanning, code quality enforcement, artifact management, and progressive delivery. These are the tools that separate a hobby pipeline from an enterprise-grade one.

Q42. [Intermediate] What is SonarQube, and how do you integrate it into a CI/CD pipeline?

Answer:

SonarQube is a static code analysis platform that scans your code for:

- **Bugs** — potential runtime errors
- **Code smells** — maintainability issues
- **Security vulnerabilities** — SQL injection, XSS, hardcoded passwords
- **Code coverage** — what percentage of your code is tested
- **Technical debt** — estimate of time needed to fix all issues
- **Duplicate code** — copy-paste violations

Integration in Jenkins:

```
stage('SonarQube Analysis') {
    steps {
        withSonarQubeEnv('SonarQube-Server') { // Configured in Jenkins
            sh """
                mvn sonar:sonar \
                  -Dsonar.projectKey=my-app \
                  -Dsonar.projectName='My Application' \
            """
        }
    }
}
```

```

-
Dsonar.coverage.jacoco.xmlReportPaths=target/site/jacoco/jacoco.xml
    ""
    }
  }
}
stage('Quality Gate') {
  steps {
    // Wait for SonarQube to finish analysis
    timeout(time: 10, unit: 'MINUTES') {
      waitForQualityGate abortPipeline: true
      // Pipeline FAILS if quality gate conditions are not met
    }
  }
}

```

SonarQube Quality Gate conditions (production standards):

Coverage on new code: $\geq 80\%$
 Duplicated lines on new code: $< 3\%$
 Maintainability rating on new code: A
 Reliability rating on new code: A
 Security rating on new code: A
 Security hotspots reviewed: 100%

Q43. [Intermediate] What is Trivy, and how do you scan Docker images for vulnerabilities in a pipeline?

Answer:

Trivy is an open-source vulnerability scanner that scans:

- Container images (OS packages, language dependencies)
- Filesystems and Git repositories
- Kubernetes clusters
- Infrastructure as Code (Terraform, CloudFormation)


```
# GitHub Actions – Trivy image scanning
- name: Run Trivy vulnerability scanner
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: '${{ env.ECR_REGISTRY }}/${{ env.ECR_REPOSITORY }}:${{
github.sha }}'
    format: 'sarif'
    output: 'trivy-results.sarif'
    severity: 'CRITICAL,HIGH'
    exit-code: '1'    # Fail the pipeline if CRITICAL or HIGH vulnerabilities
found
    ignore-unfixed: true # Ignore if no fix is available yet
- name: Upload Trivy scan results to GitHub Security
  uses: github/codeql-action/upload-sarif@v3
  if: always()
  with:
    sarif_file: 'trivy-results.sarif'
```

Running Trivy locally:

```
# Scan a Docker image
trivy image --severity CRITICAL,HIGH nginx:latest
# Scan a local filesystem
trivy fs --security-checks vuln,secret .
# Scan a running Kubernetes cluster
trivy k8s --report summary cluster
# Scan Infrastructure as Code
trivy config --severity HIGH,CRITICAL terraform/
# Generate a detailed HTML report
trivy image --format template \
  --template "@usr/local/share/trivy/templates/html.tpl" \
  --output report.html \
  myapp:latest
```

Q44. [Advanced] How do you implement a complete production-grade pipeline that incorporates all the tools discussed?

Answer:

Here is a holistic view of what a mature, production-grade pipeline looks like, integrating all the tools we've discussed:

Developer pushes code

↓

[GitHub] ← PR opens → Branch protection rules trigger

↓

[GitHub Actions / Jenkins] ← Webhook trigger

↓

Stage 1: CODE QUALITY

- └─ SonarQube static analysis (code smells, bugs, coverage)
- └─ ESLint / flake8 / golangci-lint (language-specific linting)
- └─ terraform fmt / validate (for IaC changes)

↓

Stage 2: BUILD

- └─ Compile / package application
- └─ Build Docker image (multi-stage, minimal)
- └─ Push to ECR / ACR / Docker Hub

↓

Stage 3: SECURITY SCAN

- └─ Trivy – scan Docker image for CVEs
- └─ Snyk – scan dependencies (npm, pip, maven)
- └─ OWASP Dependency Check – Java/Maven specific
- └─ Checkov – scan Terraform/K8s manifests for misconfigurations

↓

Stage 4: TEST

- └─ Unit tests (fast, run in parallel)
- └─ Integration tests (with real DB via Docker Compose / K8s job)
- └─ Contract tests (Pact – API contract validation)

↓

Stage 5: DEPLOY TO STAGING

- └─ Terraform plan → apply (if infra changes)
- └─ Ansible (if server configuration changed)
- └─ kubectl apply / Helm upgrade
- └─ Smoke tests against staging URL

↓
Stage 6: PERFORMANCE TESTING (on schedule or tagged releases)
├─ k6 or Locust load test against staging
└─ Fail if P99 latency > SLA threshold

↓
Stage 7: DEPLOY TO PRODUCTION (requires human approval)
├─ Blue/Green or Canary via CodeDeploy / Argo Rollouts
├─ Prometheus alerts monitoring during rollout
├─ Automatic rollback if error rate increases
└─ Notify Slack / PagerDuty on success or failure

↓
Stage 8: POST-DEPLOY VERIFICATION
├─ Synthetic monitoring (Grafana k6 Cloud / Pingdom)
├─ Check Kibana for error log spikes
└─ Verify Prometheus error rate alert is green

Helm — the Kubernetes package manager:

```
# helm/my-app/values.yaml
replicaCount: 3
image:
  repository: myregistry/my-app
  tag: "1.0.0"
  pullPolicy: IfNotPresent
service:
  type: ClusterIP
  port: 80
ingress:
  enabled: true
  className: nginx
  hosts:
    - host: api.myapp.com
      paths:
        - path: /
          pathType: Prefix
resources:
  requests:
```

```

    memory: "256Mi"
    cpu: "250m"
  limits:
    memory: "512Mi"
    cpu: "500m"
  autoscaling:
    enabled: true
    minReplicas: 3
    maxReplicas: 20
    targetCPUUtilizationPercentage: 70
# values-production.yaml (overrides for production)
replicaCount: 5
resources:
  requests:
    memory: "512Mi"
    cpu: "500m"
  limits:
    memory: "1Gi"
    cpu: "1000m"
# Deploy with Helm
helm upgrade --install my-app ./helm/my-app \
  --namespace production \
  --create-namespace \
  --values helm/my-app/values.yaml \
  --values helm/my-app/values-production.yaml \
  --set image.tag=$IMAGE_TAG \
  --atomic \           # Rollback automatically if deployment fails
  --timeout 5m \
  --wait \             # Wait until all pods are ready
# Rollback Helm release
helm rollback my-app 1 -n production
# View release history
helm history my-app -n production

```

Q45. [Advanced] What is Argo CD, and how does it implement GitOps for Kubernetes deployments?

Answer:

Argo CD is a declarative GitOps continuous delivery tool for Kubernetes. The core idea of **GitOps** is: Git is the single source of truth for both application code *and* infrastructure configuration. Argo CD continuously watches a Git repository and ensures that the Kubernetes cluster state matches what's defined in Git.

Git Repository (desired state)	Argo CD	Kubernetes Cluster (actual state)
deployment.yaml	← syncs	Running pods
service.yaml		Services
configmap.yaml		ConfigMaps

If someone manually changes something in Kubernetes (a rogue `kubectl apply`), Argo CD detects the drift and either alerts you or automatically reverts it.

Argo CD Application manifest:

```
# argocd-application.yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: my-api-production
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/myorg/k8s-manifests.git
    targetRevision: main
    path: services/my-api/production # Folder in the repo
  destination:
    server: https://kubernetes.default.svc
    namespace: production
  syncPolicy:
    automated:
```

```

    selfHeal: true # Auto-revert manual changes
    allowEmpty: false
  syncOptions:
    - CreateNamespace=true
    - PrunePropagationPolicy=foreground
    - RespectIgnoreDifferences=true
  retry:
    limit: 3
    backoff:
      duration: 5s
      factor: 2
      maxDuration: 3m
  ignoreDifferences:
    - group: apps
      kind: Deployment
      jsonPointers:
        - /spec/replicas # Ignore replica count (managed by HPA)

```

The GitOps workflow with Argo CD:

```

Developer merges PR to main
↓
CI pipeline (GitHub Actions), builds image, pushes to ECR
↓
CI pipeline updates image tag in k8s-manifests repo:
git commit -m "Update my-api image to sha-abc123"
↓
Argo CD detects change in k8s-manifests repo (polls every 3 minutes)
↓
Argo CD applies the change to the cluster
↓
Argo CD reports sync status (Synced/OutOfSync/Degraded) in UI and Slack

```

Updating image tag automatically in CI:

```

# In your GitHub Actions CI pipeline, after building and pushing the image:
- name: Update image tag in GitOps repo

```

```

run: |
  git clone https://x-access-token:${{ secrets.GITOPS_TOKEN
}}@github.com/myorg/k8s-manifests.git
  cd k8s-manifests
  # Use sed or yq to update the image tag
  yq e '.spec.template.spec.containers.image = "myregistry/my-api:${{
github.sha }}"' \
    -i services/my-api/production/deployment.yaml
  git config user.email "ci@company.com"
  git config user.name "CI Bot"
  git add .
  git commit -m "ci: update my-api image to ${{ github.sha }}"
  git push

```

Argo CD CLI commands:

```

# Login to Argo CD
argocd login argocd.company.com --username admin --password $ARGOCD_PASSWORD
# List all applications
argocd app list
# Get application status
argocd app get my-api-production
# Manual sync (force reconciliation)
argocd app sync my-api-production
# Rollback to previous version
argocd app rollback my-api-production 3 # 3 = history revision number
# View sync history
argocd app history my-api-production
# Create application from CLI
argocd app create my-api-staging \
  --repo https://github.com/myorg/k8s-manifests.git \
  --path services/my-api/staging \
  --dest-server https://kubernetes.default.svc \
  --dest-namespace staging \
  --sync-policy automated \
  --auto-prune \
  --self-heal

```

Q46. [Advanced] What is Argo Rollouts, and how does it implement progressive delivery?

Answer:

Argo Rollouts extends Kubernetes with advanced deployment strategies — Canary and Blue/Green — with automatic analysis and rollback. It replaces the standard Kubernetes `Deployment` with a `Rollout` resource that provides fine-grained traffic control.

```
# rollout.yaml
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: my-api
  namespace: production
spec:
  replicas: 10
  selector:
    matchLabels:
      app: my-api
  template:
    metadata:
      labels:
        app: my-api
    spec:
      container:
        - name: app
          image: myregistry/my-api:1.0.0
          ports:
            - containerPort: 8080
          resources:
            requests:
              memory: "256Mi"
              cpu: "250m"
      strategy:
        canary:
```



```

canaryService: my-api-canary
stableService: my-api-stable
trafficRouting:
  nginx:
    stableIngress: my-api-ingress
steps:
  - setWeight: 5          # Send 5% of traffic to canary
  - pause: {duration: 5m} # Wait 5 minutes
  - analysis:             # Run automated analysis
    templates:
      - templateName: success-rate-analysis
    args:
      - name: service-name
        value: my-api-canary
  - setWeight: 20
  - pause: {duration: 10m}
  - setWeight: 50
  - pause: {duration: 10m}
  - setWeight: 100        # Full rollout
# Automatically abort if analysis fails
autoPromotionEnabled: false
***
# Analysis template – queries Prometheus
apiVersion: argoproj.io/v1alpha1
kind: AnalysisTemplate
metadata:
  name: success-rate-analysis
  namespace: production
spec:
  args:
    - name: service-name
  metrics:
    - name: success-rate
      interval: 1m
      successCondition: result >= 0.95 # 95% success rate required
      failureLimit: 3                  # Allow 3 failures before
      aborting

```

```

prometheus:
  address: http://prometheus:9090
  query: |
    sum(rate(http_requests_total{
      service="{{args.service-name}}",
      status!~"5.."
    }[5m]))
  /
  sum(rate(http_requests_total{
    service="{{args.service-name}}"
  }[5m]))

```

Argo Rollouts kubectl commands:

```

# Install Argo Rollouts plugin
kubectl argo rollouts version
# Watch rollout progress in real time
kubectl argo rollouts get rollout my-api -n production --watch
# Promote rollout to next step
kubectl argo rollouts promote my-api -n production
# Abort rollout (triggers automatic rollback)
kubectl argo rollouts abort my-api -n production
# Retry a failed rollout
kubectl argo rollouts retry rollout my-api -n production

```

Q47. [Intermediate] What is Vault by HashiCorp, and how does it integrate with Kubernetes for secrets management?

Answer:

HashiCorp Vault is an enterprise-grade secrets management platform. Unlike Kubernetes Secrets (which are base64-encoded and stored in etcd), Vault encrypts secrets with AES-256-GCM, provides fine-grained access control, maintains an audit log of every secret access, and can generate short-lived, dynamic credentials.

Vault Agent Injector — the sidecar pattern:

Vault's Kubernetes integration works via a sidecar injector. When a Pod has specific annotations, Vault automatically injects a sidecar container that:

1. Authenticates with Vault using the Pod's Kubernetes service account
2. Fetches the required secrets
3. Writes them to a shared volume as files or environment variables
4. Periodically renews leases for dynamic secrets

```
# deployment-with-vault.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-api
  namespace: production
spec:
  template:
    metadata:
      annotations:
        # Enable Vault Agent injection
        vault.hashicorp.com/agent-inject: "true"
        vault.hashicorp.com/role: "my-api-production"
        # Inject database credentials as a file
        vault.hashicorp.com/agent-inject-secret-db-creds: "database/creds/my-
api-role"
        vault.hashicorp.com/agent-inject-template-db-creds: |
          {{- with secret "database/creds/my-api-role" -}}
          export DB_USER="{{ .Data.username }}"
          export DB_PASSWORD="{{ .Data.password }}"
          {{- end }}
        # Inject API key
        vault.hashicorp.com/agent-inject-secret-api-key:
"secret/data/production/api"
        vault.hashicorp.com/agent-inject-template-api-key: |
          {{- with secret "secret/data/production/api" -}}
          {{ .Data.data.key }}
          {{- end }}
    spec:
```

```

serviceAccountName: my-api    # Must match the Vault role
containers:
  - name: app
    image: myregistry/my-api:1.0
    command: ["/bin/sh", "-c"]
    args:
      - |
        source /vault/secrets/db-creds
        node src/index.js

```

Configuring Vault for Kubernetes authentication:

```

# Enable Kubernetes auth method
vault auth enable kubernetes
# Configure it with the cluster's API server
vault write auth/kubernetes/config \
  kubernetes_host="https://kubernetes.default.svc" \
  kubernetes_ca_cert=@/var/run/secrets/kubernetes.io/serviceaccount/ca.crt \
  token_reviewer_jwt=@/var/run/secrets/kubernetes.io/serviceaccount/token
# Create a policy that grants access to specific secrets
vault policy write my-api-production <<EOF
path "secret/data/production/*" {
  capabilities = ["read"]
}
path "database/creds/my-api-role" {
  capabilities = ["read"]
}
EOF
# Create a role that binds the policy to a Kubernetes service account
vault write auth/kubernetes/role/my-api-production \
  bound_service_account_names=my-api \
  bound_service_account_namespaces=production \
  policies=my-api-production \
  ttl=1h
# Enable database secrets engine for dynamic credentials
vault secrets enable database
vault write database/config/my-postgres \

```

```

plugin_name=postgresql-database-plugin \
connection_url="postgresql://{{username}}:{{password}}@postgres:5432/mydb" \
allowed_roles="my-api-role" \
username="vault_admin" \
password="vault_admin_password"
vault write database/roles/my-api-role \
  db_name=my-postgres \
  creation_statements="CREATE ROLE '{{name}}' WITH LOGIN PASSWORD
'{{password}}' VALID UNTIL '{{expiration}}'; GRANT SELECT, INSERT, UPDATE ON
ALL TABLES IN SCHEMA public TO '{{name}}';" \
  default_ttl="1h" \
  max_ttl="24h"

```

Why dynamic database credentials matter: Each time an application starts, Vault creates a *new, unique* database user with a 1-hour TTL. When the application stops, the user is automatically deleted. This means: no shared passwords, no stale credentials, and every access is auditable.

Q48. [Intermediate] What is Nexus/JFrog Artifactory, and why do you need an artifact repository in CI/CD?

Answer:

An **artifact repository** is a centralized storage and management system for build artifacts — Docker images, Maven JARs, npm packages, Helm charts, Python wheels, and more.

Why you need one:

Without an artifact repository, teams typically push Docker images to Docker Hub or store JARs on S3. This works, but creates problems at scale:

- No caching of external dependencies — every build re-downloads from the internet (slow, fragile)
- No access control on who can pull/push what
- No vulnerability scanning of artifacts before they're used
- No immutability — anyone can overwrite a published version
- No single inventory of what versions are deployed where

Nexus Repository Manager — common operations:

```
# Configure npm to use Nexus as proxy
npm config set registry https://nexus.company.com/repository/npm-group/
npm config set //nexus.company.com/repository/npm-group/:_authToken
$NEXUS_TOKEN

# Configure Maven settings.xml for Nexus
cat ~/.m2/settings.xml

# <settings>
#   <mirrors>
#     <mirror>
#       <id>nexus</id>
#       <mirrorOf>*</mirrorOf>
#       <url>https://nexus.company.com/repository/maven-public/</url>
#     </mirror>
#   </mirrors>
# </settings>

# Push Docker image to Nexus
docker tag myapp:latest nexus.company.com:8082/my-app:1.0.0
docker push nexus.company.com:8082/my-app:1.0.0

# Pull Helm chart from Nexus
helm repo add company https://nexus.company.com/repository/helm-hosted/
helm install my-app company/my-app --version 1.0.0
```

In a Dockerfile — using Nexus as a PyPI proxy:

```
FROM python:3.11-slim
# Use Nexus as PyPI mirror (faster, cached, scanned for vulnerabilities)
RUN pip config set global.index-url https://nexus.company.com/repository/pypi-group/simple
RUN pip config set global.trusted-host nexus.company.com
COPY requirements.txt .
RUN pip install -r requirements.txt
```

Q49. [Advanced] How do you implement distributed tracing in a microservices architecture?

Answer:

In a microservices system, a single user request might pass through 10 different services. When something is slow or failing, **distributed tracing** tells you exactly which service, which function, and which database query is the culprit. Without it, debugging is like trying to find a needle in 10 haystacks simultaneously.

The OpenTelemetry standard (the modern approach):

OpenTelemetry (OTel) is the industry-standard for collecting traces, metrics, and logs. You instrument your application once, and can send data to Jaeger, Zipkin, Tempo, Datadog, or any compatible backend.

Instrumenting a Node.js application:

```
// tracing.js – must be loaded before anything else
const { NodeSDK } = require('@opentelemetry/sdk-node');
const { getNodeAutoInstrumentations } = require('@opentelemetry/auto-instrumentations-node');
const { OTLPTraceExporter } = require('@opentelemetry/exporter-trace-otlp-http');
const { Resource } = require('@opentelemetry/resources');
const { SemanticResourceAttributes } = require('@opentelemetry/semantic-conventions');
const sdk = new NodeSDK({
  resource: new Resource({
    [SemanticResourceAttributes.SERVICE_NAME]: 'order-service',
    [SemanticResourceAttributes.SERVICE_VERSION]: process.env.APP_VERSION || '1.0.0',
    [SemanticResourceAttributes.DEPLOYMENT_ENVIRONMENT]: process.env.NODE_ENV,
  }),
  traceExporter: new OTLPTraceExporter({
    url: 'http://otel-collector:4318/v1/traces',
  }),
  instrumentations: [
    getNodeAutoInstrumentations({
      '@opentelemetry/instrumentation-http': { enabled: true },
    })
  ]
});
```

```

    '@opentelemetry/instrumentation-express': { enabled: true },
    '@opentelemetry/instrumentation-pg': { enabled: true },
    '@opentelemetry/instrumentation-redis': { enabled: true },
  }),
],
});
sdk.start();

```

OpenTelemetry Collector configuration:

```

# otel-collector-config.yaml
receivers:
  otlp:
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317
      http:
        endpoint: 0.0.0.0:4318
processors:
  batch:
    timeout: 1s
    send_batch_size: 1024
  memory_limiter:
    check_interval: 1s
    limit_mib: 512
  resource:
    attributes:
      - key: environment
        value: production
        action: insert
exporters:
  # Send traces to Grafana Tempo
  otlp/tempo:
    endpoint: tempo:4317
    tls:
      insecure: true
  # Send metrics to Prometheus

```



```
prometheus:
  endpoint: "0.0.0.0:8889"
# Send logs to Elasticsearch
elasticsearch:
  endpoints: ["https://elasticsearch:9200"]
  index: "otel-logs"
service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [memory_limiter, batch]
      exporters: [otlp/tempo]
    metrics:
      receivers: [otlp]
      processors: [memory_limiter, batch]
      exporters: [prometheus]
    logs:
      receivers: [otlp]
      processors: [memory_limiter, batch]
      exporters: [elasticsearch]
```

Q50. [Advanced] How do you handle database migrations safely in a CI/CD pipeline?

Answer:

Database migrations are one of the highest-risk operations in a pipeline. A bad migration can corrupt data, cause downtime, or leave the database in a partially migrated state that's hard to recover from.

The golden rules for safe database migrations:

1. **Migrations must be backward-compatible** — the new code must work with both old and new schema during the deployment window
2. **Separate migration from deployment** — run migrations before deploying new code
3. **Never drop columns or tables immediately** — deprecate first, remove in a later release

4. **Test migrations against a production-size dataset** — a migration that takes 2 seconds on dev might take 2 hours on production

Using Flyway for Java/Spring applications:

```
-- V1__initial_schema.sql
CREATE TABLE users (
    id BIGSERIAL PRIMARY KEY,
    email VARCHAR(255) UNIQUE NOT NULL,
    created_at TIMESTAMP DEFAULT NOW()
);

-- V2__add_user_profile.sql
-- Safe: adding a nullable column is backward-compatible
ALTER TABLE users ADD COLUMN full_name VARCHAR(255);
ALTER TABLE users ADD COLUMN phone VARCHAR(20);

-- V3__add_index_on_email.sql
-- Safe: CONCURRENTLY means no table lock in PostgreSQL
CREATE INDEX CONCURRENTLY idx_users_email ON users(email);

-- V4__rename_column.sql (THREE-PHASE approach)
-- Phase 1 (this migration): Add new column, copy data
ALTER TABLE users ADD COLUMN username VARCHAR(255);
UPDATE users SET username = email WHERE username IS NULL;
-- Phase 2 (next release): Update app code to use 'username' instead of
'email'
-- Phase 3 (release after that): DROP COLUMN email
```

Running Flyway in a pipeline:

```
# In GitHub Actions, run migration before deploying application
- name: Run Database Migrations
  env:
    FLYWAY_URL: jdbc:postgresql://${{ secrets.DB_HOST }}:5432/mydb
    FLYWAY_USER: ${{ secrets.DB_MIGRATION_USER }}
    FLYWAY_PASSWORD: ${{ secrets.DB_MIGRATION_PASSWORD }}
  run: |
    flyway \
      -url=$FLYWAY_URL \
```

```

    -user=$FLYWAY_USER \
    -password=$FLYWAY_PASSWORD \
    -locations=filesystem:db/migrations \
    migrate
- name: Deploy Application
  # Only runs if migration succeeded
  run: kubectl set image deployment/my-app app=$IMAGE_TAG

```

Using Liquibase for multi-database support:

```

<!-- db/changelog/001-add-orders-table.xml -->
<databaseChangeLog xmlns="http://www.liquibase.org/xml/ns/dbchangelog">
  <changeSet id="001" author="jane.doe" runInTransaction="true">
    <createTable tableName="orders">
      <column name="id" type="BIGINT" autoIncrement="true">
        <constraints primaryKey="true" nullable="false"/>
      </column>
      <column name="user_id" type="BIGINT">
        <constraints nullable="false" foreignKeyName="fk_order_user"
          references="users(id)"/>
      </column>
      <column name="total_amount" type="DECIMAL(10,2)">
        <constraints nullable="false"/>
      </column>
      <column name="status" type="VARCHAR(50)" defaultValue="pending">
        <constraints nullable="false"/>
      </column>
      <column name="created_at" type="TIMESTAMP"
        defaultValueComputed="NOW()"/>
    </createTable>
    <createIndex tableName="orders" indexName="idx_orders_user_id">
      <column name="user_id"/>
    </createIndex>
  </changeSet>
  <changeSet id="002" author="jane.doe">
    <addColumn tableName="orders">
      <column name="shipping_address" type="TEXT"/>
    </addColumn>
  </changeSet>
</databaseChangeLog>

```

```
</addColumn>  
<rollback>  
    <dropColumn tableName="orders" columnName="shipping_address"/>  
</rollback>  
</changeSet>  
</databaseChangeLog>
```

Prepared by Arvind Verma

CHAPTER 13: PRACTICAL SCENARIO-BASED QUESTIONS

These questions test how you apply your knowledge in real-world situations — the kind of questions that come up in senior-level interviews.

Q51. [Advanced] Production is down. The error rate just jumped to 40%. Walk me through your incident response.

Answer:

This question tests your incident response instincts. The interviewer wants to see systematic thinking, not panic.

The OODA loop for incidents: Observe → Orient → Decide → Act

Minute 0-2: TRIAGE

- |— Check monitoring dashboard (Grafana) – what metrics changed?
 - | |— Error rate jumped at 14:32 UTC
 - | |— CPU looks normal on app servers
 - | |— Database connection errors spiking
- |— Check recent deployments – was anything deployed in the last 30 minutes?
 - | |— `kubectl rollout history deployment/my-api -n production`
- |— Declare incident severity level → notify on-call team via PagerDuty

Minute 2-5: IMMEDIATE MITIGATION

- |— If recent deployment → ROLLBACK IMMEDIATELY, ask questions later
 - | |— `kubectl rollout undo deployment/my-api -n production`
- |— If no recent deployment → investigate further
- |— Scale up pods to handle potential load issue
 - | |— `kubectl scale deployment/my-api --replicas=10 -n production`

Minute 5-15: INVESTIGATION (if not resolved by rollback)

- |— Check application logs in Kibana
 - | |— Filter: `kubernetes.namespace: production AND app.level: ERROR AND`

- └─ Check database metrics
 - | └─ RDS CloudWatch: DatabaseConnections, ReadLatency, WriteLatency
 - | └─ Look for connection pool exhaustion
- └─ Check downstream dependencies
 - | └─ Are third-party API calls failing?
 - (payment gateway, email service)
- └─ Check Prometheus for cascading failures
 - | └─ Which microservices are reporting high error rates?
- └─ Check AWS Service Health Dashboard – any regional outages?

Minute 15+: RESOLUTION AND COMMUNICATION

- └─ Apply fix (targeted, minimal change)
- └─ Verify error rate returning to normal in Grafana
- └─ Update status page (statuspage.io) for customer visibility
 - └─ Write incident timeline in shared doc for postmortem

After Incident: POSTMORTEM (within 48 hours)

- └─ Timeline of events
- └─ Root cause analysis (5 Whys)
- └─ What worked well in the response?
- └─ What could be improved?
- └─ Action items with owners and deadlines

Key diagnostic commands during an incident:

```
# Quick cluster health check
kubectl get nodes
kubectl get pods -n production | grep -v Running
# Check recent events
kubectl get events -n production --sort-by='.lastTimestamp' | tail -20
# Check pod restart counts (OOMKills, CrashLoopBackOff)
kubectl get pods -n production -o wide
# Describe a crashing pod
kubectl describe pod my-api-abc123 -n production
# Get logs from the last 5 minutes
kubectl logs -n production -l app=my-api --since=5m
# Check HPA scaling events
kubectl describe hpa my-api -n production
# Quick database connectivity test from within cluster
```

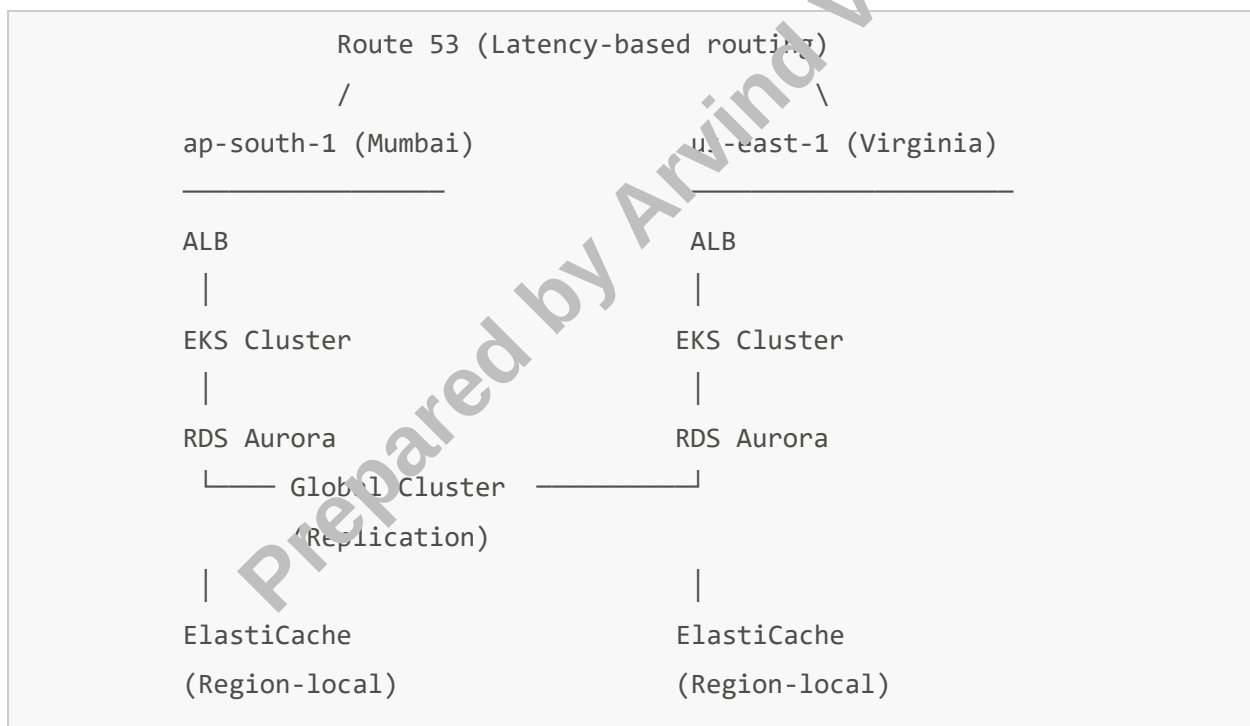
```
kubectl run db-test --image=postgres:14 --restart=Never --rm -it -- \
  psql -h mydb.cluster.local -U app -c "SELECT 1"
```

Q52. [Advanced] How do you design a multi-region active-active deployment on AWS?

Answer:

Multi-region active-active means users in every region are served by their local infrastructure simultaneously — it's not failover, it's parallel serving. This is the highest complexity in cloud architecture.

Architecture overview:



Key components:

1. Route 53 Latency-Based Routing:

```
resource "aws_route53_record" "api" {
  zone_id = var.hosted_zone_id
  name    = "api.myapp.com"
```

```

type      = "A"
# Mumbai record
set_identifier = "ap-south-1"
latency_routing_policy {
    region = "ap-south-1"
}
alias {
    name                = aws_alb.mumbai.dns_name
    zone_id             = aws_alb.mumbai.zone_id
    evaluate_target_health = true
}
}
resource "aws_route53_record" "api_us" {
    zone_id = var.hosted_zone_id
    name     = "api.myapp.com"
    type     = "A"
    set_identifier = "us-east-1"
    latency_routing_policy {
        region = "us-east-1"
    }
    alias {
        name                = aws_alb.virginia.dns_name
        zone_id             = aws_alb.virginia.zone_id
        evaluate_target_health = true
    }
}

```

2. Aurora Global Database:

```

resource "aws_rds_global_cluster" "main" {
    global_cluster_identifier = "my-app-global"
    engine                   = "aurora-postgresql"
    engine_version           = "15.4"
    database_name            = "myapp"
    storage_encrypted        = true
}
# Primary cluster (writes go here)

```



```

resource "aws_rds_cluster" "primary" {
  provider                = aws.ap_south_1
  cluster_identifier      = "my-app-primary"
  engine                  = "aurora-postgresql"
  global_cluster_identifier = aws_rds_global_cluster.main.id
  master_username         = "admin"
  manage_master_user_password = true    # Secrets Manager managed
}

# Secondary cluster (reads served locally, < 1 second lag)
resource "aws_rds_cluster" "secondary" {
  provider                = aws.us_east_1
  cluster_identifier      = "my-app-secondary"
  engine                  = "aurora-postgresql"
  global_cluster_identifier = aws_rds_global_cluster.main.id
  # No master credentials – it replicates from primary
}

```

3. Application design considerations for active-active:

- **Write routing:** All writes must go to the primary region (or use conflict-free data types — CRDTs)
- **Session affinity:** User session id must be stored in a shared store (DynamoDB Global Tables, not local Redis) or be stateless (JWT tokens)
- **Idempotency:** All write operations must be idempotent — re-processing a request due to network retry should not cause duplicate data
- **Eventual consistency:** Accept that reads in the secondary region might be slightly behind the primary (typically < 1 second with Aurora Global)

Q53. [Advanced] How do you implement cost optimization in a Kubernetes cluster?

Answer:

Running Kubernetes in production can be expensive if not managed carefully. Here are the key cost optimization strategies with concrete implementations:

1. Cluster Autoscaler — right-size your node pool:

```
# cluster-autoscaler deployment on EKS
apiVersion: apps/v1
kind: Deployment
metadata:
  name: cluster-autoscaler
  namespace: kube-system
spec:
  template:
    spec:
      containers:
        - name: cluster-autoscaler
          image: registry.k8s.io/autoscaling/cluster-autoscaler:v1.28.0
          command:
            - ./cluster-autoscaler
            - --v=4
            - --stderrthreshold=info
            - --cloud-provider=aws
            - --skip-nodes-with-local-storage=false
            - --expander=least-waste # Choose node type that wastes least
resources
            - --node-group-auto-discovery=asg:tag=k8s.io/cluster-
autoscaler/enabled,k8s.io/cluster-autoscaler/my-cluster
            - --scale-down-delay-after-add=5m
            - --scale-down-unnneeded-time=10m # Remove idle nodes after 10
minutes
            - --scale-down-utilization-threshold=0.5 # Node < 50% utilized =
candidate for removal
```

2. Spot/Preemptible instances for non-critical workloads:

```
# Use node affinity to schedule batch jobs on spot instances
spec:
  affinity:
    nodeAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
```

```

    - weight: 100
      preference:
        matchExpressions:
          - key: eks.amazonaws.com/capacityType
            operator: In
            values: ["SPOT"]
  tolerations:
    - key: "spot"
      operator: "Equal"
      value: "true"
      effect: "NoSchedule"

```

3. Vertical Pod Autoscaler (VPA) — right-size your Pod requests:

```

apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: my-api-vpa
  namespace: production
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: my-api
  updatePolicy:
    updateMode: "Off" # Just recommend, don't auto-apply in production
  resourcePolicy:
    containerPolicies:
      - containerName: app
        minAllowed:
          cpu: 100m
          memory: 128Mi
        maxAllowed:
          cpu: 2000m
          memory: 2Gi
# View VPA recommendations
kubectl describe vpa my-api-vpa -n production

```

```
# Output:
# Recommendation:
#   Container Recommendations:
#     Container Name: app
#     Lower Bound: cpu: 150m, memory: 200Mi
#     Target:      cpu: 280m, memory: 380Mi ← Use these values in your
Deployment
#     Upper Bound: cpu: 800m, memory: 1Gi
```

4. Kubecost — visibility into per-namespace/per-team costs:

```
# Install Kubecost
helm repo add cost-analyzer https://kubecost.github.io/cost-analyzer/
helm install kubecost cost-analyzer/cost-analyzer \
  --namespace kubecost \
  --create-namespace \
  --set kubecostToken="your-token" \
  --set prometheus.enabled=false \
  --set prometheus.fqdn=http://prometheus-monitoring.svc.cluster.local:9090
# Query Kubecost API for namespace costs
curl
http://kubecost.company.com/model/allocation?window=7d&aggregate=namespace
```

FINAL CHAPTER: KEY INTERVIEW TIPS

How to Present Answers Professionally

After all the technical knowledge, how you communicate it matters enormously. Here are behaviors that separate great candidates from good ones:

Structure your answers using STAR or situation-action-result:

- *"In our last production environment, we had [SITUATION]. I designed [ACTION]. The result was [RESULT — specific numbers where possible]."*

Show depth by mentioning tradeoffs:

- Every tool has weaknesses. Saying "Kubernetes is the best solution for everything" signals inexperience. Saying "We chose Kubernetes because we needed fine-grained scaling, though we accepted the operational overhead of managing the control plane and initially invested in team training" signals maturity.

Use real numbers whenever possible:

- "We reduced build time from 18 minutes to 4 minutes by implementing build caching and parallel test execution."
- "After implementing HPA, we reduced our EC2 costs by 35% during off-peak hours."

Connect tools to business outcomes:

- Don't just say "we implemented Prometheus." Say "we implemented Prometheus alerting which reduced our MTTR (Mean Time to Recovery) from 45 minutes to 8 minutes because engineers got alerted before customers noticed."

Key Takeaways — Complete Guide

Jenkins: Orchestrator, not a builder. Pipeline-as-Code in Jenkinsfile. Dynamic Kubernetes agents

Docker: Multi-stage builds reduce image size dramatically. Layer cache order matters. Never run as root.

Git/GitHub/GitLab: Branching strategy should match release cadence. GitLab CI/CD is batteries-included; Jenkins is more flexible.

AWS DevOps: CodePipeline + CodeBuild + CodeDeploy for native AWS CI/CD. CDK for IaC with programmatic control. Blue/Green on ECS for zero-downtime.

GitHub Actions: OIDC for AWS authentication eliminates long-lived credentials. Reusable workflows scale across many repos.

Kubernetes: Pods → Deployments → Services → Ingress. Resource requests/limits are non-negotiable. HPA with stabilization windows prevents yo-yo scaling.

Prometheus & Grafana: Pull model. Use `rate()` for counters. RED method for dashboards. `for: 2m` prevents flapping alerts.

ELK Stack: Log in JSON from your applications. elasticsearch DaemonSet in Kubernetes. ILM for index lifecycle cost management.

Ansible: Idempotent by design. Ansible for config management + Terraform for infra provisioning. Vault for encrypted secrets.

Terraform: Remote state always. Modules for DRY code. Separate plan/apply IAM roles. `prevent_destroy` on databases.

Azure DevOps: Multi-stage YAML pipelines. Environment gates for production approvals. Service connections for cloud authentication.

GitOps (Argo CD): Git is the single source of truth. Auto-sync with self-heal eliminates config drift. Separate app repo and GitOps repo.

Common Pitfalls Across All Tools

- **Not having a rollback plan** — every deployment must have a rollback procedure documented before deployment starts
- **Treating security as an afterthought** — SAST, image scanning, and secret management must be part of the pipeline, not added later
- **Not testing the pipeline itself** — pipelines need maintenance; they break when tools are upgraded or APIs change
- **Alert fatigue** — too many low-quality alerts train engineers to ignore all alerts, including critical ones
- **Manual snowflake servers** — any server that was configured by hand and cannot be reproduced from code is a liability
- **No resource limits in Kubernetes** — a single pod can consume all cluster resources and starve everything else
- **Committing secrets to Git** — even if you immediately remove them, they exist in history forever

Operational Lessons

- **Start simple, add complexity only when pain justifies it** — you don't need Argo CD, Vault, and Istio on day one
- **Observability before optimization** — you cannot optimize what you cannot measure
- **The pipeline is production code** — treat your Jenkinsfiles, Terraform, and GitHub Actions workflows with the same engineering rigor as application code
- **Documentation is part of the pipeline** — auto-generate architecture diagrams, runbooks should link to dashboards, every alert should have a runbook URL
- **Chaos engineering** — regularly test failure scenarios (kill a pod, fail a node, cut network between services) to build confidence in your resilience mechanisms; tools like Chaos Monkey and Litmus Chaos make this automated

This question bank covers all the major tools and scenarios you'll encounter in DevOps interviews at all levels — from junior to staff engineer. The answers are designed to show practical experience, architectural thinking, and awareness of real-world tradeoffs rather than just theoretical knowledge.

APPENDIX: Common Mistakes to Avoid

During DevOps Interviews

PART 1: TECHNICAL KNOWLEDGE MISTAKES

1. Skipping Linux & Networking Fundamentals

- Jumping directly to tools like Kubernetes, Docker, or Terraform without mastering Linux basics
- Unable to explain file permissions, process management, or how to debug a Linux system
- Not knowing commands like `grep`, `awk`, `sed`, `find`, `netstat`, `ss`, `lsof`, `strace`
- Failing to explain how DNS resolution works, what a subnet mask is, or how TCP/IP handshake works
- Cannot explain the difference between hard link and soft link, or what `/proc` filesystem contains
- Not understanding `systemd`, `journalctl`, and service management — basic operational knowledge
- **Fix:** Always start your preparation with Linux. Every DevOps tool runs on Linux. If you can't troubleshoot a server, you are not ready for DevOps interviews

2. Tool Name Dropping Without Depth

- Listing 15 tools on a resume but being unable to explain *why* you chose one tool over another
- Saying "I worked with Jenkins" but unable to explain stages, rollback strategies, or parallel execution
- Knowing the tool name but not knowing its limitations, failure modes, or alternatives
- Memorizing Kubernetes YAML syntax but being unable to debug a `CrashLoopBackOff` or `OOMKilled` pod
- Cannot explain the tradeoff between ECS vs EKS, Ansible vs Terraform, Prometheus vs Datadog

- Claiming expertise in a tool you've only read about — interviewers probe deeply with follow-up questions
- Saying "I know Terraform" but unable to explain what happens when state drift occurs
- **Fix:** For every tool on your resume, prepare: what problem it solves, why you chose it, a real example of using it, and what its limitations are

3. Answering "What" Without Answering "Why"

- Saying "I would use EKS" without explaining *why* EKS over ECS, Fargate, or a VM-based approach
- Describing *what* a tool does but not *why* your team chose it for your specific context
- Not connecting technical decisions to business outcomes (cost reduction, reliability improvement, faster delivery)
- Giving textbook definitions instead of explaining real-world usage and trade-offs
- **Fix:** For every answer, follow the pattern: "We used X because [problem], which resulted in [outcome — use numbers]"

4. Weak CI/CD Pipeline Knowledge

- Claiming CI/CD experience but unable to draw a complete pipeline from commit to production
- Not knowing how to handle pipeline failures, rollback strategies, or artifact versioning
- Cannot explain the difference between Continuous Integration, Continuous Delivery, and Continuous Deployment
- Not knowing what happens when a build fails halfway — how do you recover?
- how do you notify?
- Unable to explain how you'd implement a canary or blue/green deployment in a pipeline
- Not understanding branch protection rules, quality gates, or approval workflows
- Cannot explain how secrets are injected into pipelines without hardcoding
- **Fix:** Build at least 2-3 real pipelines from scratch (one with Jenkins, one with GitHub Actions). Be able to whiteboard the entire flow end-to-end

5. Shallow Kubernetes Knowledge

- Memorizing Kubernetes terms without understanding the underlying mechanisms
- Confusing Pods, Deployments, ReplicaSets, and StatefulSets — when to use each
- Unable to explain how a Service routes traffic to Pods (label selectors, endpoints)
- Not knowing how the Kubernetes scheduler makes decisions (resource requests, affinity, taints)
- Cannot debug common issues: **Pending** pod (resource constraints), **ImagePullBackOff** (registry issues), **OOMKilled** (memory limits)
- Not knowing the difference between resource requests and limits — and why they both matter
- Cannot explain what happens during a rolling update at each step
- Unaware of security practices: non-root containers, read-only filesystems, network policies, PSA
- **Fix:** Run a real Kubernetes cluster (minikube, kind, or EKS). Break things deliberately and fix them. The ability to debug live issues is what separates strong candidates

6. Ignoring Security (DevSecOps)

- Having no knowledge of secrets management — storing secrets in environment variables, code, or Docker images
- Not knowing what SAST, DAST, or SCA tools are, or how to integrate them in CI
- Unable to explain IAM least privilege — giving **AdministratorAccess** to Lambda functions
- Not knowing how to scan Docker images for vulnerabilities (Trivy, Snyk, ECR scanning)
- Cannot explain how to prevent secrets from leaking in Git history
- No knowledge of Kubernetes RBAC — how to restrict what a ServiceAccount can do
- Not aware of network policies — treating all pods as freely communicating
- **Fix:** Modern DevOps interviews *always* include security questions. Prepare: image scanning, secrets management (Vault/Secrets Manager), SAST tools, RBAC, and IAM policies

7. Poor Infrastructure as Code (IaC) Depth

- Cannot write a basic Terraform configuration from memory — only knows concepts

- Not understanding Terraform state — what it is, why it matters, and what happens when it's corrupted
- Unable to explain the `terraform plan` → `apply` workflow and what could go wrong
- Not knowing the difference between `count` and `for_each` and when each breaks
- Cannot explain Terraform module structure or why modularizing is important
- No understanding of drift detection or how to handle manually changed resources
- Not knowing how remote state locking prevents race conditions in team environments
- **Fix:** Write Terraform code daily. Practice creating VPCs, ECS clusters, RDS instances, and IAM roles from scratch. Run `plan` and `apply` against a real AWS account

8. Weak Monitoring & Observability Answers

- Unable to differentiate between monitoring and observability
- Saying "I use CloudWatch" without knowing how to write meaningful alarms or log insights queries
- Not knowing the difference between metrics, logs, and traces — and when to use each
- Cannot explain the RED method or USE method for diagnosing performance issues
- Not knowing what an SLI, SLO, or error budget is — critical for senior roles
- Cannot explain how to reduce alert fatigue or design actionable alerts
- No knowledge of PromQL — cannot write `rate()`, `irate()`, or `histogram_quantile()` expressions
- **Fix:** Set up a full observability stack (Prometheus + Grafana + Loki) locally. Create real dashboards with RED method metrics and write alerts with proper `for:` duration

PART 2: COMMUNICATION & BEHAVIORAL MISTAKES

9. Giving Vague, Theoretical Answers

- Answering "How would you handle a production outage?" with generic steps instead of a real story
- Not using specific numbers — "it was slow" instead of "P99 latency jumped from 80ms to 4 seconds"
- Describing projects without explaining *your specific contribution* vs what the team did

- Saying "we improved performance" instead of "we reduced build time from 18 minutes to 4 minutes by implementing layer caching"
- **Fix:** For every technical area, prepare at least one STAR story (Situation, Task, Action, Result) with real numbers, tools used, and what you personally did

10. Claiming to Work Alone (The "I" Trap)

- DevOps is fundamentally collaborative — presenting yourself as a lone hero raises red flags
- Never mentioning how you worked with developers, QA, security, or business teams
- Not explaining how you communicated about infrastructure changes or downtime
- Missing examples of knowledge sharing, writing runbooks, or mentoring junior engineers
- Not highlighting how you navigated disagreements or constraints (e.g., developer pushback on security policies)
- **Fix:** Prepare 2-3 stories that explicitly demonstrate cross-functional collaboration. Show how you communicated technical decisions to non-technical stakeholders

11. Exaggerating Experience

- Claiming "expert" in Terraform when you've only written basic EC2 resources — interviewers probe deeply
- Saying "I managed a 500-node Kubernetes cluster" when you can't explain how etcd backup works
- Listing tool as skills when you've only watched a tutorial or read documentation
- Overstating your role in projects — if you contributed to a pipeline, say what specifically you did
- **Fix:** Be precise about your experience level. "I've used Terraform for 2 years in production — we managed 200+ resources across 3 AWS accounts" is far better than vague claims. Saying "I'm actively learning Kubernetes and have run workloads in minikube" is honest and respected

12. Not Admitting You Don't Know Something

- Attempting to bluff through questions you don't know — experienced interviewers detect this immediately
- Making up tool names, features, or behaviors rather than admitting unfamiliarity
- Guessing at answers to security or networking questions and getting them wrong
- Not following up with "but here's how I would find out" or "here's what I'd do to learn it"
- **Fix:** Say: *"I haven't worked with that specific tool, but I'm familiar with the concept — my experience with [similar tool] would transfer. Here's how I'd approach learning it."* Honesty + resourcefulness is respected more than fake confidence

13. Failing Scenario-Based Questions

- Giving a textbook answer to "your pod is in CrashLoopBackOff" without walking through your actual debugging process
- Not thinking out loud during troubleshooting questions — interviewers want to see your reasoning
- Jumping to the solution immediately without diagnosing the root cause
- Forgetting to mention rollback options, customer communication, and postmortem in incident scenarios
- Not considering failure modes — only describing happy paths
- **Fix:** Practice the OODA loop for incident scenarios: Observe → Orient → Decide → Act. Always start with "First I'd check X metric to confirm Y hypothesis, then..."

14. Poor Communication of Complex Concepts

- Over-explaining with jargon when a clear, simple explanation would be better
- Under-explaining assuming the interviewer knows everything — read the room
- Not using analogies or examples to explain complex topics (e.g., explaining Kubernetes scheduling without an example)
- Speaking too fast when nervous — the interviewer needs time to absorb technical details
- Not checking in with "Does that make sense?" or "Would you like me to go deeper on any part?"
- **Fix:** Practice explaining Kubernetes, Terraform, and CI/CD concepts as if explaining to a junior developer. Clarity of explanation signals depth of understanding

PART 3: PREPARATION MISTAKES

15. Preparing Only for "What" Questions, Not "How" & "Why"

- Memorizing definitions ("Kubernetes is a container orchestration platform") without knowing real-world application
- Not preparing for follow-up questions — every answer opens 3 more questions
- Studying tool documentation but never building anything with the tool
- Preparing answers for common questions but freezing on uncommon variations
- **Fix:** For each topic, prepare at three levels: (1) What is it?
- (2) How does it work internally?
- (3) When would you use/not use it, and what can go wrong?

16. Not Researching the Company

- Not knowing what cloud platform the company uses (AWS vs Azure vs GCP)
- Not reading the job description carefully — missing that they use GitLab CI while you prepared only Jenkins
- Not knowing whether they run Kubernetes, ECS, or bare-metal — so examples are irrelevant
- Not understanding the product/domain — unable to ask intelligent questions at the end
- **Fix:** Research the company's tech stack via their engineering blog, LinkedIn, GitHub, and job postings. Tailor your answers to their environment

17. Neglecting to Prepare Questions to Ask

- Saying "I have no questions" at the end — signals low engagement and preparation
- Asking questions that are answered on the company's public website — signals lack of research
- Only asking about salary and benefits before showing technical enthusiasm
- Not asking about their biggest operational challenges, on-call culture, or deployment frequency
- **Good questions to ask:**

- "What does your deployment pipeline look like today, and what's the biggest bottleneck?"
- "How do you handle on-call rotations and postmortems?"
- "What would success look like for this role in the first 90 days?"
- "What's your current observability stack and what gaps are you trying to fill?"

18. Resume-Related Mistakes That Affect Interviews

- Listing tools you can't answer questions about — your resume sets the interview agenda
- Vague project descriptions: "Worked on CI/CD pipelines" vs. "Built a Jenkins pipeline reducing deployment time from 45 to 8 minutes for a 20-service microservices system"
- Not including scale/numbers — "managed Kubernetes cluster" vs. "managed 3 EKS clusters with 200+ nodes across 2 AWS regions"
- Listing certifications you can't back up with practical knowledge
- **Fix:** Every bullet on your resume should be a conversation you're prepared to have for 10 minutes. If you can't, remove it or study it thoroughly

PART 4: MINDSET & ATTITUDE MISTAKES

19. Treating DevOps as Just a Collection of Tools

- Failing to explain DevOps as a cultural and philosophical shift, not just a toolset
- Saying "DevOps is Docker + Kubernetes + Jenkins" — this is a major red flag for senior interviewers
- Not understanding that DevOps is about breaking silos between Dev and Ops, shared ownership, and faster feedback loops
- Cannot articulate the four key metrics of DORA (Deployment Frequency, Lead Time, MTTR, Change Failure Rate)
- **Fix:** Be able to articulate: "DevOps is about shortening the feedback loop between writing code and delivering value to users, with shared responsibility for reliability and quality." Then demonstrate it with examples

20. Showing No Curiosity or Learning Mindset

- Not mentioning any recent learning — new AWS services, tools, or industry trends
- Unable to discuss what's changed in the past year (e.g., Karpenter replacing Cluster Autoscaler, AWS deprecating CodeCommit)
- No opinions on current debates (GitOps vs traditional CD, service mesh adoption, FinOps)
- Not acknowledging AI/LLM impact on DevOps (Agentic AI, AIOps, code generation tools)
- **Fix:** Follow AWS blogs, CNCF announcements, and engineering blogs (Netflix, Uber, Shopify). Mention something recent: "I was just reading about..." shows genuine passion

21. Being Defensive About Failures or Gaps

- Defensive or dismissive when asked about a production incident or mistake — "that wasn't my fault"
- Not having a clear story about a failure you caused and what you learned from it
- Refusing to admit gaps in knowledge rather than framing them as learning opportunities
- Reacting poorly to challenging technical pushback — interviewers sometimes intentionally challenge your answer to test composure
- **Fix:** Prepare a "failure story" using this format: *"I caused [incident]. Here's what happened, why I made that decision, and the three things I changed permanently after that."* This demonstrates maturity

22. Not Demonstrating Business Impact Awareness

- Talking only about technical metrics without connecting to business outcomes
- Not knowing what downtime costs the business — unavailable systems = revenue loss
- Cannot explain how your work reduced costs, improved reliability, or accelerated delivery
- Presenting automation projects without quantifying the time saved
- **Fix:** Attach business value to every technical story: *"By implementing auto-scaling, we reduced our EC2 costs by 35% during off-peak hours, saving approximately \$8,000/month."* Numbers are memorable

PART 5: INTERVIEW-DAY EXECUTION MISTAKES

23. Answering Too Fast Without Thinking

- Immediately jumping to an answer without taking time to understand the full question
- Not asking clarifying questions when a scenario question is ambiguous
- Providing the wrong answer confidently because you didn't pause to think
- **Fix:** It is completely acceptable — and respected — to say: *"That's a good question. Give me 30 seconds to think through this properly."* Interviewers prefer a thoughtful answer over a fast wrong one

24. Not Structuring Answers

- Rambling for 5 minutes without a clear beginning, middle, and end
- Mixing multiple topics in one answer without clear separation
- Not using the STAR format for behavioral questions
- Going too deep on one detail while skipping the broader answer
- **Fix:** Start with a one-sentence summary answer, then expand with details. *"My approach would be X. Here's why: [details]. In practice, I did this at [company] when [story]."*

25. Forgetting to Mention Monitoring, Rollback, and Security in Every Scenario

- Answering "how would you deploy this?" without mentioning health checks, rollback plan, and monitoring
- Describing a Kubernetes setup without mentioning RBAC, NetworkPolicies, or resource limits
- Proposing a CI/CD pipeline without mentioning secret scanning, SAST, or image scanning
- **Fix:** Develop a mental checklist for every solution you propose: ☒ How do you monitor it?
- ☒ How do you roll back if it fails?
- ☒ How is it secured?
- ☒ What's the cost impact?

26. Poor Whiteboard / Architecture Drawing

- Jumping into drawing without first clarifying requirements
- Drawing only the happy path — no failure domains, redundancy, or cross-zone design

- Not labeling components clearly or explaining what you're drawing as you draw
- Drawing a beautiful architecture but unable to explain the network traffic flow through it
- Not considering scalability — design works for 100 users but breaks at 1 million
- **Fix:** Before drawing, always say: *"Before I start, let me clarify the requirements — expected traffic, availability SLA, budget constraints, and compliance needs."* Then draw top-down (user → DNS → LB → app → DB), label everything, and narrate as you draw.

27. Giving Only One Solution to Every Problem

- Proposing a single solution without acknowledging that alternatives exist
- Not explaining under what circumstances you'd choose a different approach
- Appearing inflexible — senior DevOps engineers know that "it depends" is often the correct answer
- Not considering constraints: team size, budget, existing tooling, skill set
- **Fix:** Always say: *"For this scenario, I'd choose X. However, if the team is smaller or budget is constrained, Y would also work because..."* Showing a range of thinking signals seniority

28. Not Handling Stress Questions Well

- Freezing when asked a question you've never seen before
- Showing visible frustration or anxiety when challenged on a technical answer
- Saying "I don't know" and going silent — with no attempt to reason through the answer
- **Fix:** When you don't know the exact answer, reason out loud: *"I haven't done this specific thing, but based on how [related concept] works, I'd hypothesize that... and here's how I'd verify it."* Reasoning under uncertainty is a core DevOps skill

29. Underestimating Culture-Fit Questions

- Dismissing soft questions like "tell me about a time you disagreed with a teammate" as unimportant
- Not preparing behavioral answers (STAR format) with the same rigor as technical answers
- Giving generic answers: "I'm a team player and I work hard" — meaningless without examples

- Failing to show empathy for the on-call experience, developer frustration, or business pressure
- **Fix:** Prepare 5-6 STAR stories covering: conflict resolution, handling failure, improving a process, mentoring someone, and advocating for a technical decision

30. Closing the Interview Poorly

- Not thanking the interviewer or showing genuine interest in the role at the end
- Forgetting to ask about next steps in the process
- Not sending a follow-up email within 24 hours — a brief thank-you that references one specific topic from the interview
- Not reflecting on the interview to identify gaps for future improvement
- **Fix:** End every interview with: *"Thank you — I genuinely enjoyed this conversation, especially the discussion about [specific topic]. I'm very interested in the role. What are the next steps?"*

PART 6: EXPERIENCE-LEVEL SPECIFIC MISTAKES

For Junior/Entry-Level Candidates:

- Claiming senior-level experience on a resume when you have 1-2 years of learning
- Not showing a portfolio — no GitHub repos, no personal projects, no blogs
- Expecting to be judged only on theoretical knowledge without hands-on proof
- Not demonstrating eagerness to learn — juniors are hired for potential, not perfection
- **Fix:** Build a GitHub portfolio: a Terraform project that deploys a full 3-tier application, a Jenkins or GitHub Actions pipeline, and a Kubernetes deployment with Helm. These speak louder than certifications

For Mid-Level Candidates (3-5 years):

- Not demonstrating ownership — you've used the tools but have you designed systems with them?
- Cannot explain decisions: "my senior architect chose that, I just implemented it"
- Weak incident response stories — never been the primary responder

- No examples of improving existing systems (cost reduction, performance, reliability improvements)
- **Fix:** For mid-level, the key expectation is *ownership*. Prepare stories where *you* identified a problem, proposed the solution, implemented it, and measured the result

For Senior-Level Candidates (5+ years):

- Cannot articulate platform engineering or developer experience improvements
- No opinion on architectural trade-offs (monorepo vs polyrepo, service mesh necessity, GitOps maturity)
- Weak on DORA metrics, SLOs, and error budget management
- Cannot discuss organizational impact — how you influenced other teams, established standards, or created shared tooling
- No experience with FinOps or cost governance at scale
- Not able to discuss multi-region, multi-account AWS architecture
- **Fix:** Senior candidates are judged on *influence and judgment*, not just execution. Prepare stories about platform strategy, architectural decisions, and engineering standards you set for the wider organization

QUICK REFERENCE: PRE-INTERVIEW CHECKLIST

One week before:

- Review every tool listed on your resume — be ready for 10 minutes of deep questioning on each
- Prepare 6 STAR stories covering: success, failure, conflict, leadership, learning, and business impact
- Research the company's tech stack, engineering blog, and recent DevOps job postings
- Practice whiteboarding a 3-tier AWS architecture from memory with HA and security considerations
- Practice explaining 5 core concepts to a non-technical person (CI/CD, containers, IaC, monitoring, GitOps)

One day before:

- Review Linux fundamentals, networking basics, and Kubernetes debugging commands
- Prepare 3-4 meaningful questions to ask the interviewer
- Test your screen share (for virtual interviews) and have your IDE/terminal ready
- Sleep well — cognitive performance drops 20% when sleep-deprived

Day of the interview:

- Join 5-10 minutes early
- Have a notepad ready to jot down scenario details
- For every question: pause → clarify if needed → structure your answer → answer → check in
- When stuck: think out loud, use analogies, connect to known concepts
- End with a strong closing question and express genuine interest

THE GOLDEN RULES — Summary

1. **Never fake depth** — shallow knowledge is discovered in 2 follow-up questions
2. **Always connect to business impact** — every tool exists to solve a business problem
3. **Show your reasoning, not just your answer** — interviewers hire for judgment
4. **Admit gaps honestly** — pair with "here's how I'd close that gap"
5. **Prepare failure stories** — they reveal character and learning ability
6. **Use numbers always** — "faster" means nothing; "reduced from 18 minutes to 4 minutes" is memorable
7. **Research the company deeply** — tailor every answer to their stack and scale
8. **DevOps is culture first, tools second** — always anchor technical answers to collaboration and delivery outcomes
9. **Think in systems, not components** — show how pieces connect, fail, and recover together
10. **Close every solution with monitoring + rollback + security** — these three are non-negotiable in production thinking

BONUS: PHRASES THAT HURT VS PHRASES THAT HELP

✗ Avoid Saying This	☑ Say This Instead
"I don't know" (<i>and go silent</i>)	"I haven't done that specifically, but here's how I'd reason through it..."
"We used Jenkins"	"We chose Jenkins because our team needed X flexibility — it gave us Y result"
"I'm an expert in everything"	"I'm strongest in AWS and Terraform — I have working knowledge of GCP"
"That was my team's work"	"I led the design; my teammates implemented the frontend integration"
"It depends" (<i>no follow-up</i>)	"It depends on X and Y — if X, I'd choose A; if Y, I'd choose B"
"I've never had a production incident"	"I've been fortunate, but I've participated in incident reviews and here's what I learned..."
"I just followed the documentation"	"I started with the docs, then adapted for our constraints — here's what I changed and why"
"Kubernetes is better than Docker Swarm"	"For our scale and team's Kubernetes expertise, EKS gave us better autoscaling and ecosystem support"
"I did everything myself"	"I drove the initiative and collaborated with the security team and developers to implement it"
"I can learn it quickly" (<i>no evidence</i>)	"I learned Terraform in 3 weeks and deployed production infrastructure — here's the repo"

BONUS: TOPICS MOST COMMONLY UNDER-PREPARED

Based on real interview feedback, these topics are consistently weak spots that candidates overlook:

1. **Networking fundamentals** — subnets, CIDR, VPC routing, security groups vs NACLs
2. **Terraform state management** — locking, remote backends, drift, corruption recovery
3. **Kubernetes resource management** — requests vs limits, QoS classes, eviction behavior
4. **Security in pipelines** — secrets injection, image scanning, SAST/DAST integration

5. **Incident response process** — structured severity framework, communication, postmortem
6. **Monitoring design** — not just "I use Prometheus" but designing meaningful SLO-based alerting
7. **Database operations in DevOps** — migrations, connection pooling, backup strategies in Kubernetes
8. **Cost optimization** — FinOps principles, rightsizing, spot instances, reserved capacity
9. **GitOps principles** — not just Argo CD installation but the philosophy and reconciliation loop
10. **Linux performance debugging** — `strace`, `perf`, `vmstat`, `iostat`, `dmesg` for real troubleshooting

FINAL THOUGHT

"The best DevOps engineers are not the ones who know the most tools — they are the ones who can reason clearly about complex systems, communicate that reasoning to others, and learn new tools in days because they understand the underlying principles."

Prepare with depth, practice with real systems, speak with honesty, and demonstrate that you think in systems — not just in commands. That combination wins interviews at every level.